

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BTL LẦN 3
CÔNG NGHỆ PHẦN MỀM (CO3001)

Lớp: A01 - Nhóm 1
GVHD: Mai Đức Trung

STT	Họ và tên	MSSV	Tên lớp	Tên ngành
1	Nguyễn Văn Hùng	2311301	A01	Kỹ thuật máy tính
2	Trần Minh Trí	2313626	A01	Kỹ thuật máy tính
3	Nguyễn Lưu Khánh Trình	2313638	A01	Kỹ thuật máy tính
4	Nguyễn Tô Quốc Việt	2313898	A01	Kỹ thuật máy tính
5	Lê Công Vinh	2313912	A01	Kỹ thuật máy tính

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 4 NĂM 2026



BẢNG PHÂN CHIA CÔNG VIỆC

STT	Họ và tên	MSSV	Đóng góp	Công việc cá nhân
1	Nguyễn Văn Hùng	2311301	100%	Scenario cho nhóm thanh toán và các diagram tương ứng
2	Trần Minh Trí	2313626	100%	Scenario cho nhóm tiện ích bãi xe và các diagram tương ứng
3	Nguyễn Lưu Khánh Trình	2313638	100%	Scenario cho nhóm các chức năng quản lý tài khoản và các diagram tương ứng
4	Nguyễn Tô Quốc Việt	2313898	100%	Scenario cho nhóm Admin và các diagram tương ứng
5	Lê Công Vinh	2313912	100%	Scenario cho nhóm IoT và các diagram tương ứng

Mục lục

Danh sách hình vẽ	5
1 Tổng quan dự án	7
1.1 Bối cảnh dự án	7
1.2 Mục tiêu dự án	7
1.3 Stakeholders	7
1.4 Phạm vi dự án	8
1.5 Quy trình nghiệp vụ cốt lõi	8
2 Tổng quan chức năng và ranh giới hệ thống	10
2.1 Tổng quan chức năng	10
2.1.1 Nhóm chức năng hướng người dùng	11
2.1.2 Nhóm chức năng hướng vận hành	11
3 Ranh giới hệ thống	12
3.0.1 Bên trong ranh giới hệ thống	12
3.0.2 Bên ngoài ranh giới hệ thống	12
4 Các non-interactive functional requirement	12
5 Các non-functional requirement	13
6 Screenshot và các diagram cho các nhóm chức năng	15
6.1 Nhóm chức năng quản lý tài khoản	15
6.1.1 Use-case U1.1: Đăng nhập vào hệ thống xác thực tập trung (SSO)	15
6.2 Nhóm tiện ích bãi xe	18
6.2.1 Use-case U2.1: Xác nhận và ghi nhận ra vào tự động	18
6.2.2 Use-case U2.2: Xác nhận ra vào thủ công	21
6.2.3 Use-case U2.3: Đăng ký biển số xe	24
6.2.4 Use-case U2.4: Đăng ký / Gia hạn gói theo tháng	27
6.3 Nhóm thanh toán	30
6.3.1 Use-case U3.1: Xem lịch sử giao dịch cá nhân	30
6.3.2 Use-case U3.2: Thanh toán hóa đơn	32
6.3.3 Use-case U3.3: Thanh toán thủ công	34
6.4 Nhóm IoT	36
6.4.1 Use-case U4.1: Cập nhật trạng thái vị trí đỗ	36
6.4.2 Use-case U4.2: Hiển thị trạng thái bãi xe	38
6.4.3 Use-case U4.3: Thông báo cảm biến lỗi	40
6.5 Nhóm Admin	42
6.5.1 Use-case 5.2: Quản lý tài khoản nhân viên	42
6.5.2 Use-case 5.3: Xuất báo cáo	46
6.5.3 Use-case 5.4: Thay đổi chính sách giá	50
6.5.4 Use-case 5.5: Truy cập nhật ký hệ thống	54

6.5.5	Use-case 5.6: Giám sát tình trạng bãi đỗ xe	58
6.6	Một số state chart cho toàn hệ thống	64
6.7	Development / Implementation View	65
6.7.1	Component Diagram	65
6.8	Cấu trúc mã nguồn và Tổ chức Package	65
6.8.1	Cấu trúc Backend Core (Spring Boot / Node.js)	65
6.8.2	Cấu trúc Frontend (React)	66
6.8.3	Quản lý phụ thuộc và Thư viện sử dụng	66
6.9	Deployment View	67
6.9.1	Mô tả các thực thể và Các thành phần	67
6.9.2	Giao thức giao tiếp (Communication Protocols)	68
7	UI design	69
7.1	Giao diện bắt đầu	69
7.2	Giao diện đăng ký biển số xe	70
7.3	Các giao diện cho các tiện ích khác của người dùng	72
7.4	Các giao diện cho Admin	73
8	Danh sách các Method và Class Diagram	74
8.1	Các Enumeration trong Class Diagram	74
8.2	Các Class trong Class Diagram	75
8.2.1	Lớp thực thể: Account (Tài khoản)	75
8.2.2	Lớp thực thể: UserSession (Phiên làm việc)	76
8.2.3	Lớp điều khiển: AuthenticationController	77
8.2.4	Lớp biên: HCMUTSSOConnector và HCMUTDataCoreConnector	77
8.2.5	Lớp thực thể: ParkingSession (Phiên gửi xe)	77
8.2.6	Lớp thực thể: ParkingSpot (Vị trí đỗ xe)	78
8.2.7	Lớp thực thể: Vehicle (Phương tiện)	78
8.2.8	Lớp thực thể: RFIDCard (Thẻ từ)	79
8.2.9	Lớp điều khiển: Employee (Nhân viên vận hành)	79
8.2.10	Lớp điều khiển: GuestCustomer (Khách vắng lai)	79
8.2.11	Lớp điều khiển: MemberCustomer (Khách thành viên)	80
8.2.12	Lớp Invoice	80
8.2.13	Lớp Transaction	81
8.2.14	Lớp IoTDevice	82
8.2.15	Lớp ParkingSpot (Vị trí đỗ xe)	83
8.2.16	Lớp: Admin	84
8.2.17	Lớp: PricingService	84
8.2.18	Lớp: PricingPolicy	85
8.2.19	Lớp: MonthlyTicket	85
9	Testcase	85



9.1	U3.1 – Xem lịch sử giao dịch cá nhân	85
9.2	U3.2 – Thanh toán hóa đơn	86
9.3	U3.3 – Thanh toán thủ công	87
9.4	U2.1 — Xác nhận và ghi nhận ra vào tự động	88

Danh sách hình vẽ

1	Use-case Diagram dành cho toàn bộ dự án này.	10
2	Sequence diagram cho Use-case U1.1: Đăng nhập vào hệ thống xác thực tập trung (SSO)	16
3	Activity diagram cho Use-case U1.1: Đăng nhập vào hệ thống xác thực tập trung (SSO)	17
4	Activity Diagram — U2.1: Xác nhận và ghi nhận ra vào tự động	19
5	Sequence Diagram — U2.1: Xác nhận và ghi nhận ra vào tự động	20
6	Activity Diagram — U2.2: Xác nhận ra vào thủ công	22
7	Sequence Diagram — U2.2: Xác nhận ra vào thủ công	23
8	Activity Diagram — U2.3: Đăng ký biển số xe	25
9	Sequence Diagram — U2.3: Đăng ký biển số xe	26
10	Activity Diagram — U2.4: Đăng ký gói theo tháng	28
11	Sequence Diagram — U2.4: Đăng ký gói theo tháng	29
12	Sequence diagram cho Use-case U3.1: Xem lịch sử giao dịch cá nhân	31
13	Activity diagram cho Use-case U3.1: Xem lịch sử giao dịch cá nhân	31
14	Sequence diagram cho Use-case U3.2: Thanh toán hóa đơn	33
15	Activity diagram cho Use-case U3.2: Thanh toán hóa đơn	33
16	Sequence diagram cho Use-case U3.3: Thanh toán thủ công	35
17	Activity diagram cho Use-case U3.3: Thanh toán thủ công	35
18	Sequence Diagram — U4.1: Cập nhật trạng thái vị trí đỗ	37
19	Activity Diagram — U4.1: Cập nhật trạng thái vị trí đỗ	37
20	Sequence Diagram — U4.2: Hiển thị trạng thái bãi xe	39
21	Activity Diagram — U4.2: Hiển thị trạng thái bãi xe	39
22	Sequence Diagram — U4.3: Thông báo cảm biến lỗi	41
23	Activity Diagram — U4.3: Thông báo cảm biến lỗi	41
24	Sequence Diagram — UC5.2: Quản lý tài khoản nhân viên	44
25	Activity Diagram — UC5.2: Quản lý tài khoản nhân viên	45
26	Sequence Diagram — UC5.3: Xuất báo cáo	48
27	Activity Diagram — UC5.3: Xuất báo cáo	49
28	Sequence Diagram — UC5.4: Thay đổi chính sách giá	52
29	Activity Diagram — UC5.4: Thay đổi chính sách giá	53
30	Sequence Diagram — UC5.5: Truy cập nhật ký hệ thống	56
31	Activity Diagram — UC5.5: Truy cập nhật ký hệ thống	57
32	Sequence Diagram — UC5.6: Giám sát tình trạng bãi đỗ xe	62
33	Activity Diagram — UC5.6: Giám sát tình trạng bãi đỗ xe	63
34	Các State Chart trong hệ thống	64
35	Deployment Diagram của hệ thống IoT-SPMS	69

36	Ảnh chụp màn hình đăng nhập	69
37	Giao diện trang chủ web	70
38	Tổng hợp giao diện cho các bước đăng ký biển số xe	70
39	Giao diện xét duyệt đăng ký biển số bên phía nhân viên	71
40	Tổng hợp các giao diện cho các tiện ích khác của người dùng	72
41	Tổng hợp các giao diện cho các chức năng của Admin	73
42	Class diagram cho toàn bộ hệ thống	74

1 Tổng quan dự án

1.1 Bối cảnh dự án

Trường Đại học Bách khoa - ĐHQG TP.HCM mỗi ngày phục vụ một lượng lớn phương tiện của sinh viên, học viên cao học, nghiên cứu sinh, giảng viên, cán bộ - nhân viên và khách vãng lai. Trong bối cảnh mật độ phương tiện ngày càng tăng trong khi sức chứa bãi xe có hạn, hệ thống quản lý hiện tại đang gặp nhiều khó khăn như tình trạng ùn tắc tại cổng ra/vào vào giờ cao điểm và việc khai thác chỗ đậu xe chưa được tối ưu.

1.2 Mục tiêu dự án

Nhằm giải quyết các vấn đề trên, dự án này sẽ xây dựng Hệ thống Quản lý Bãi đỗ xe Thông minh dựa trên IoT (IoT-based Smart Parking Management System – IoT-SPMS). Hệ thống này được thiết kế để tự động hóa kiểm soát ra/vào, giám sát trạng thái chỗ đậu xe, điều hướng giao thông nội bộ thông qua bảng điện tử, cũng như tích hợp cơ chế tính phí, thanh toán và tích hợp với hạ tầng công nghệ. Hơn nữa, hệ thống phải đảm bảo hoạt động ổn định trong các điều kiện thực tế như số lượng người dùng đồng thời lớn, kết nối mạng có thể gián đoạn, dữ liệu IoT có thể bị trễ hoặc không nhất quán, và phải phục vụ nhiều nhóm người dùng với các chính sách khác nhau.

1.3 Stakeholders

- **Nhóm người gửi xe:**

- *Sinh viên, học viên, nghiên cứu sinh, giảng viên, cán bộ:* Sử dụng thẻ sinh viên/cán bộ để ra/vào bãi xe; có thể đăng ký tài khoản trong ứng dụng để hưởng một số quyền lợi phí gửi xe được cộng dồn theo chu kỳ và thanh toán qua BKPay (có thể trả sau) và đăng ký các gói gửi xe theo tháng. Kỳ vọng: vào/ra nhanh chóng, không gặp ùn tắc và chi phí được minh bạch.
- *Khách vãng lai:* Do không có thẻ sinh viên/cán bộ nên không thể không có tài khoản nên sẽ không thể sử dụng các tiện ích của ứng dụng, thay vào đó sẽ nhận thẻ gửi xe tạm thời tại cổng và thanh toán thủ công bằng tiền mặt khi ra khỏi bãi khi ra khỏi bãi. Tuy nhiên phải đảm bảo kỳ vọng của họ là tương tự với các người gửi xe khác.

- **Nhân viên vận hành bãi xe:** Hỗ trợ xác thực thủ công trong các trường hợp ngoại lệ, quản lý thẻ tạm thời và xử lý sự cố tại cổng. Kỳ vọng: một giao diện dễ sử dụng, cho phép thao tác nhanh và giảm thiểu sai sót.

- **Admin:** Phụ trách các công việc như cấu hình chính sách giá, phân quyền người dùng, giám sát hạ tầng, xuất báo cáo, thống kê và theo dõi nhật ký hệ thống. Kỳ vọng: hệ

thống phải cho phép quản lý và cấu hình linh hoạt một cách ổn định và bảo mật.

- **Các hệ thống bên ngoài:**

- *HCMUT_SSO*: Xác thực người dùng nội bộ.
- *HCMUT_DATACORE*: Đồng bộ thông tin cá nhân và vai trò ở chế độ chỉ đọc.
- *BKPay*: Xử lý thanh toán phí gửi xe.
- *IoT Sensors & Gateway*: Cảm biến phát hiện trạng thái chỗ đậu và Gateway truyền dữ liệu về hệ thống trung tâm.

1.4 Phạm vi dự án

Trong phạm vi phát triển, hệ thống bao gồm các chức năng chính để đảm bảo vận hành toàn diện. Chức năng kiểm soát ra/vào hỗ trợ ghi nhận tự động bằng thẻ định danh, cấp thẻ tạm cho khách hoặc các trường hợp ngoại lệ, và cho phép xác thực thủ công khi cần thiết. Chức năng quản lý chỗ đậu xe cung cấp khả năng cập nhật trạng thái chỗ đậu theo thời gian thực, phát hiện lỗi cảm biến và hiện thị trạng thái bãi xe như còn chỗ, gần đầy, hoặc đầy. Đối với việc thanh toán, hệ thống sẽ tự động tính phí theo chu kỳ, gửi yêu cầu thanh toán qua BKPay và hỗ trợ thanh toán tiền mặt cho khách vắng lai. Hệ thống cũng quản lý tài khoản và phân quyền thông qua việc tích hợp HCMUT_SSO, đồng bộ dữ liệu từ HCMUT_DATACORE. Cuối cùng, chức năng báo cáo và truy vết đảm bảo lưu toàn bộ hoạt động, phục vụ việc xuất báo cáo tài chính và vận hành.

Ngược lại, dự án cũng xác định rõ các yếu tố ngoài phạm vi triển khai. Những hạng mục này bao gồm việc phát triển phần cứng IoT thực tế, phát triển một hệ thống thanh toán riêng để thay thế BKPay, triển khai quy mô toàn thành phố, cũng như việc tích hợp các công nghệ AI nâng cao như nhận diện biển số bằng deep learning.

1.5 Quy trình nghiệp vụ cốt lõi

Định nghĩa Phiên gửi xe:

Phiên gửi xe là một chuỗi hành động và trạng thái liên tục của một phương tiện từ khi tiến vào bãi đỗ cho đến khi rời đi và hoàn tất nghĩa vụ tài chính. Vòng đời của một phiên gửi xe dành cho người dùng có tài khoản được chia thành 4 giai đoạn chính:

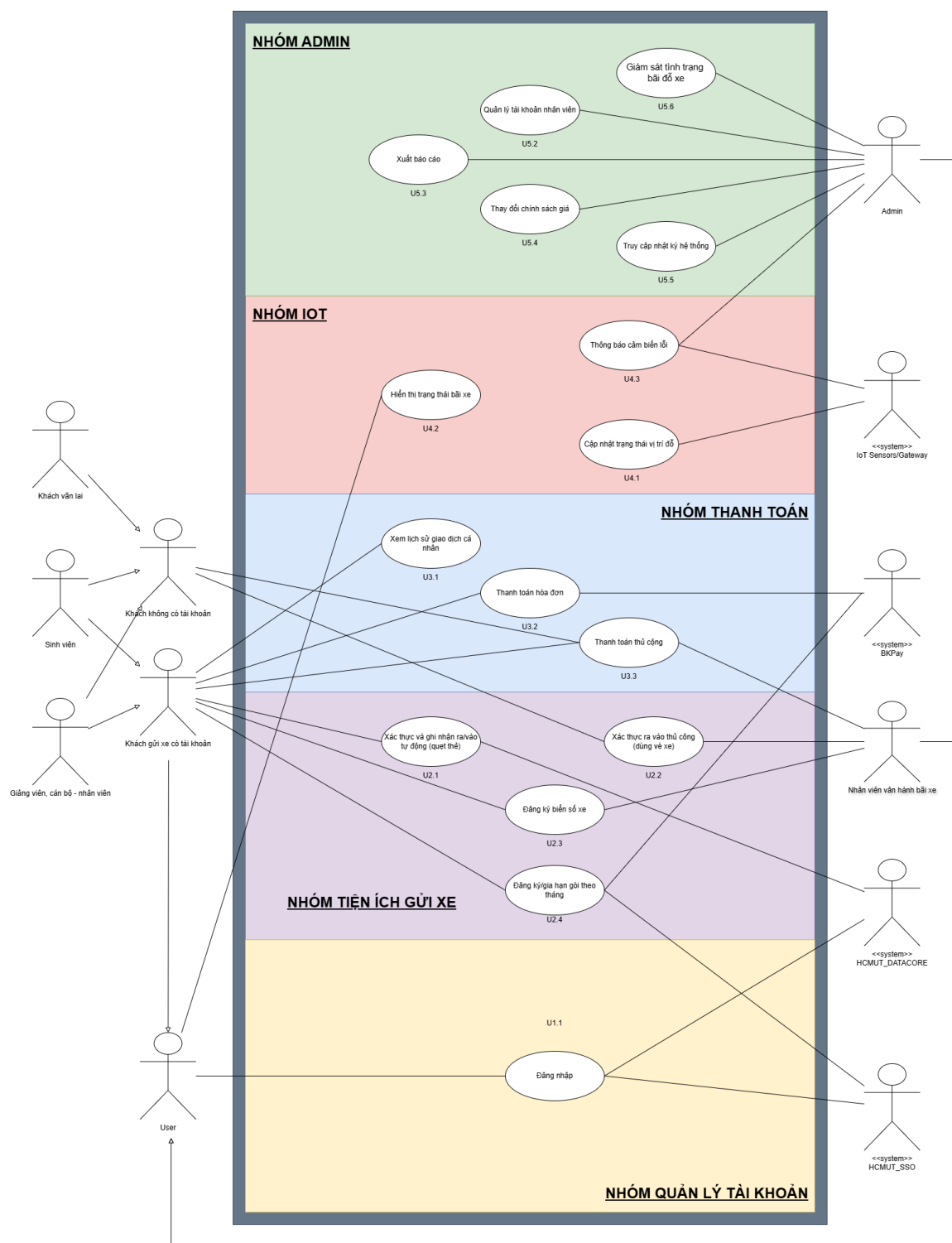
- **Giai đoạn 1 - Khởi tạo phiên:** Khách hàng quẹt thẻ RFID (hoặc thẻ định danh) tại cổng vào. Hệ thống xác thực danh tính, đối chiếu biển số xe qua camera và khởi tạo một "Phiên gửi xe" mới trên cơ sở dữ liệu với trạng thái *Đang hoạt động*, ghi nhận thời điểm bắt đầu.
- **Giai đoạn 2 - Cập nhật vị trí và Tính giờ đỗ:** Sau khi qua cổng, khách hàng di chuyển xe đến vị trí đỗ. Cảm biến IoT tại vị trí đỗ phát hiện có xe và gửi tín hiệu về

Gateway. Hệ thống trung tâm liên kết vị trí đỗ này với phiên gửi xe vừa khởi tạo, đồng thời duy trì trạng thái đỗ để làm cơ sở tính toán thời gian thực tế.

- **Giai đoạn 3 - Kết thúc phiên:** Khách hàng di chuyển xe ra cổng và quét thẻ. Hệ thống đối chiếu dữ liệu cổng ra với cổng vào, xác nhận hợp lệ và đóng phiên gửi xe. Trạng thái phiên chuyển sang *Đã hoàn tất*, đồng thời ghi nhận thời điểm kết thúc.
- **Giai đoạn 4 - Xử lý hậu kỳ:** Dựa trên tổng thời gian của phiên đỗ xe và chính sách giá đã thiết lập, hệ thống tự động tính toán chi phí cho phiên này. Chi phí này được cộng dồn vào tài khoản ghi nợ của khách hàng. Lúc này, phiên gửi xe sẽ được chuyển sang trạng thái *Chờ thanh toán*. Sau khi khách hàng hoàn tất nghĩa vụ tài chính vào cuối tháng, phiên sẽ chính thức khép lại với trạng thái *Đã thanh toán*.
- **Giai đoạn ngoại lệ:** Trong quá trình hoạt động, nếu xảy ra sự cố (khách mất thẻ, xe lưu bãi quá thời gian quy định, lỗi cảm biến, ...), phiên gửi xe sẽ bị treo ở trạng thái *Ngoại lệ*. Lúc này, cần sự can thiệp thủ công của nhân viên vận hành để xác minh và đóng phiên hợp lệ.

2 Tổng quan chức năng và ranh giới hệ thống

2.1 Tổng quan chức năng



Hình 1: Use-case Diagram dành cho toàn bộ dự án này.

Hệ thống IoT-SPMS được thiết kế nhằm tối ưu hóa quy trình quản lý bãi xe thông qua việc phân tách rõ ràng các nhóm chức năng. Để đảm bảo tính toàn diện, các chức năng được chia thành hai hướng tiếp cận chính: hướng người dùng cuối và hướng vận hành hệ thống.

2.1.1 Nhóm chức năng hướng người dùng

Nhóm chức năng này tập trung vào việc cung cấp trải nghiệm thuận tiện và nhanh chóng cho sinh viên, cán bộ và khách vãng lai.

- **Quản lý định danh và tài khoản:** Hỗ trợ đăng nhập qua hệ thống xác thực tập trung HCMUT_SSO và đồng bộ thông tin cá nhân từ DATACORE.
- **Tiện ích gửi xe cá nhân:** Cho phép người dùng tự đăng ký biển số xe trực tuyến, đăng ký các gói gửi xe theo tháng và theo dõi sơ đồ vị trí trống trong bãi xe theo thời gian thực.
- **Tương tác ra/vào bãi:** Thực hiện quy trình quét thẻ và xác thực biển số xe tự động tại các cổng kiểm soát.
- **Thanh toán và tra cứu:** Người dùng có thể xem lại toàn bộ lịch sử giao dịch cá nhân và thực hiện thanh toán hóa đơn điện tử thông qua cổng BKPay.

2.1.2 Nhóm chức năng hướng vận hành

Nhóm chức năng này phục vụ đội ngũ nhân viên và Admin nhằm đảm bảo hệ thống luôn hoạt động ổn định và minh bạch.

- **Hỗ trợ vận hành trực tiếp:** Cung cấp giao diện cho nhân viên bãi xe thực hiện xác nhận ra/vào thủ công, cấp phát thẻ tạm cho khách vãng lai và xử lý các tình huống ngoại lệ tại cổng.
- **Giám sát hạ tầng IoT:** Theo dõi trạng thái hoạt động (online/offline) của các cảm biến và Gateway; tự động tiếp nhận thông báo cảnh báo khi có thiết bị gặp sự cố hoặc gửi dữ liệu bất thường.
- **Quản trị chính sách và nhân sự:** Cho phép Admin cấu hình linh hoạt chính sách giá gửi xe theo khung giờ và đối tượng; quản lý và phân quyền tài khoản cho đội ngũ nhân viên vận hành.
- **Kiểm soát và báo cáo tài chính:** Hệ thống tự động tổng hợp phí gửi xe, tạo hóa đơn định kỳ và hỗ trợ xuất các báo cáo thống kê doanh thu, lưu lượng xe phục vụ công tác hậu kiểm.
- **Truy vết hệ thống:** Cung cấp khả năng truy cập nhật ký raw (Logs) để điều tra sự cố và kiểm toán các hành động thay đổi dữ liệu trong hệ thống.

3 Ranh giới hệ thống

Ranh giới hệ thống xác định giới hạn mà phần mềm IoT-SPMS trực tiếp quản lý và xử lý dữ liệu, phân tách với các tác nhân con người và các hệ thống ngoại vi. Cụ thể được phân định như sau:

3.0.1 Bên trong ranh giới hệ thống

- **Logic xử lý nghiệp vụ:** Bao gồm việc khởi tạo và quản lý phiên gửi xe, tự động hóa quy trình kiểm soát ra/vào và tính toán chi phí dựa trên các chính sách giá đã thiết lập.
- **Quản lý dữ liệu tập trung:** Lưu trữ và xử lý thông tin biển số xe đã đăng ký, trạng thái chiếm dụng chỗ đỗ theo thời gian thực, lịch sử giao dịch và nhật ký hệ thống phục vụ kiểm toán.
- **Giao diện người dùng:** Cung cấp các màn hình chức năng cho người dùng như xem sơ đồ bãi xe, lịch sử thanh toán và Dashboard quản trị cho Admin để giám sát thiết bị IoT, xuất báo cáo thống kê.
- **Hệ thống thông báo:** Tự động tạo và gửi các cảnh báo khi phát hiện lỗi cảm biến hoặc gửi nhắc nhở thanh toán hóa đơn cho người dùng.

3.0.2 Bên ngoài ranh giới hệ thống

- **Các hệ thống định danh và dữ liệu của trường:** Hệ thống kết nối và truy vấn thông tin từ *HCMUT_SSO* để xác thực và *HCMUT_DATACORE* để đồng bộ vai trò người dùng. Hệ thống không trực tiếp quản lý kho dữ liệu gốc của các đơn vị này.
- **Cổng thanh toán ngoại vi:** Việc xử lý giao dịch tài chính thực tế được thực hiện hoàn toàn qua *BKPay*. Hệ thống IoT-SPMS chỉ gửi yêu cầu thanh toán và nhận kết quả phản hồi thông qua cơ chế Webhook.
- **Hạ tầng phần cứng IoT:** Mặc dù hệ thống tiếp nhận dữ liệu từ các *IoT Sensors & Gateway*, nhưng việc vận hành vật lý, bảo trì phần cứng và phát triển firmware cho các thiết bị này nằm ngoài phạm vi phần mềm của dự án.
- **Tác nhân con người:** Bao gồm Sinh viên, Giảng viên, Khách vãng lai, Nhân viên vận hành và Admin. Đây là các đối tượng tương tác với hệ thống qua các thiết bị đầu cuối nhưng không thuộc quyền kiểm soát nội tại của logic phần mềm.

4 Các non-interactive functional requirement

- Hệ thống sẽ làm nổi bật những khu vực có nhiều vị trí trống lên màn hình

- Tự tạo hóa đơn định kỳ:
 - Tổng hợp phí gửi xe cuối mỗi chu kỳ (cuối tháng): đăng ký gói tháng hoặc ghi nợ.
 - Tạo hóa đơn cho từng người dùng.
 - Lưu hóa đơn vào cơ sở dữ liệu của hệ thống.
- Tự tạo thông báo thanh toán:
 - Gửi thông báo khi tạo hóa đơn
 - Gửi thông báo khi thanh toán thành công / thất bại
 - Gửi nhắc nhở khi sắp hết hoặc quá thời gian thanh toán

5 Các non-functional requirement

- **NFR1 - Cập nhật trạng thái thời gian thực (Performance):** Hệ thống phải cập nhật trạng thái chỗ đỗ xe từ sensor lên giao diện người dùng trong thời gian không quá 5 giây kể từ khi sensor phát hiện thay đổi.
 - Metric: Độ trễ trung bình từ sensor $\rightarrow$ hiển thị ≤ 5 giây; độ trễ tối đa ≤ 10 giây.
- **NFR2 - Khả năng chịu tải đồng thời (Scalability):** Hệ thống phải hoạt động ổn định trong giờ cao điểm khi lượng người dùng tăng đột biến (gần đầu giờ học, ngay lúc tan học). Hệ thống phải hỗ trợ tối thiểu 500 người dùng đồng thời mà không suy giảm hiệu năng quá ngưỡng cho phép.
 - Metric: Thời gian phản hồi trung bình ≤ 3 giây khi có 500 người dùng đồng thời, tỷ lệ lỗi $< 1\%$.
- **NFR3 - Tính sẵn sàng (Availability):** Hệ thống phải duy trì hoạt động liên tục để đảm bảo kiểm soát ra vào bãi xe không bị gián đoạn.
 - Metric: Uptime tối thiểu 99.5% theo tháng, tương đương thời gian ngừng hoạt động không quá 3.6 giờ/tháng.
- **NFR4 - Tương thích đa thiết bị (Usability):** Giao diện hệ thống phải hiển thị và hoạt động chính xác trên cả thiết bị di động và máy tính để bàn, đảm bảo trải nghiệm người dùng nhất quán.
 - Metric: Giao diện hỗ trợ responsive từ độ phân giải 360px (mobile) đến 1920px (desktop), các chức năng cốt lõi phải sử dụng được đầy đủ trên cả hai loại thiết bị.
- **NFR5 - Hỗ trợ đa ngôn ngữ (Usability):** Hệ thống phải hỗ trợ tiếng Việt và tiếng Anh để phục vụ cả người dùng trong nước và sinh viên/giảng viên quốc tế.

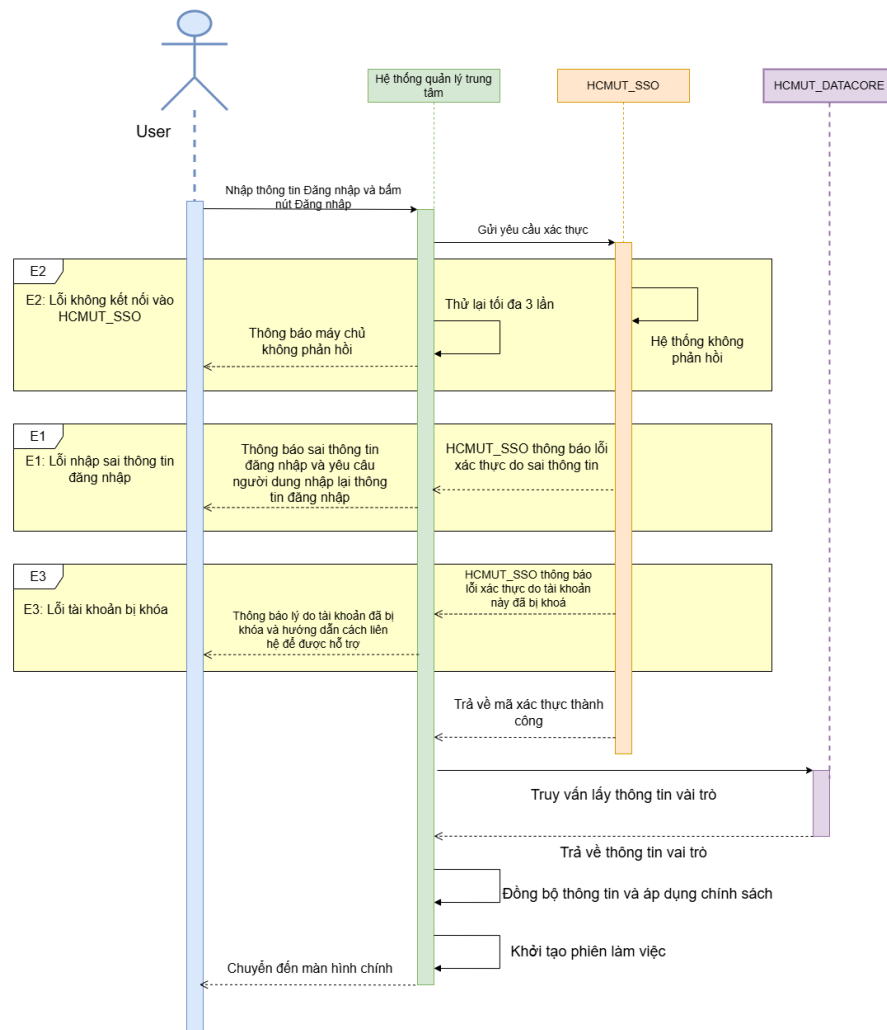
- Metric: 100% nội dung giao diện người dùng (label, thông báo, hướng dẫn) phải có bản dịch đầy đủ cho cả hai ngôn ngữ. Người dùng có thể chuyển đổi ngôn ngữ mà không cần đăng nhập lại.
- **NFR6 - Khả năng chịu lỗi (Fault Tolerance):** Hệ thống phải có cơ chế dự phòng để duy trì hoạt động cơ bản hoặc bảo toàn dữ liệu khi một thành phần (sensor, gateway hoặc kết nối mạng) gặp sự cố.
 - Metric: Khi mất kết nối Internet, Gateway phải có khả năng lưu trữ tạm thời (local buffering) ít nhất 24 giờ dữ liệu và tự động đồng bộ lại khi có kết nối. Thời gian phục hồi hệ thống sau sự cố (MTTR) không quá 30 phút.

6 Screenshot và các diagram cho các nhóm chức năng

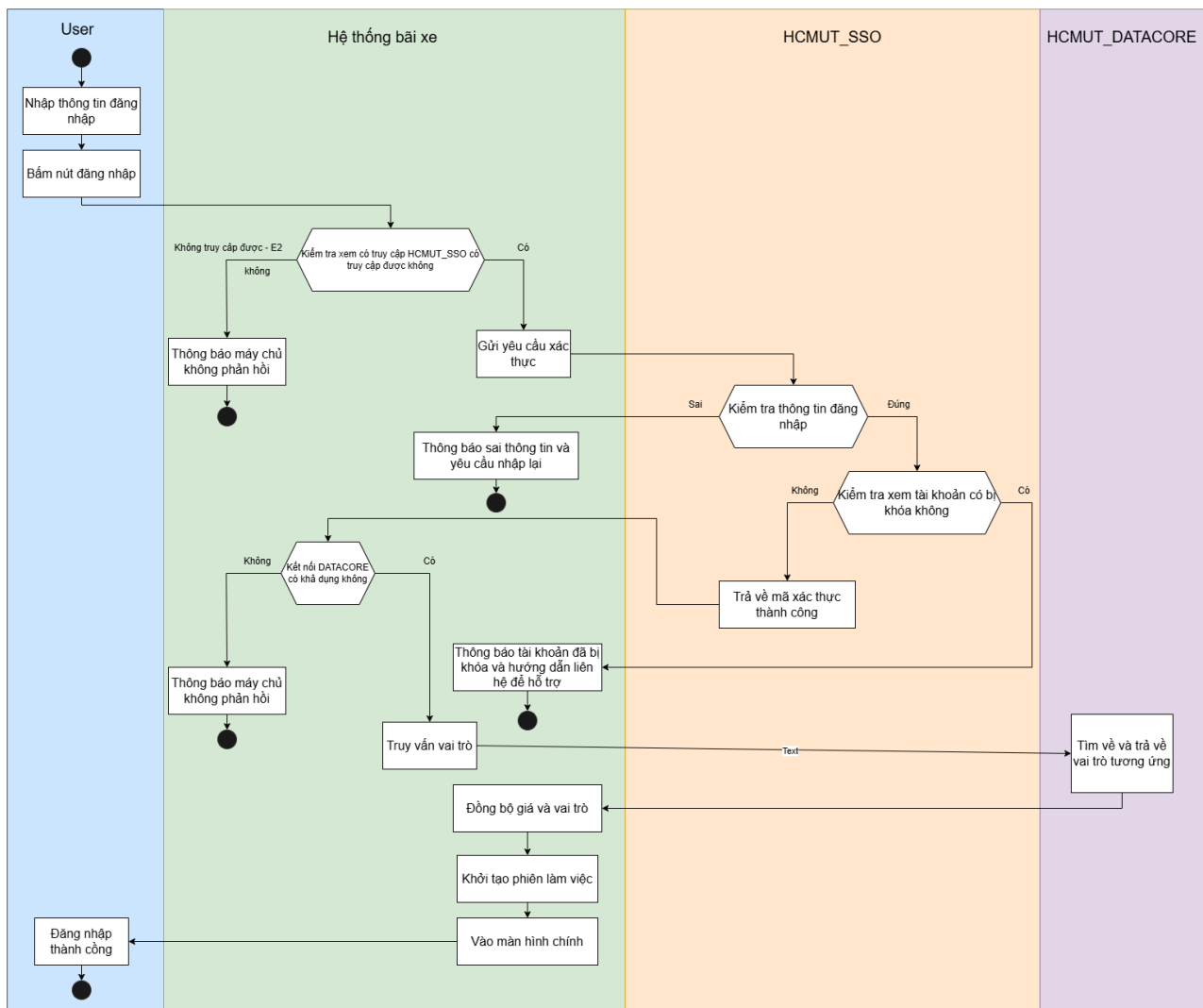
6.1 Nhóm chức năng quản lý tài khoản

6.1.1 Use-case U1.1: Đăng nhập vào hệ thống xác thực tập trung (SSO)

Use-case ID	U1.1
Use-case name	Đăng nhập vào hệ thống xác thực tập trung (SSO)
Use-case overview	Xác thực danh tính của thành viên nhà trường (Sinh viên, Giảng viên, Cán bộ) thông qua hệ thống HCMUT_SSO để cho phép đăng nhập vào hệ thống bãi đỗ xe.
Actors	1. Người dùng (Sinh viên, giảng viên) 2. HCMUT_SSO 3. HCMUT_DATACORE.
Preconditions	1. Người dùng đã có tài khoản định danh chính thức của nhà trường. 2. Hệ thống đang ở màn hình đăng nhập. 3. Kết nối mạng tới các máy chủ xác thực khả dụng.
Trigger	Bấm nút “Đăng nhập”.
Steps	1. Người dùng nhập tên đăng nhập và mật khẩu. 2. Người dùng nhấn nút xác nhận đăng nhập. 3. Hệ thống gửi yêu cầu xác thực tới máy chủ HCMUT_SSO. 4. HCMUT_SSO kiểm tra thông tin và trả về mã xác thực thành công. 5. Hệ thống truy vấn HCMUT_DATACORE để lấy thông tin vai trò người dùng. 6. Hệ thống đồng bộ thông tin vai trò để áp dụng chính sách giá và quyền lợi. 7. Hệ thống khởi tạo phiên làm việc (session) bảo mật. 8. Hệ thống chuyển hướng người dùng đến màn hình điều khiển chính.
Post conditions	Người dùng đã được xác thực, phiên làm việc được thiết lập và quyền hạn đã được phân định dựa trên vai trò từ DATACORE.
Exception flow	- E1 (Thông tin không chính xác): Hệ thống nhận mã lỗi từ HCMUT_SSO. Hệ thống hiển thị thông báo lỗi và yêu cầu nhập lại mật khẩu. - E2 (Lỗi kết nối): Hệ thống không nhận được phản hồi từ HCMUT_SSO hoặc DATACORE sau 5s. Sau đó thông báo máy chủ không phản hồi nếu không thể kết nối SSO hoặc DATACORE. - E3 (Tài khoản bị khóa): HCMUT_SSO trả về trạng thái tài khoản đang bị tạm ngưng. Thông báo lý do và hướng dẫn liên hệ quản trị viên.



Hình 2: Sequence diagram cho Use-case U1.1: Đăng nhập vào hệ thống xác thực tập trung (SSO)

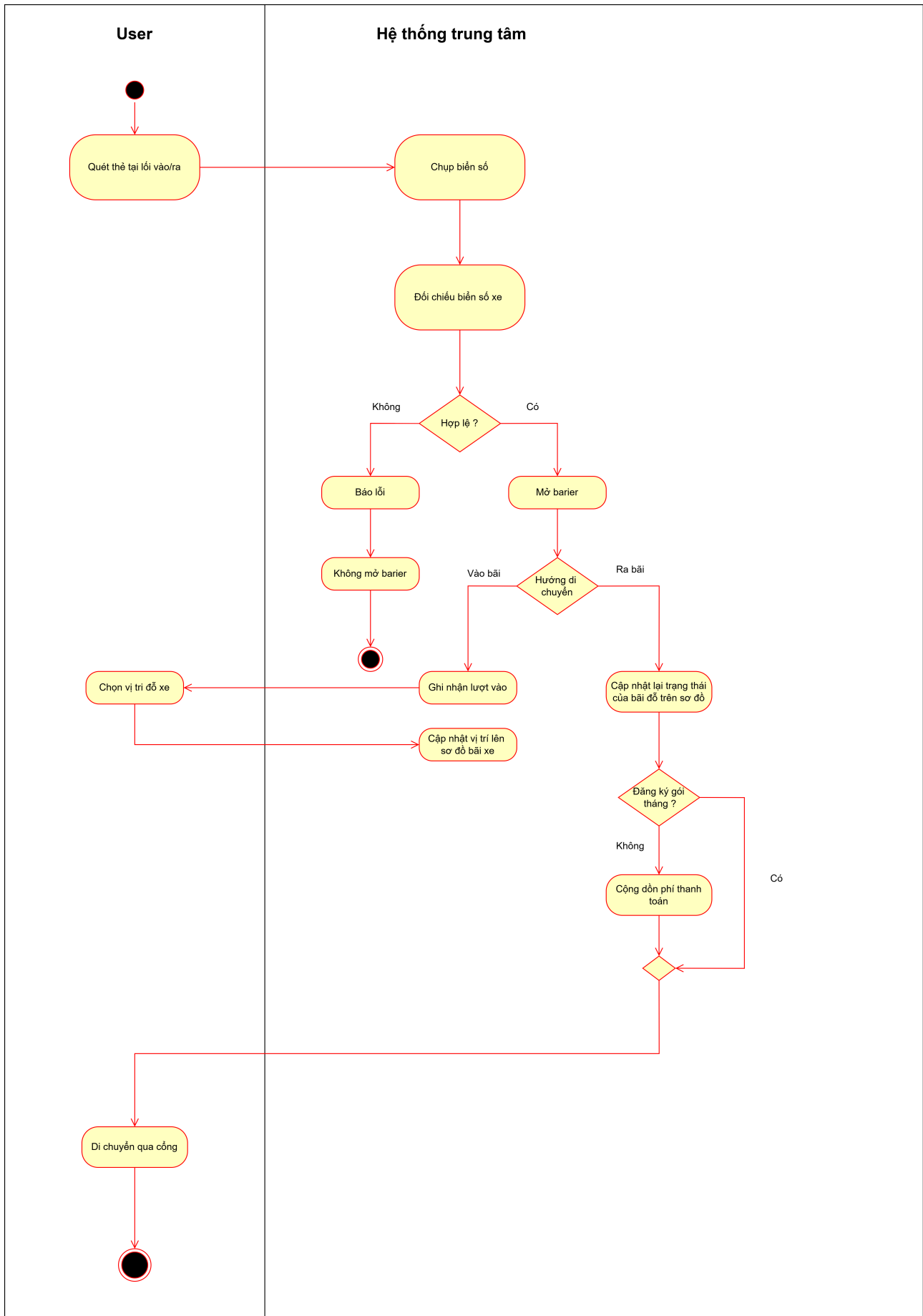


Hình 3: Activity diagram cho Use-case U1.1: Đăng nhập vào hệ thống xác thực tập trung (SSO)

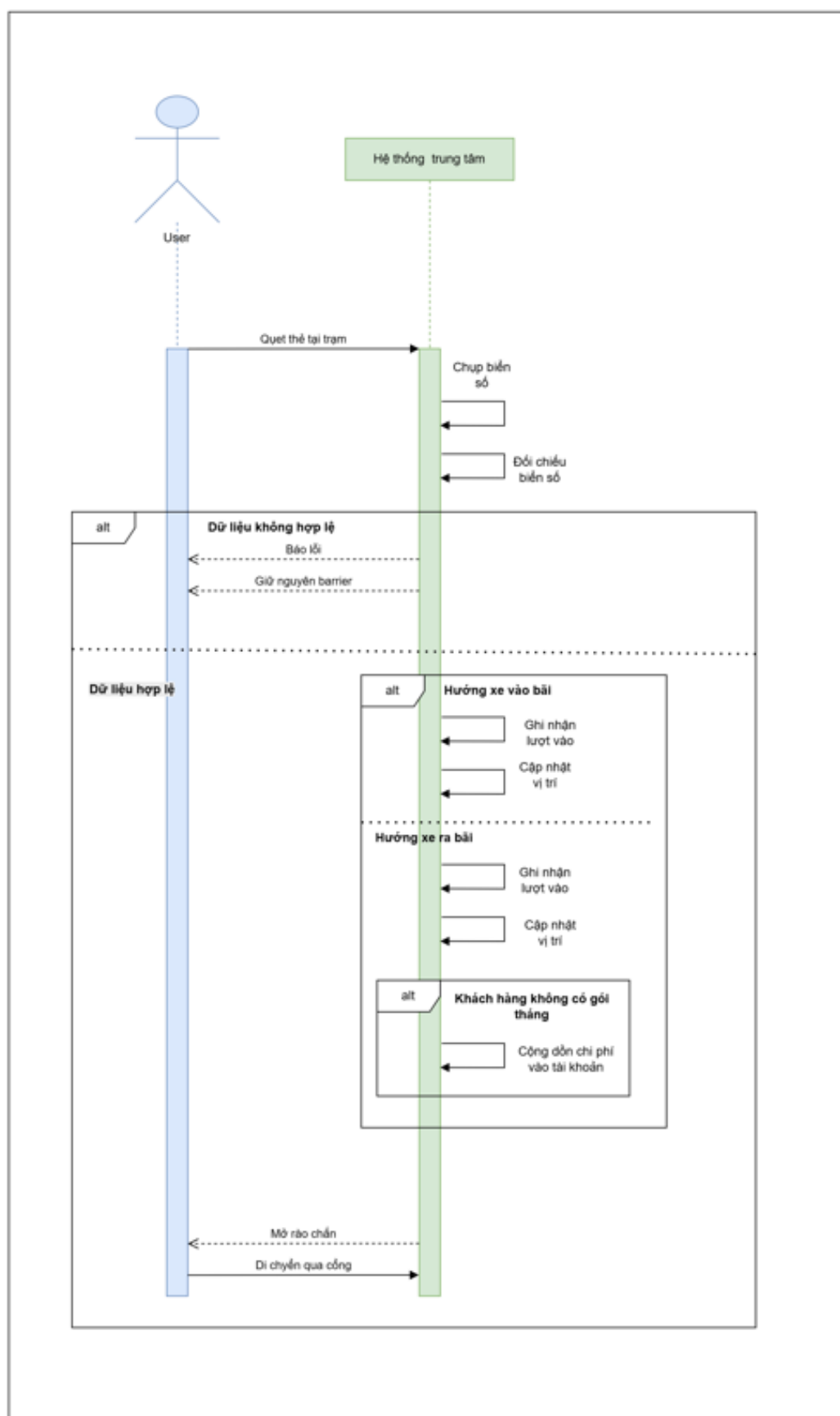
6.2 Nhóm tiện ích bãi xe

6.2.1 Use-case U2.1: Xác nhận và ghi nhận ra vào tự động

Use-case ID	U2.1
Use-case name	Xác nhận và ghi nhận ra vào tự động
Use-case overview	Tự động ghi nhận lượt ra/vào bằng cách quét thẻ RFID và camera nhận diện mà không cần nhân viên thao tác.
Actors	1. Khách gửi xe có tài khoản 2. HCMUT_DATACORE
Preconditions	1. Thẻ sinh viên / giảng viên còn hiệu lực. 2. Biển số xe đã được đăng ký trên hệ thống.
Trigger	Đầu đọc thẻ tại trạm kiểm soát bắt được tín hiệu từ thẻ của khách.
Steps	1. Khách quét thẻ sinh viên/giảng viên tại cổng. 2. Hệ thống đọc mã số sinh viên/giảng viên từ thẻ. 3. Hệ thống kiểm tra tài khoản trên HCMUT_DATACORE. 4. Hệ thống kích hoạt camera chụp biển số xe. 5. Hệ thống kiểm tra biển số có khớp với dữ liệu đã đăng ký. 6. Nếu hợp lệ, hệ thống ghi nhận lượt xe và cập nhật vị trí trên Parking Map sau 5s cảm biến ổn định. 7. Hệ thống khởi tạo "Phiên gửi xe", bắt đầu ghi nhận thời gian đỗ xe thực tế để làm cơ sở tính phí.
Alternative flow	A1: Nếu khách không đăng ký gói tháng, hệ thống tự động cộng dồn phí gửi xe. A2: Nếu khách có gói tháng còn hiệu lực, hệ thống bỏ qua bước tính phí.
Post conditions	Hệ thống ghi nhận lượt vào ra.
Exception flow	E1: Thẻ không hợp lệ (hết hạn hoặc bị khóa), hệ thống báo lỗi và rào chắn không mở. E2: Biển số không khớp với đăng ký, hệ thống báo lỗi và rào chắn không mở.



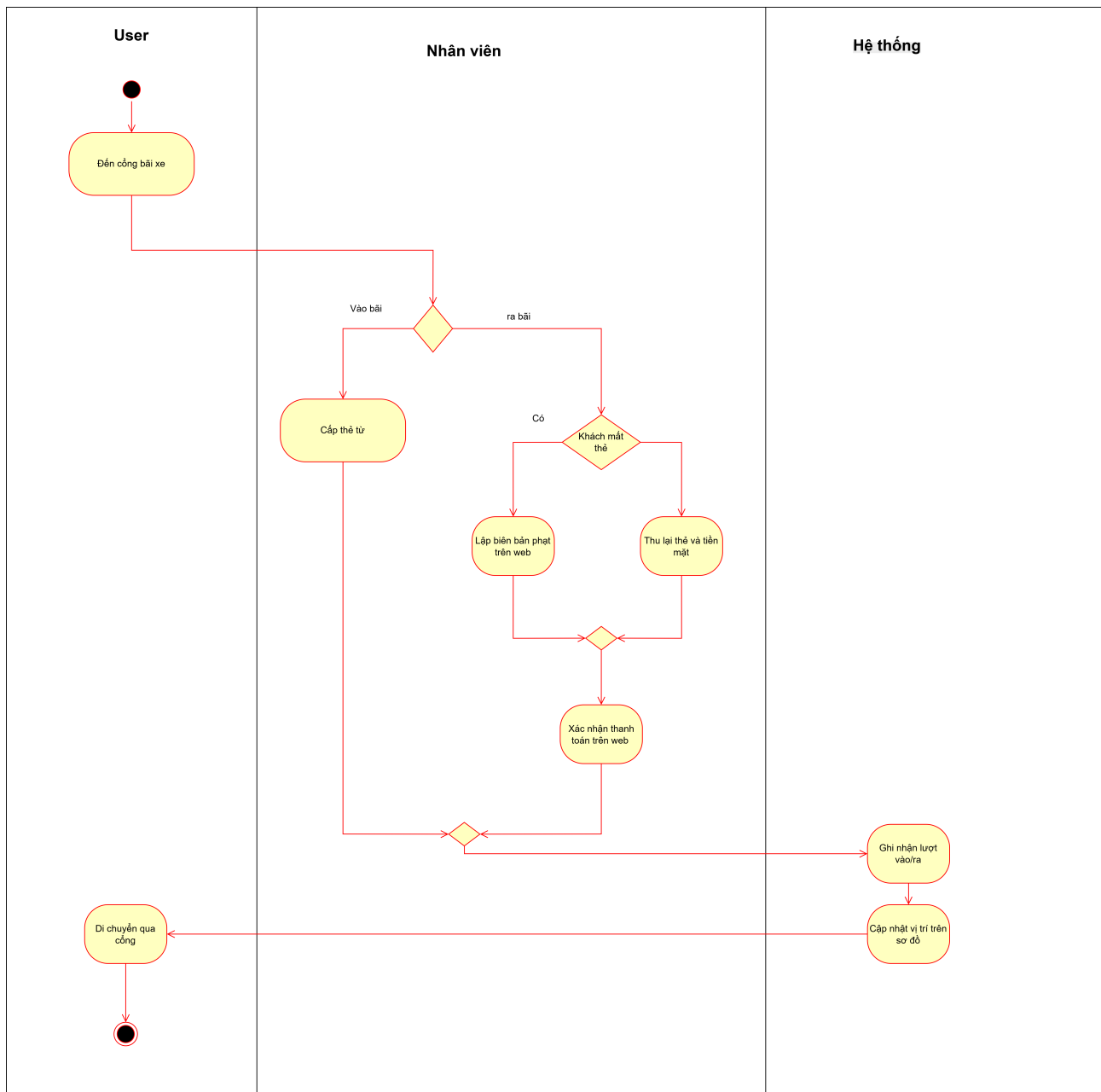
Hình 4: Activity Diagram — U2.1: Xác nhận và ghi nhận ra vào tự động



Hình 5: Sequence Diagram — U2.1: Xác nhận và ghi nhận ra vào tự động

6.2.2 Use-case U2.2: Xác nhận ra vào thủ công

Use-case ID	U2.2
Use-case name	Xác nhận ra vào thủ công
Use-case overview	Nhân viên hỗ trợ khách gửi xe không có tài khoản bằng thẻ từ và thanh toán thủ công.
Actors	1. Khách không có tài khoản 2. Nhân viên vận hành bãi xe
Preconditions	Khách được cấp thẻ từ tại cổng vào.
Trigger	Khách điều khiển xe đến cổng vào/ra.
Steps	1. Nhân viên cấp thẻ từ cho khách khi vào bãi. 2. Khách điều khiển xe vào bãi giữ xe. 3. Hệ thống ghi nhận lượt xe, chụp biển số và cập nhật vị trí trên Parking Map. 4. Khi khách ra, khách đưa thẻ từ cho nhân viên. 5. Nhân viên nhận tiền mặt từ khách. 6. Nhân viên quét thẻ để hệ thống đóng phiên và xác nhận lượt ra.
Alternative flow	A1: Khách làm mất thẻ từ khi lấy xe. - Bước 1: Khách báo mất thẻ cho nhân viên. - Bước 2: Nhân viên yêu cầu xuất trình giấy tờ xe (cà vẹt) và CCCD. - Bước 3: Nhân viên tra cứu biển số trên hệ thống để đối chiếu với hình ảnh camera lúc vào. - Bước 4: Sau khi xác nhận đúng xe, nhân viên thu phí gửi xe và phí phạt mất thẻ. - Bước 5: Nhân viên chọn "Xác nhận ra thủ công (Mất thẻ)" trên phần mềm để đóng phiên và mở cổng. A2: Khách yêu cầu thanh toán chuyển khoản thay vì tiền mặt. - Bước 1: Nhân viên cung cấp mã QR thanh toán của bãi xe. - Bước 2: Khách thực hiện chuyển khoản thành công, nhân viên quét thẻ để xác nhận lượt ra.
Post conditions	Hệ thống ghi nhận lượt vào ra, đóng phiên gửi xe và nhân viên xác nhận thanh toán.
Exception flow	Không có.

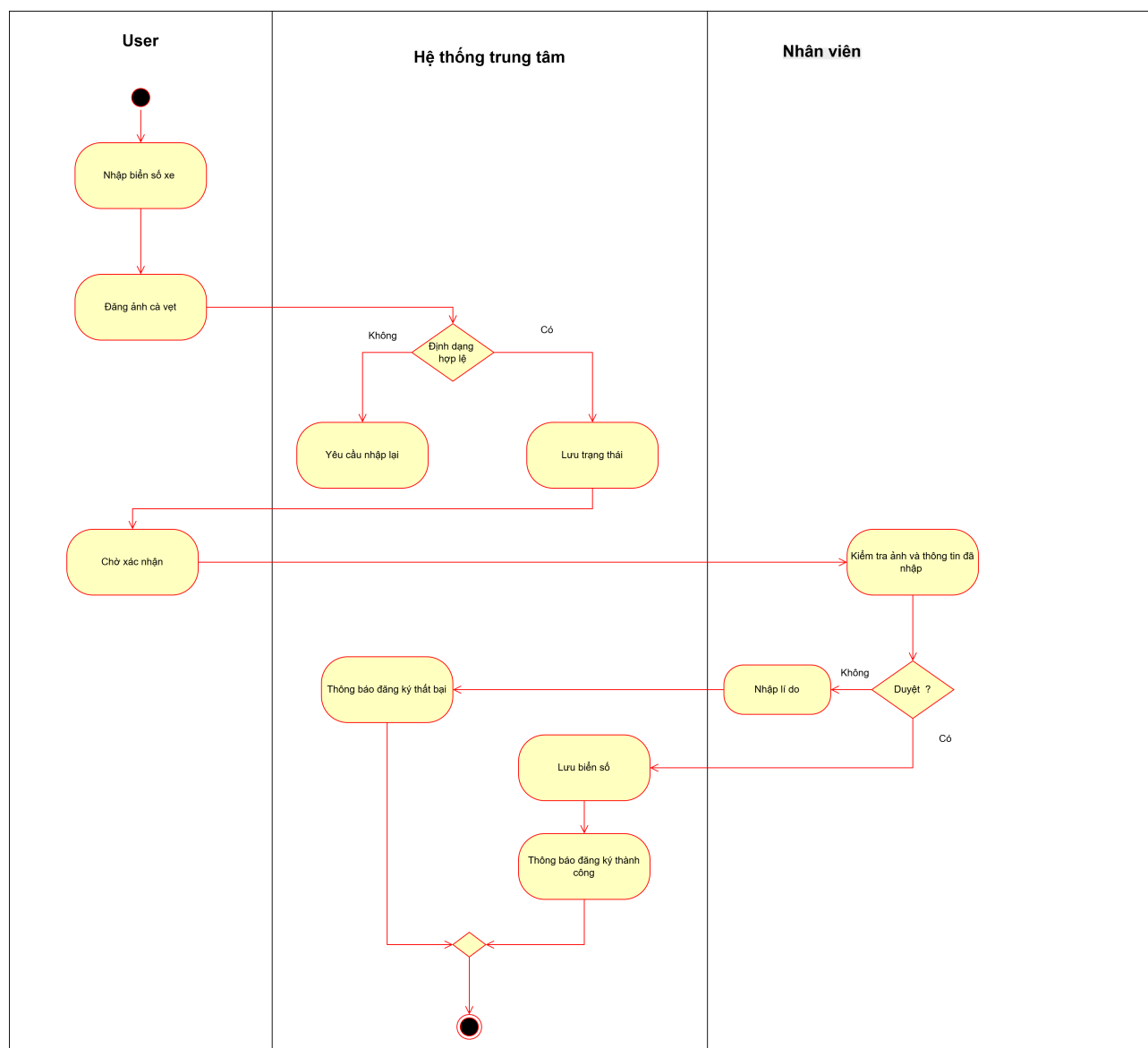


Hình 6: Activity Diagram — U2.2: Xác nhận ra vào thủ công

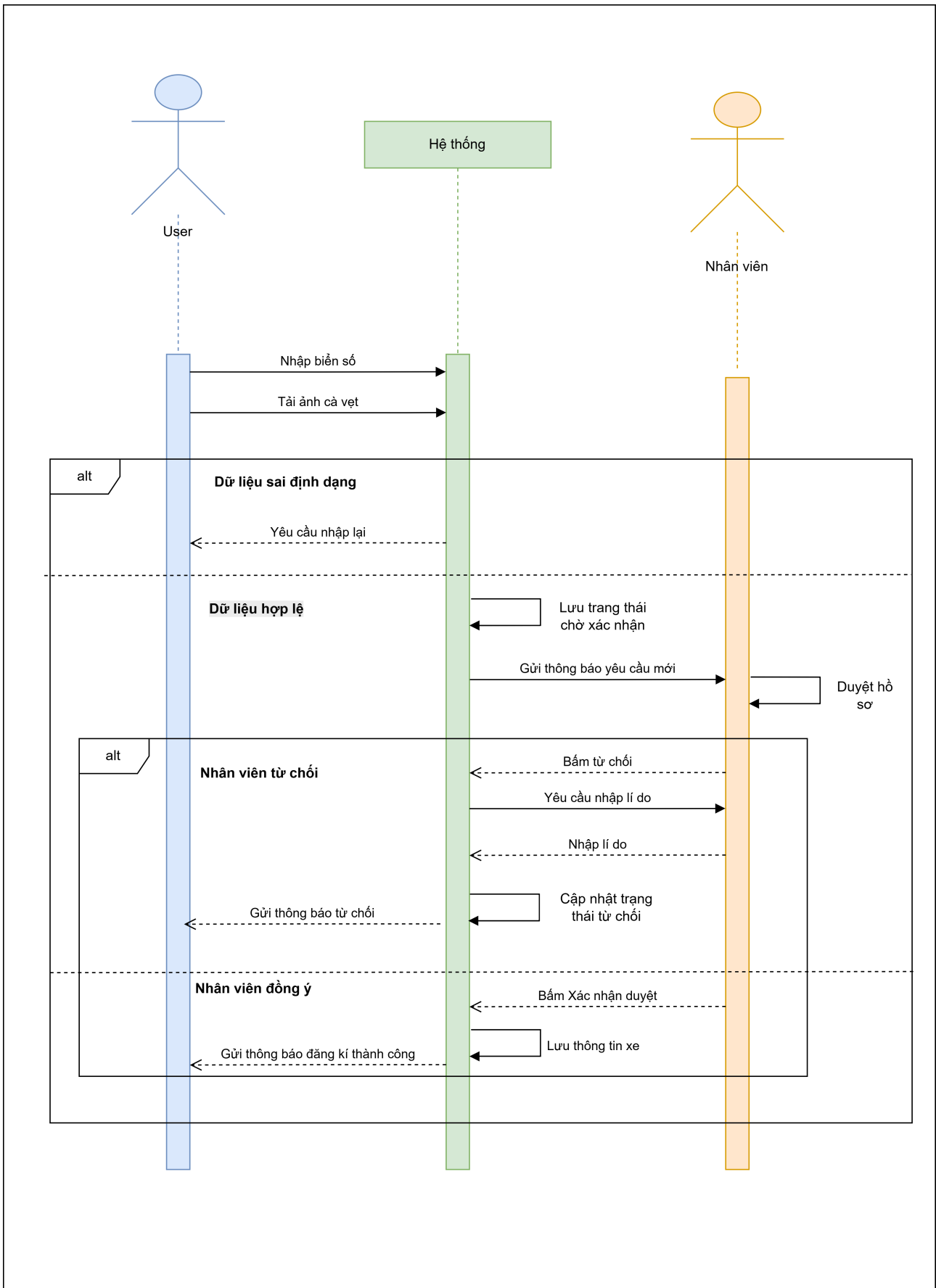


6.2.3 Use-case U2.3: Đăng ký biển số xe

Use-case ID	U2.3
Use-case name	Đăng ký biển số xe
Use-case overview	Khách hàng đăng ký biển số xe lên hệ thống và chờ nhân viên xác nhận.
Actors	1. Khách hàng có tài khoản 2. Nhân viên vận hành bãi xe
Preconditions	Khách hàng đã đăng nhập vào hệ thống.
Trigger	Khách nhấn nút "Đăng ký biển số xe".
Steps	1. Khách nhập biển số xe và tải ảnh giấy tờ (cà vẹt). 2. Hệ thống lưu dữ liệu với trạng thái "Chờ xác nhận". 3. Hệ thống gửi thông báo tới giao diện quản lý của nhân viên. 4. Nhân viên kiểm tra thông tin do khách nhập và đối chiếu với hình ảnh giấy tờ. 5. Nhân viên nhấn "Xác nhận duyệt". 6. Hệ thống cập nhật trạng thái "Đã duyệt" và liên kết biển số với tài khoản của khách.
Alternative flow	A1: Nhân viên phát hiện biển số khách nhập không khớp với hình ảnh cà vẹt (sai biển số), hoặc hình ảnh tải lên bị mờ/không hợp lệ. A2: Nhân viên chọn "Từ chối duyệt" và nhập lý do cụ thể (Ví dụ: "Biển số không khớp với giấy tờ xe"). A3: Hệ thống cập nhật trạng thái "Từ chối" cho yêu cầu này. A4: Hệ thống gửi thông báo (Notification) kèm lý do chi tiết để khách hàng tiến hành chỉnh sửa và đăng ký lại.
Post conditions	Biển số được lưu vào hệ thống và liên kết với tài khoản khách.
Exception flow	E1: Lỗi định dạng biển số hoặc thiếu dữ liệu bắt buộc, hệ thống yêu cầu nhập lại ngay trên giao diện. E2: Biển số khách nhập đã tồn tại và đang liên kết với một tài khoản khác trong hệ thống, thao tác bị chặn.



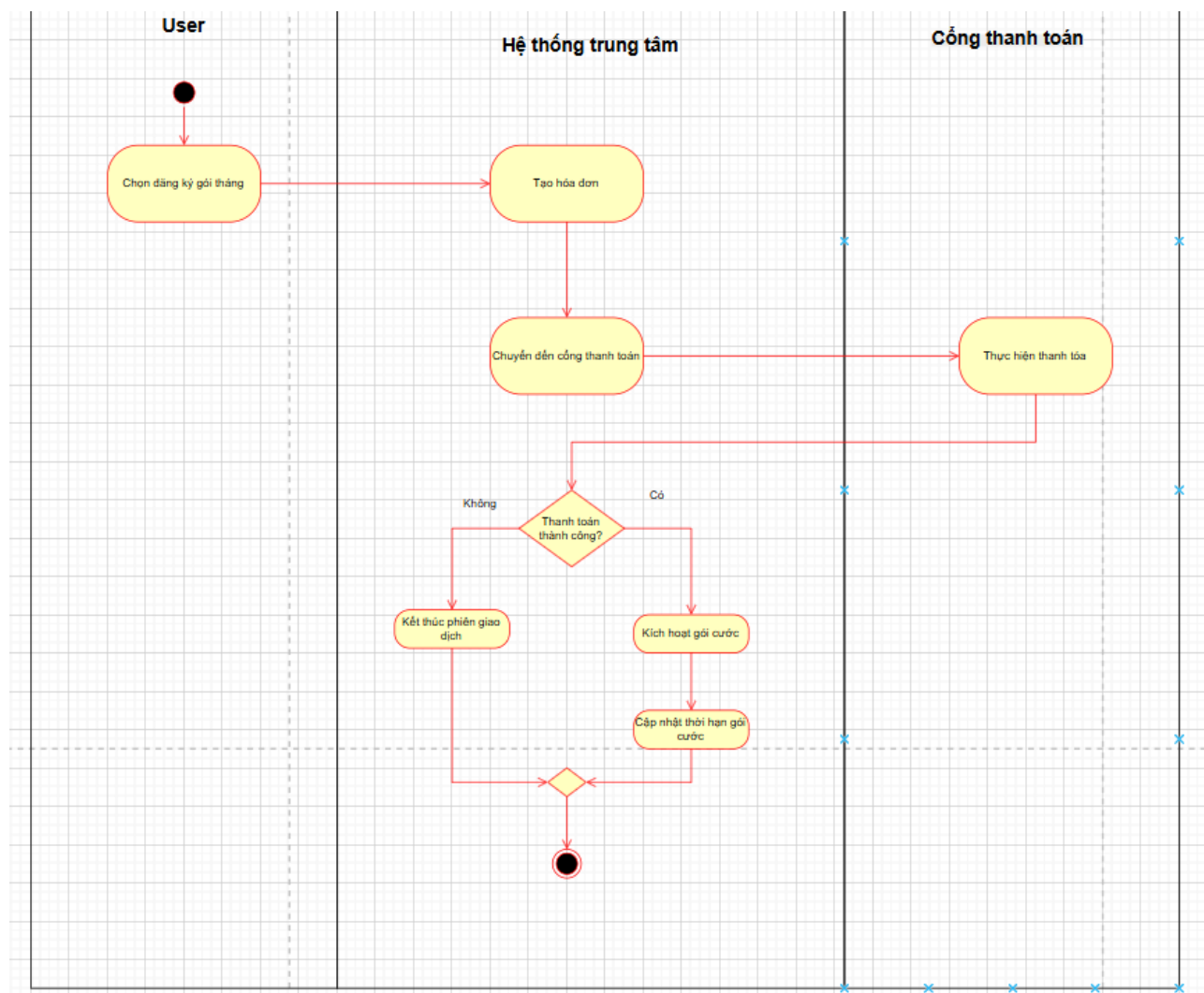
Hình 8: Activity Diagram — U2.3: Đăng ký biển số xe



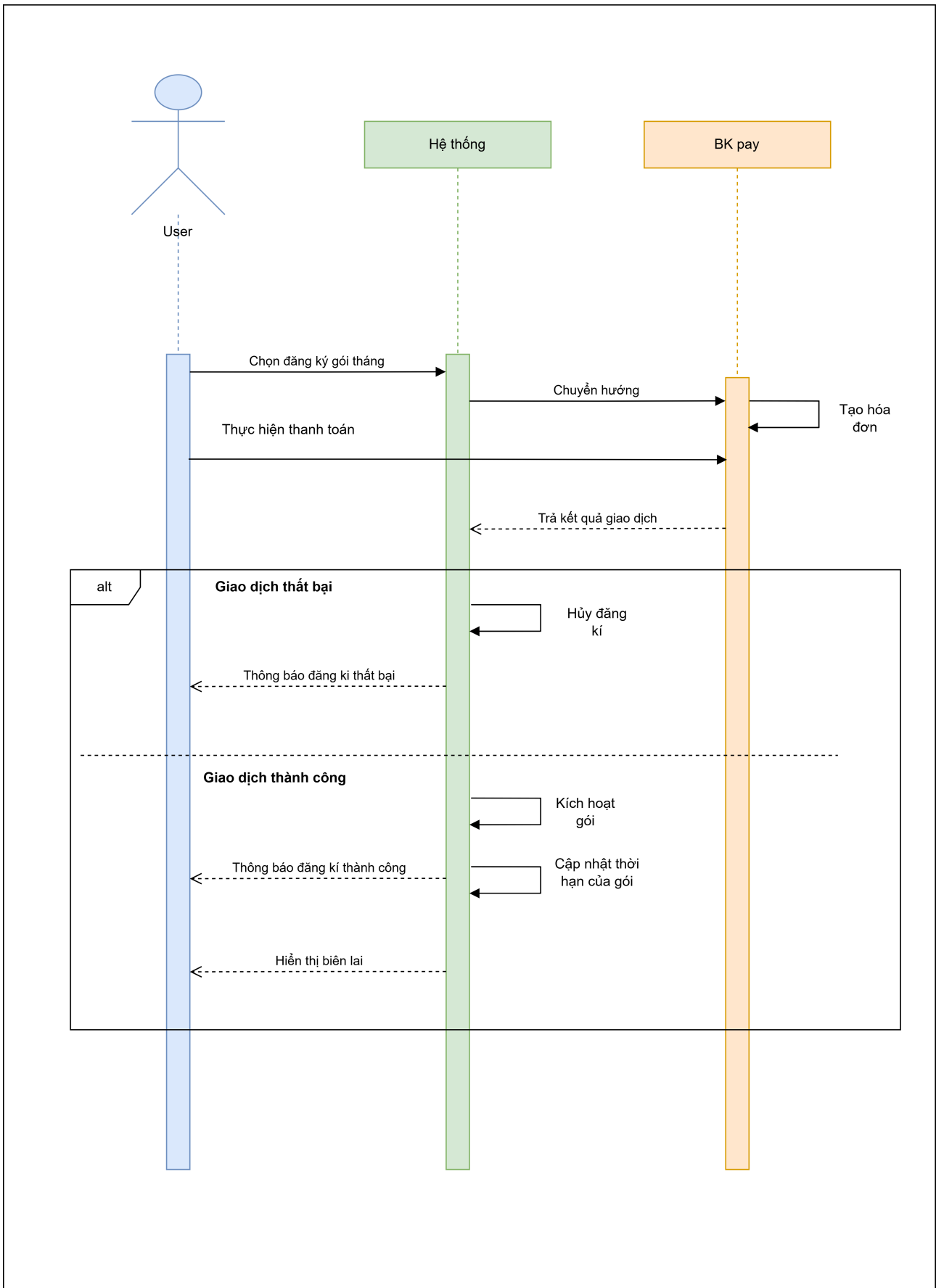
Hình 9: Sequence Diagram — U2.3: Đăng ký biển số xe

6.2.4 Use-case U2.4: Đăng ký / Gia hạn gói theo tháng

Use-case ID	U2.4
Use-case name	Đăng ký / Gia hạn gói theo tháng
Use-case overview	Khách hàng mua mới hoặc gia hạn gói gửi xe theo tháng qua cổng thanh toán BKPay.
Actors	1. Khách hàng có tài khoản 2. BKPay 3. HCMUT_SSO
Preconditions	Khách đã đăng nhập thành công qua HCMUT_SSO và đã có ít nhất 1 biển số xe được duyệt.
Trigger	Khách nhấn nút "Đăng ký gói tháng"(cho đăng ký mới) hoặc nút "Gia hạn"(cho gói đang sử dụng).
Steps	1. Khách hàng chọn biển số xe cần đăng ký gói mới, hoặc chọn gói đang sử dụng để yêu cầu gia hạn. 2. Hệ thống tính toán chi phí và tạo mã đơn thanh toán. 3. Hệ thống chuyển hướng khách sang cổng thanh toán BKPay. 4. Khách thực hiện các bước thanh toán trên giao diện BKPay. 5. BKPay gửi tín hiệu xác nhận giao dịch thành công về hệ thống. 6. Hệ thống xử lý dịch vụ: Kích hoạt gói mới (nếu đăng ký mới) hoặc cộng dồn thêm chu kỳ thời gian vào gói hiện tại (nếu gia hạn). 7. Hệ thống hiển thị biên lai thanh toán và cập nhật ngày hết hạn mới cho khách hàng.
Alternative flow	A1: Khách nhấn "Hủy giao dịch"trên giao diện BKPay. A2: Hệ thống nhận được thông báo hủy, tự động cập nhật trạng thái đơn thành "Đã hủy"và quay lại trang dịch vụ.
Post conditions	Gói vé tháng được kích hoạt thành công hoặc thời hạn gói cũ được gia hạn thêm.
Exception flow	E1: Quá 5 phút khách hàng không hoàn thành thanh toán trên BKPay, hệ thống tự động đánh dấu đơn hàng là "Hết hạn"và hủy đơn.



Hình 10: Activity Diagram — U2.4: Đăng ký gói theo tháng

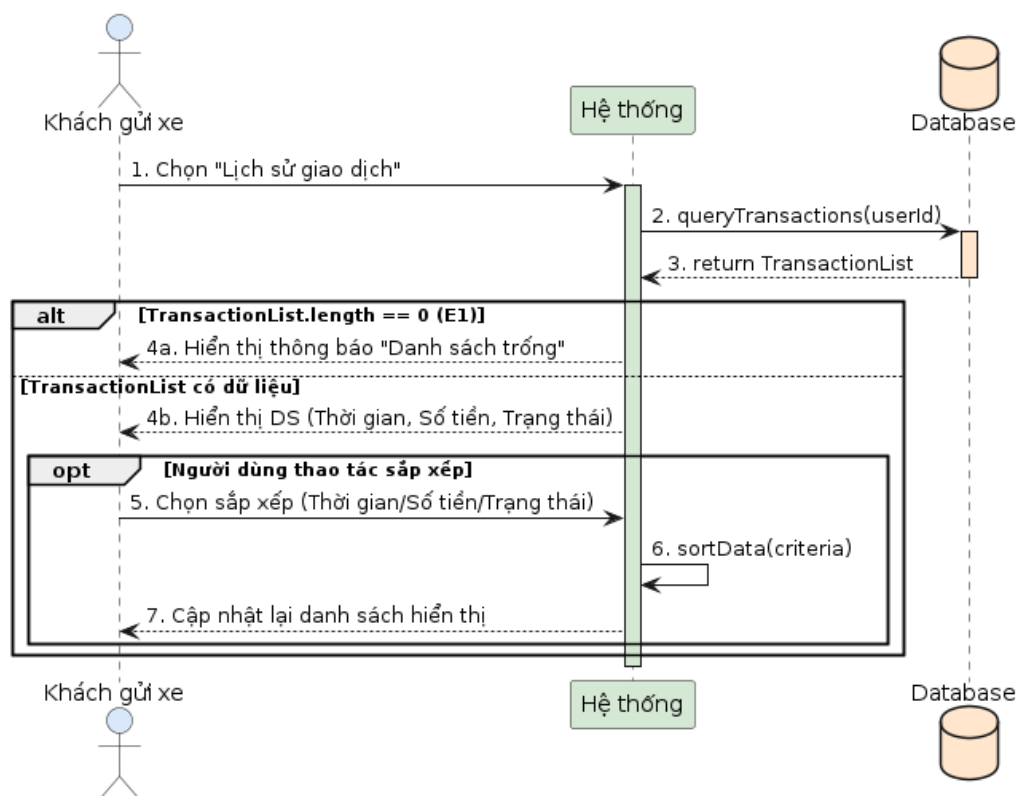


Hình 11: Sequence Diagram — U2.4: Đăng ký gói theo tháng

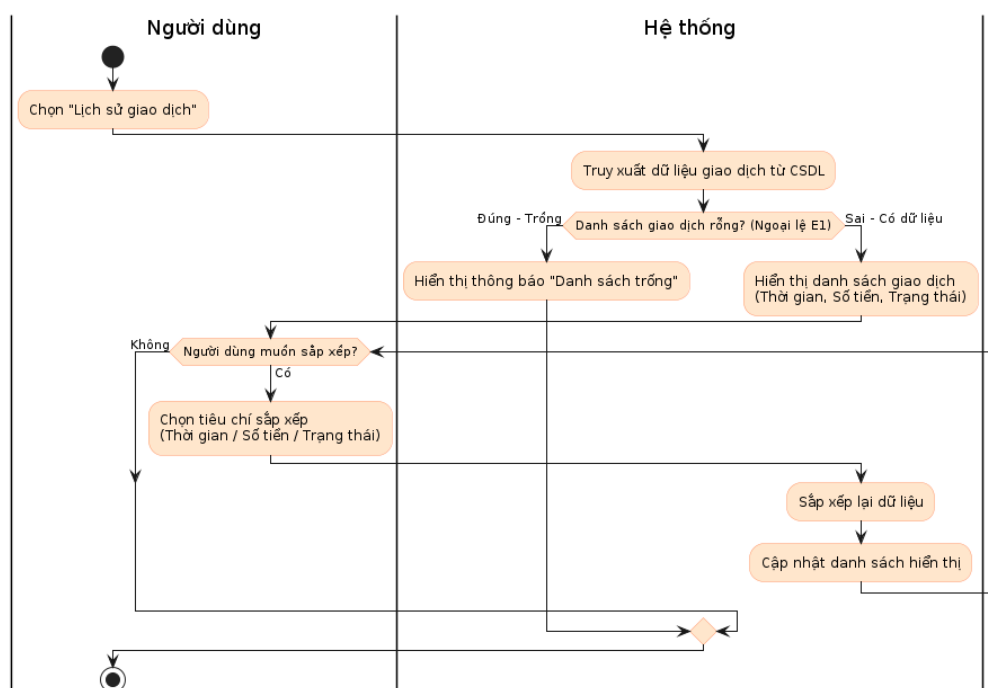
6.3 Nhóm thanh toán

6.3.1 Use-case U3.1: Xem lịch sử giao dịch cá nhân

Use-case ID	U3.1
Use-case name	Xem lịch sử giao dịch cá nhân
Use-case overview	Cho phép người dùng xem toàn bộ lịch sử giao dịch.
Actors	1. Khách gửi xe có tài khoản
Preconditions	1. Hệ thống kết nối tới các máy chủ đã xác thực và khả dụng. 2. Người dùng đã đăng nhập vào hệ thống.
Trigger	Người dùng chọn “Lịch sử giao dịch”.
Steps	1. Người dùng chọn mục "Lịch sử giao dịch" trên giao diện hệ thống. 2. Hệ thống gửi yêu cầu truy xuất dữ liệu giao dịch của người dùng đó từ cơ sở dữ liệu. 3. Cơ sở dữ liệu trả về danh sách các giao dịch thành công. 4. Hệ thống hiển thị danh sách giao dịch lên màn hình, bao gồm các thông tin: Thời gian, Số tiền, Trạng thái thanh toán. 5. (Tùy chọn) Người dùng chọn tính năng sắp xếp danh sách theo tiêu chí (Thời gian / Số tiền / Trạng thái). Hệ thống sắp xếp dữ liệu và tự động cập nhật lại hiển thị.
Post conditions	Lịch sử giao dịch được hiển thị thành công.
Exception flow	- E1 (Danh sách giao dịch trống) : Xảy ra tại Bước 3 của Steps. Khi cơ sở dữ liệu trả về kết quả rỗng (người dùng chưa từng thực hiện giao dịch nào). Hệ thống sẽ bỏ qua Bước 4 và 5, lập tức hiển thị thông báo "Bạn chưa có giao dịch nào" (hoặc "Danh sách trống") ra màn hình. Use-case kết thúc.



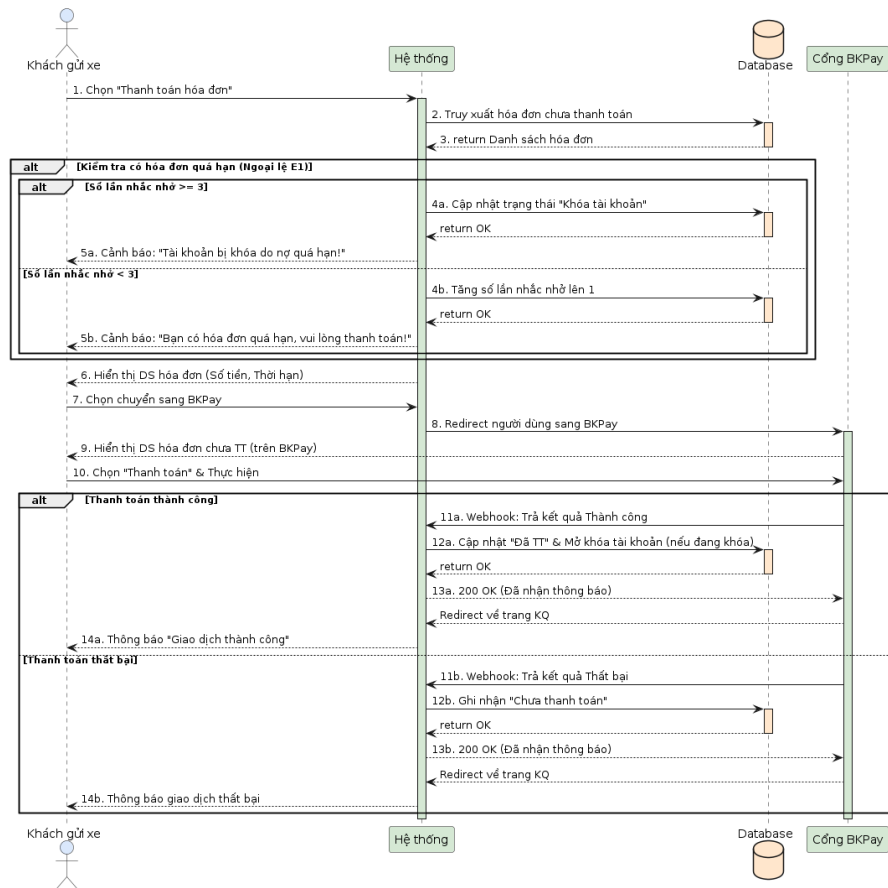
Hình 12: Sequence diagram cho Use-case U3.1: Xem lịch sử giao dịch cá nhân



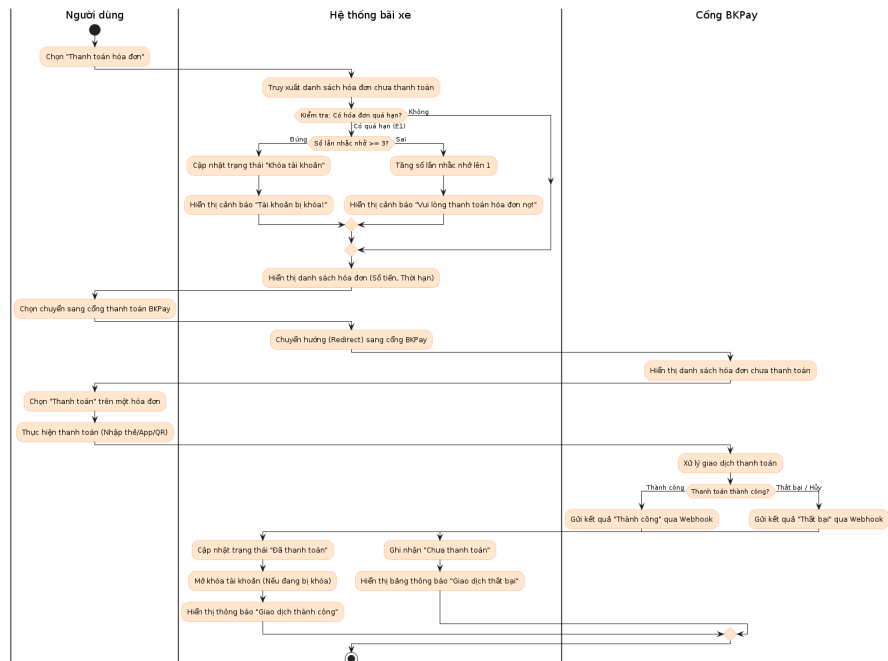
Hình 13: Activity diagram cho Use-case U3.1: Xem lịch sử giao dịch cá nhân

6.3.2 Use-case U3.2: Thanh toán hóa đơn

Use-case ID	U3.2
Use-case name	Thanh toán hóa đơn
Use-case overview	Cho phép người dùng thanh toán hóa đơn gửi xe thông qua cổng thanh toán BKPay.
Actors	1. Khách gửi xe có tài khoản. 2. BKPay.
Preconditions	1. Hệ thống kết nối tới các máy chủ đã xác thực và khả dụng 2. Người dùng đã đăng nhập 3. Hệ thống đã tính toán phí gửi xe và gửi yêu cầu thanh toán lên BKPay 4. Tồn tại hóa đơn chưa thanh toán.
Trigger	Người dùng chọn “Thanh toán hóa đơn” trên hệ thống.
Steps	1. Hệ thống hiển thị danh sách hóa đơn (gồm số tiền, thời hạn thanh toán). 2. Người dùng chọn chuyển sang BKPay. 3. BKPay hiển thị danh sách hóa đơn chưa thanh toán. 4. Người dùng chọn “Thanh toán” trên một hóa đơn. 5. Người dùng thanh toán trên BKPay. 6. BKPay trả kết quả về hệ thống: • Thành công → cập nhật trạng thái “Đã thanh toán”. • Thất bại → ghi nhận “Chưa thanh toán”.
Post conditions	1. Giao dịch được lưu vào lịch sử giao dịch. 2. Người dùng nhận thông báo kết quả “Giao dịch thành công”.
Exception flow	E1 (Hết hạn thanh toán): Gửi nhắc nhở 3 lần. Nếu vẫn chưa thanh toán thì khóa tài khoản. Dù tài khoản bị khóa không cho gửi xe nữa, nhưng hệ thống vẫn cho phép thanh toán để gỡ khóa.



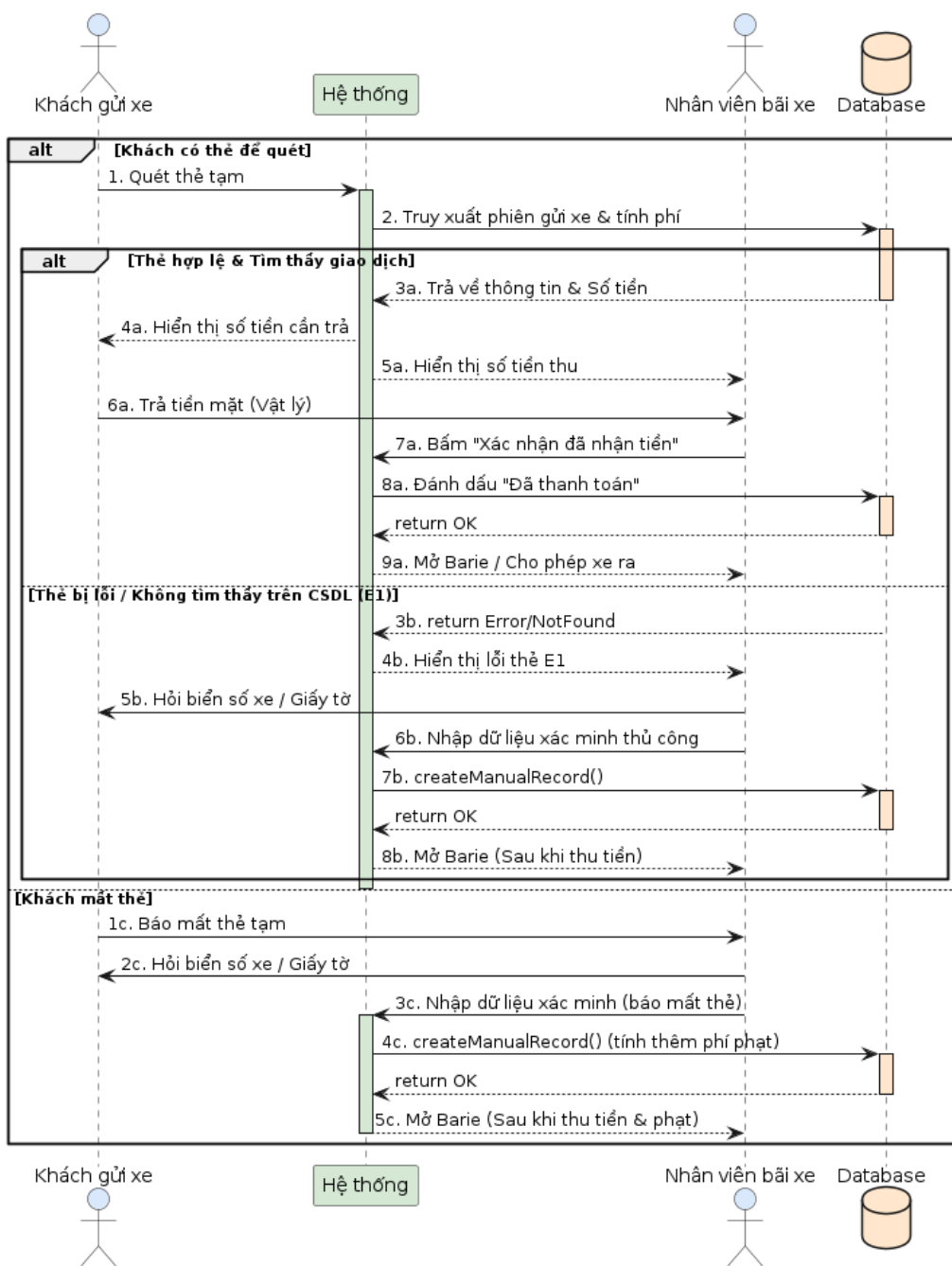
Hình 14: Sequence diagram cho Use-case U3.2: Thanh toán hóa đơn



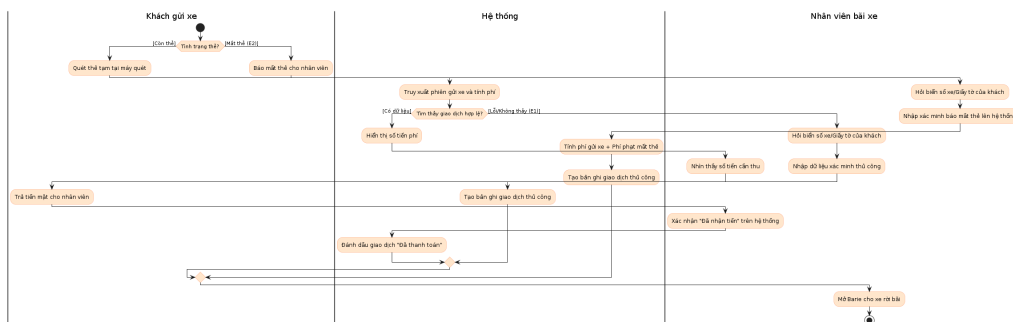
Hình 15: Activity diagram cho Use-case U3.2: Thanh toán hóa đơn

6.3.3 Use-case U3.3: Thanh toán thủ công

Use-case ID	U3.3
Use-case name	Thanh toán thủ công
Use-case overview	Cho phép thanh toán thủ công, hỗ trợ bởi nhân viên bãi xe.
Actors	1. Khách gửi xe có tài khoản hoặc không có tài khoản. 2. Nhân viên bãi xe
Preconditions	1. Người gửi xe chọn thanh toán thủ công khi vào cổng và được phát thẻ tạm. 2. Hệ thống ghi nhận giao dịch, biển số xe và thời gian bắt đầu gửi xe. 2. Hệ thống đã tính được phí gửi xe.
Trigger	Người gửi xe có mặt tại cổng ra.
Steps	1. Người gửi xe sử dụng thẻ tạm để quét vào hệ thống. 2. Hệ thống truy xuất phiên gửi xe và tính phí từ cơ sở dữ liệu. 3. Hệ thống hiển thị số tiền cần trả cho cả người gửi xe và nhân viên bãi xe nhìn thấy. 4. Người gửi xe trả tiền mặt (vật lý) cho nhân viên. 5. Nhân viên bấm “Xác nhận đã nhận tiền” trên hệ thống. 6. Hệ thống đánh dấu giao dịch “Đã thanh toán” vào cơ sở dữ liệu. 7. Hệ thống ra lệnh mở barie, cho phép xe rời bãi.
Post conditions	1. Giao dịch được đánh dấu đã thanh toán. 2. Xe được phép rời bãi.
Exception flow	- E1 (Thẻ bị lỗi / Không tìm thấy giao dịch): Xảy ra ở bước 2 nếu CSDL không tìm thấy thông tin hoặc thẻ hỏng. Hệ thống báo lỗi. Nhân viên hỏi thông tin (biển số/giấy tờ) của khách và nhập dữ liệu xác minh thủ công. Hệ thống tạo lập giao dịch thủ công. Chuyển sang bước thanh toán và mở barie. - E2 (Khách mất thẻ): Khách không thể thực hiện bước 1. Khách báo mất thẻ cho nhân viên. Nhân viên hỏi thông tin (biển số/giấy tờ) và nhập xác minh “báo mất thẻ” lên hệ thống. Hệ thống tạo giao dịch thủ công và tính cộng thêm phí phạt. Khách trả tiền và nhân viên mở barie.



Hình 16: Sequence diagram cho Use-case U3.3: Thanh toán thủ công

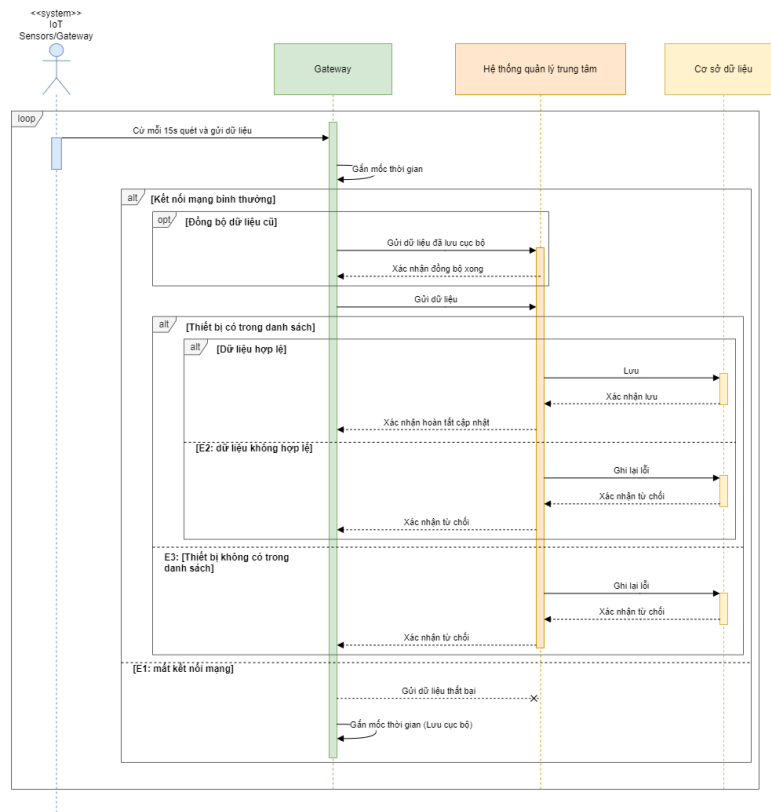


Hình 17: Activity diagram cho Use-case U3.3: Thanh toán thủ công

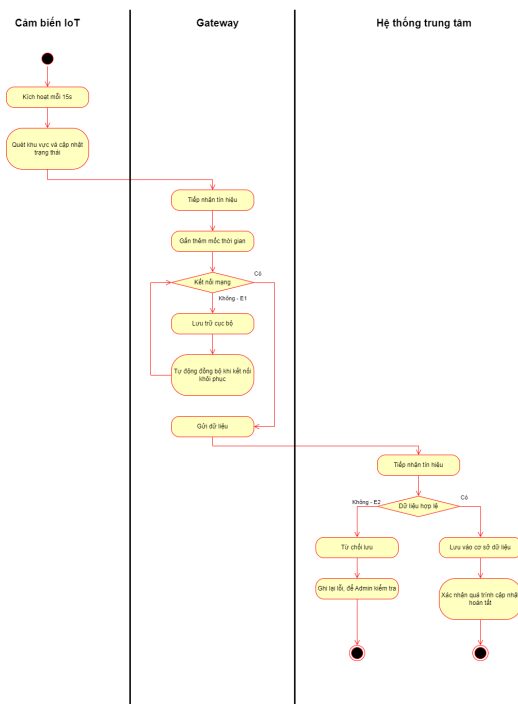
6.4 Nhóm IoT

6.4.1 Use-case U4.1: Cập nhật trạng thái vị trí đỗ

Use-case ID	U4.1
Use-case name	Cập nhật trạng thái vị trí đỗ
Use-case overview	Cho phép hệ thống IoT thu thập và cập nhật liên tục tình trạng của từng vị trí đỗ xe lên hệ thống trung tâm.
Actors	1. IoT Sensors/Gateway
Preconditions	1. Hệ thống đang hoạt động bình thường (không mất kết nối, không bị quá tải) 2. Các cảm biến IoT tại vị trí đỗ đã được cấp nguồn và có kết nối mạng ổn định tới Gateway.
Trigger	Cứ mỗi 15s cảm biến thu thập và gửi thông tin về sự thay đổi vật lý tại vị trí đỗ.
Steps	1. Cảm biến quét khu vực đỗ và cập nhật trạng thái. 2. IoT Gateway tiếp nhận tín hiệu từ cảm biến và gán thêm mốc thời gian cụ thể. 3. IoT Gateway truyền dữ liệu trạng thái cùng định danh của vị trí đỗ về hệ thống quản lý trung tâm. 4. Hệ thống trung tâm tiếp nhận, lưu trữ dữ liệu vào cơ sở dữ liệu và xác nhận quá trình cập nhật hoàn tất.
Post conditions	Trạng thái mới nhất của vị trí đỗ xe được lưu trữ chính xác trong cơ sở dữ liệu của hệ thống.
Exception flow	- E1 (Mất kết nối mạng) : IoT Gateway không thể gửi dữ liệu lên hệ thống. Dữ liệu trạng thái sẽ được lưu cục bộ tại Gateway và tự động đồng bộ lại ngay khi kết nối được khôi phục. - E2 (Dữ liệu không hợp lệ) : Nếu hệ thống phát hiện gói tin sai cấu trúc, hệ thống sẽ từ chối lưu bản ghi và tự động ghi lại lỗi để Admin kiểm tra. - E3 (Thiết bị không có trong danh sách) : Nếu hệ thống phát hiện gói tin được gửi từ một thiết bị nằm trong danh sách thiết bị, hệ thống sẽ từ chối lưu bản ghi và tự động ghi lại lỗi để Admin kiểm tra.



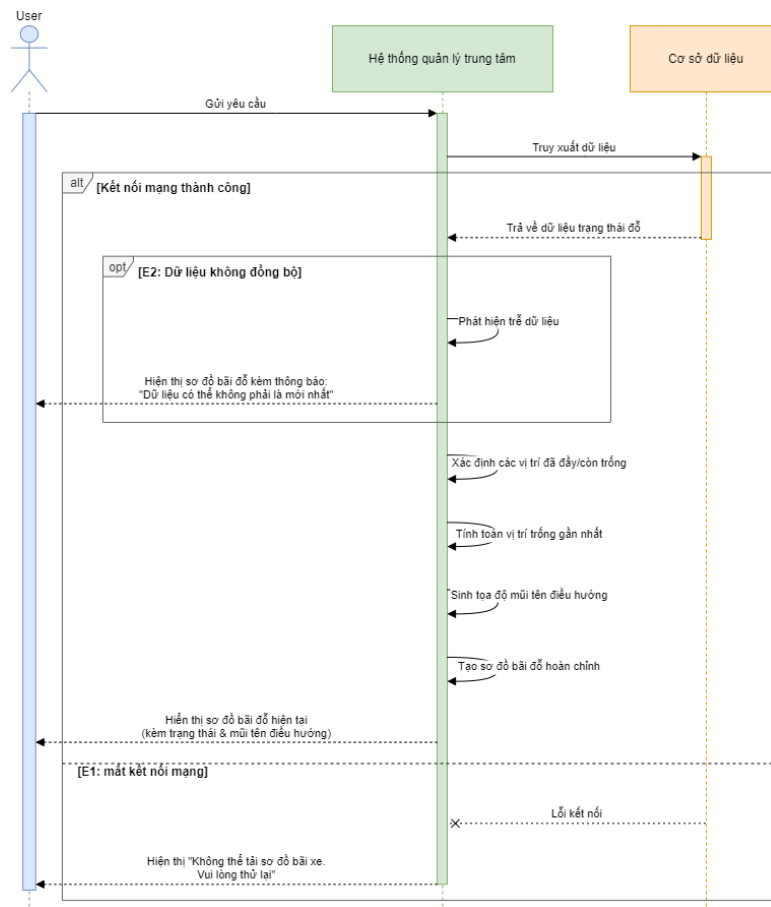
Hình 18: Sequence Diagram — U4.1: Cập nhật trạng thái vị trí đồ



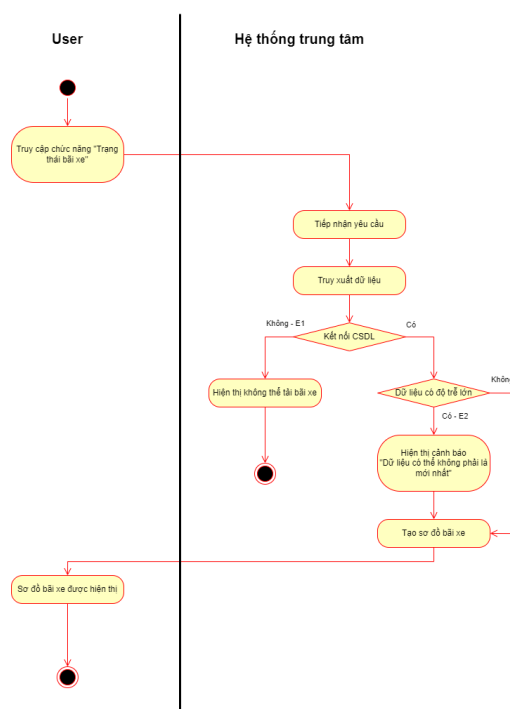
Hình 19: Activity Diagram — U4.1: Cập nhật trạng thái vị trí đồ

6.4.2 Use-case U4.2: **Hiển thị trạng thái bãi xe**

Use-case ID	U4.2
Use-case name	Hiển thị trạng thái bãi xe
Use-case overview	Cung cấp cho User cái nhìn tổng quan theo thời gian thực về tình trạng sức chứa của bãi xe (số chỗ trống, số chỗ đã đầy, sơ đồ vị trí).
Actors	1. User
Preconditions	1. Hệ thống đang hoạt động bình thường (không mất kết nối, không bị quá tải). 2. User đã đăng nhập thành công vào ứng dụng/tài khoản của bãi xe. 3. Hệ thống đã thu thập được dữ liệu trạng thái từ các IoT Gateway.
Trigger	User truy cập vào chức năng "Trạng thái bãi xe".
Steps	1. Hệ thống tiếp nhận yêu cầu xem trạng thái bãi xe từ User. 2. Hệ thống truy xuất cơ sở dữ liệu để lấy dữ liệu trạng thái mới nhất của tất cả các vị trí đỗ. 3. Hệ thống hiển thị chính xác của những vị trí chỗ trống/đã có xe và xuất lên giao diện dưới dạng sơ đồ bãi đỗ chứa các thông tin như: và mũi tên hướng dẫn đến vị trí đỗ xe gần nhất, trạng thái vị trí đỗ có trống hay không, bãi giữ xe có đang đầy hay không.
Post conditions	Giao diện màn hình của User hiển thị chính xác và trực quan trạng thái hiện tại của toàn bộ bãi đỗ xe.
Exception flow	- E1 (Lỗi kết nối với CSDL trung tâm): Hệ thống không thể lấy dữ liệu, hiển thị thông báo "Không thể tải sơ đồ bãi xe lúc này, vui lòng thử lại sau ít phút". - E2 (Dữ liệu không đồng bộ): Nếu có độ trễ lớn từ IoT Gateway, hệ thống sẽ hiển thị cảnh báo "Dữ liệu có thể không phải là mới nhất".



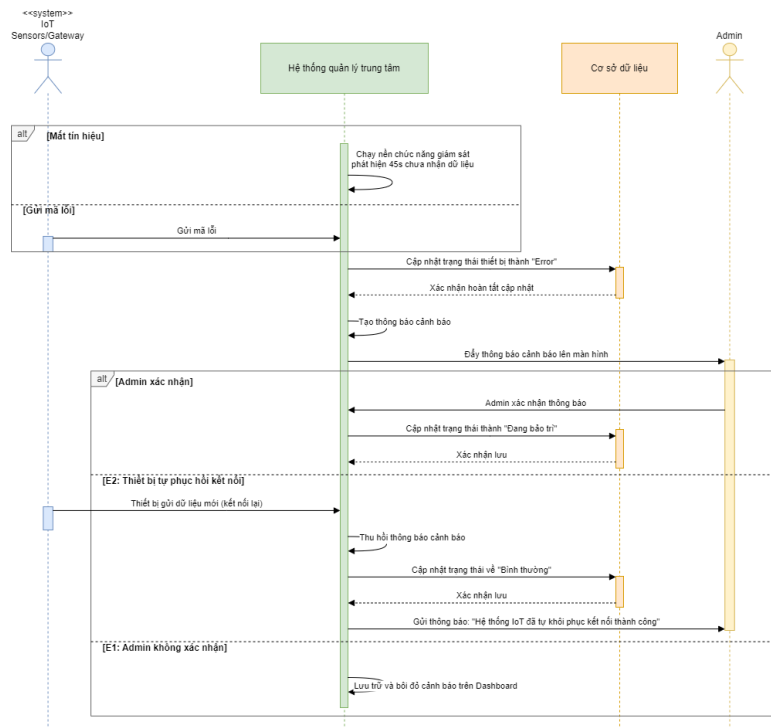
Hình 20: Sequence Diagram — U4.2: Hiển thị trạng thái bãi xe



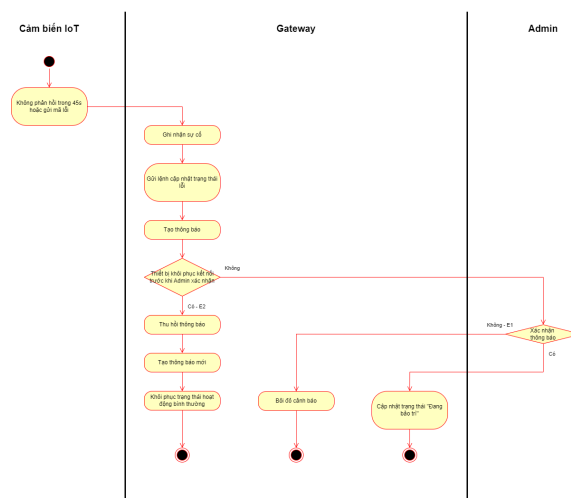
Hình 21: Activity Diagram — U4.2: Hiển thị trạng thái bãi xe

6.4.3 Use-case U4.3: Thông báo cảm biến lỗi

Use-case ID	U4.3
Use-case name	Thông báo cảm biến lỗi
Use-case overview	Hệ thống tự động theo dõi tình trạng hoạt động của các thiết bị IoT và cảnh báo cho Admin khi phát hiện thiết bị có dấu hiệu hư hỏng hoặc mất kết nối.
Actors	IoT Sensors/Gateway, Admin.
Preconditions	1. Chức năng giám sát tình trạng thiết bị (Heartbeat/Ping) của hệ thống đang chạy nền. 2. Toàn bộ thiết bị IoT đã được đăng ký mã định danh vào hệ thống.
Trigger	Hệ thống trung tâm không nhận được tín hiệu phản hồi từ cảm biến/Gateway trong khoảng thời gian cho phép (45s), hoặc nhận được mã lỗi từ thiết bị gửi về.
Steps	1. Hệ thống ghi nhận sự cố từ một thiết bị phần cứng cụ thể. 2. Hệ thống cập nhật trạng thái của thiết bị đó thành "Error" trong cơ sở dữ liệu. 3. Hệ thống tự động tạo một thông báo cảnh báo chi tiết (bao gồm mã thiết bị, vị trí, thời gian xảy ra sự cố). 4. Hệ thống đẩy thông báo hiển thị lên màn hình của Admin. 5. Admin xác nhận thông báo, cập nhật trạng thái "Đang bảo trì" cho thiết bị đó trong cơ sở dữ liệu.
Post conditions	Thông tin lỗi thiết bị được ghi vào hệ thống và Admin nhận được thông báo cảnh báo kịp thời.
Exception flow	- E1 : Nếu Admin không thực hiện việc xác nhận, cảnh báo vẫn sẽ được lưu trữ và bôi đỏ trực tiếp trên giao diện Dashboard. - E2 (Thiết bị tự phục hồi kết nối) : Nếu thiết bị bị mất kết nối mạng tạm thời nhưng sau đó tự kết nối lại được trước khi Admin kịp xác nhận báo lỗi, hệ thống tự động thu hồi thông báo lỗi ban đầu và thay bằng một thông báo mới "Hệ thống IoT đã tự khôi phục kết nối thành công".



Hình 22: Sequence Diagram — U4.3: Thông báo cảm biến lỗi



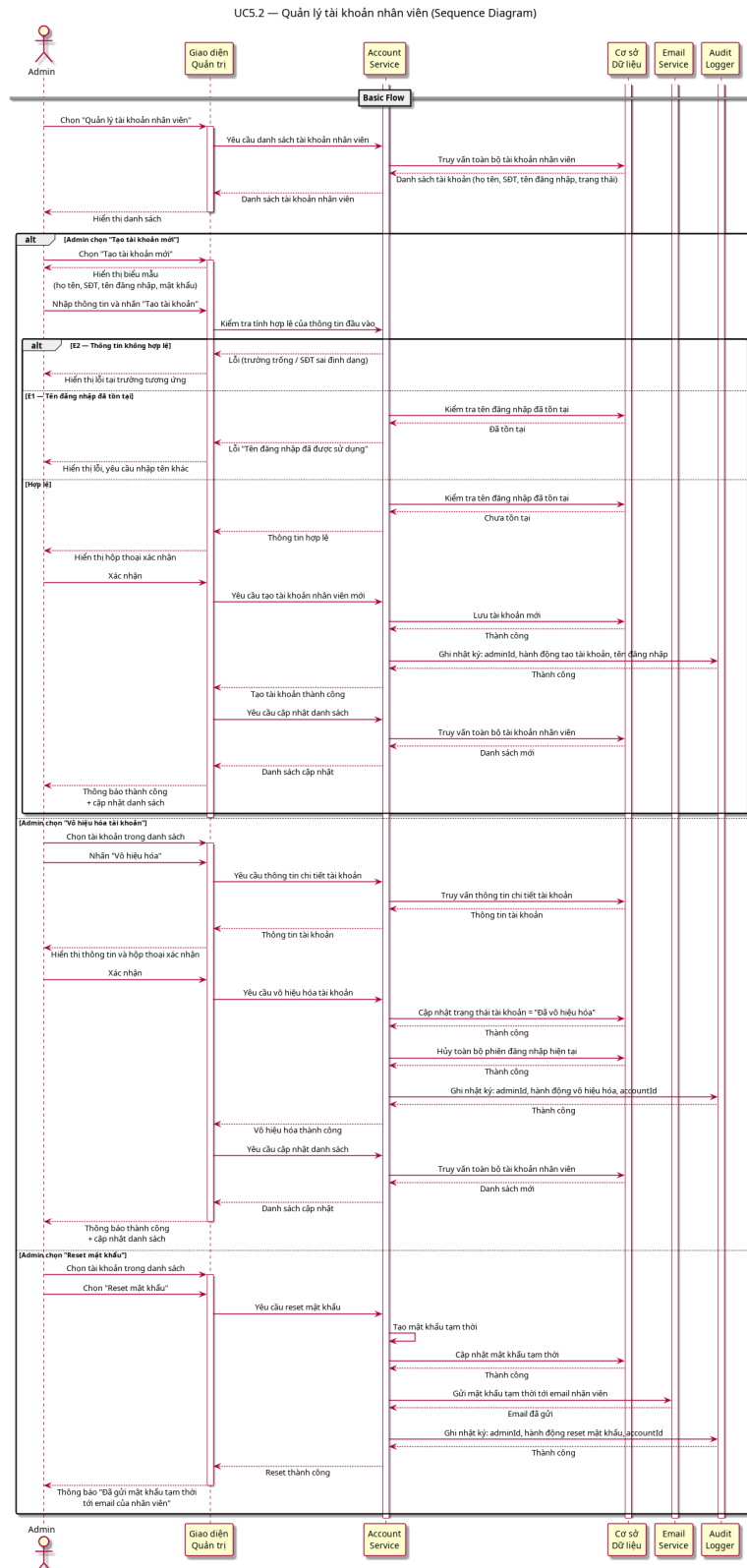
Hình 23: Activity Diagram — U4.3: Thông báo cảm biến lỗi

6.5 Nhóm Admin

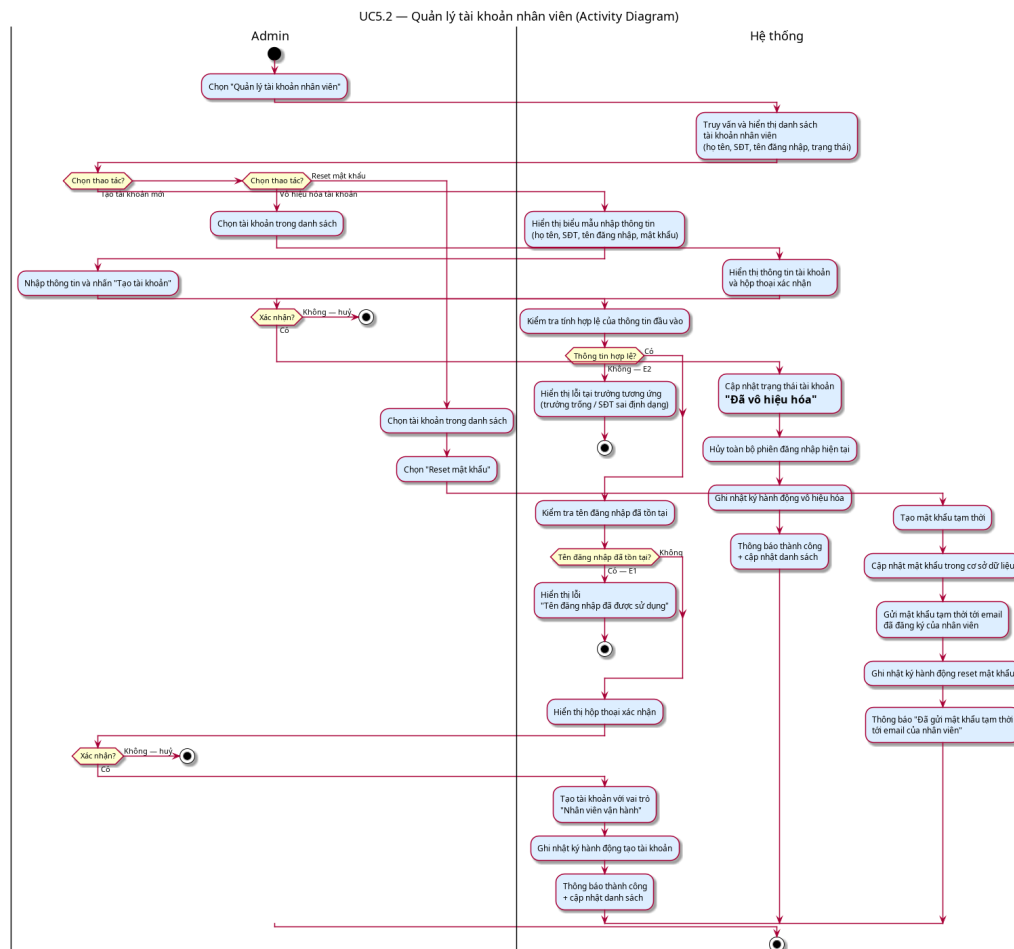
6.5.1 Use-case 5.2: Quản lý tài khoản nhân viên

Use-case ID	U5.2
Use-case name	Quản lý tài khoản nhân viên
Use-case overview	Cho phép quản trị viên tạo mới, vô hiệu hóa và reset mật khẩu tài khoản đăng nhập cho nhân viên vận hành bãi đỗ xe.
Actors	Primary: Admin
Internal Components	Account Service: Xử lý logic tạo, vô hiệu hóa, mã hóa và quản lý vòng đời tài khoản nhân viên. Email Service: Gửi thông báo và mật khẩu tạm thời tới địa chỉ email đã đăng ký của nhân viên. Audit Logger: Hệ thống ghi vết, đảm bảo tính toàn vẹn và minh bạch của mọi hành động thay đổi dữ liệu từ Admin. Cơ sở Dữ liệu: Lưu trữ toàn bộ dữ liệu nghiệp vụ, tiếp nhận các yêu cầu đọc/ghi từ các service nội bộ thông qua lớp truy cập dữ liệu.
Preconditions	- Admin đã đăng nhập và được phân quyền quản trị hệ thống. - Kết nối cơ sở dữ liệu khả dụng.
Trigger	Admin chọn mục “Quản lý tài khoản nhân viên” trên giao diện quản trị.
Steps	- Giao diện Quản trị gửi yêu cầu lấy danh sách tài khoản tới Account Service. Account Service truy vấn Cơ sở Dữ liệu và trả về danh sách tài khoản nhân viên hiện có, bao gồm: họ tên, số điện thoại, tên đăng nhập, trạng thái (đang hoạt động / đã vô hiệu hóa). - Admin chọn “Tạo tài khoản mới”. - Giao diện Quản trị hiển thị biểu mẫu yêu cầu nhập họ tên, số điện thoại, tên đăng nhập và mật khẩu. - Admin nhập thông tin và nhấn “Tạo tài khoản”. - Account Service xác thực thông tin đầu vào (kiểm tra trường bắt buộc, định dạng số điện thoại) và kiểm tra tính duy nhất của tên đăng nhập trong Cơ sở Dữ liệu. Nếu hợp lệ, Giao diện Quản trị hiển thị hộp thoại yêu cầu Admin xác nhận thao tác. - Admin xác nhận. - Account Service tạo bản ghi tài khoản mới trong Cơ sở Dữ liệu với vai trò “Nhân viên vận hành”, đồng thời gửi yêu cầu ghi nhật ký tới Audit Logger (bao gồm adminId, hành động tạo tài khoản, tên đăng nhập đích, timestamp). - Giao diện Quản trị nhận xác nhận thành công, tải lại danh sách tài khoản từ Account Service và hiển thị thông báo kết quả.

Use-case ID	U5.2
Use-case name	Quản lý tài khoản nhân viên
Alternative flows	AF1 - Vô hiệu hóa tài khoản (tại Step 2): 2.1. Thay vì tạo mới, Admin chọn một tài khoản đã có trong danh sách. 2.2. Admin nhấn “Vô hiệu hóa”. 2.3. Account Service trả về thông tin tài khoản được chọn. Giao diện Quản trị hiển thị thông tin và yêu cầu xác nhận. 2.4. Admin xác nhận. 2.5. Account Service cập nhật trạng thái tài khoản thành “Đã vô hiệu hóa” trong Cơ sở Dữ liệu và gửi yêu cầu hủy phiên đăng nhập hiện tại của nhân viên đó (nếu có). Audit Logger ghi nhận hành động vô hiệu hóa (adminId, hành động vô hiệu hóa tài khoản, targetAccountId, timestamp). 2.6. Tiếp tục Step 8 của Basic Flow. AF2 - Reset mật khẩu nhân viên (tại Step 2): 2.1. Admin chọn một tài khoản đã có trong danh sách. 2.2. Admin chọn “Reset mật khẩu”. 2.3. Account Service tạo mật khẩu tạm thời, cập nhật Cơ sở Dữ liệu và gửi yêu cầu gửi mật khẩu tạm thời tới Email Service. Email Service gửi mật khẩu đến địa chỉ email đã đăng ký của nhân viên. 2.4. Audit Logger ghi nhận hành động đặt lại mật khẩu (adminId, hành động reset mật khẩu, targetAccountId, timestamp). Giao diện Quản trị hiển thị thông báo “Đã gửi mật khẩu tạm thời tới email của nhân viên”. 2.5. UC kết thúc thành công.
Post conditions	Tài khoản nhân viên được tạo mới, vô hiệu hóa hoặc reset mật khẩu thành công trong Cơ sở Dữ liệu. Danh sách tài khoản nhân viên được cập nhật phản ánh đúng trạng thái hiện tại.
Exception flow	- E1 (Tên đăng nhập đã tồn tại): Nếu Account Service phát hiện tên đăng nhập trùng với tài khoản đã có, Account Service trả về lỗi. Giao diện Quản trị hiển thị thông báo “Tên đăng nhập đã được sử dụng” và yêu cầu nhập tên khác. - E2 (Thông tin không hợp lệ): Nếu Account Service phát hiện trường bắt buộc bị bỏ trống hoặc số điện thoại sai định dạng, Account Service trả về lỗi validation. Giao diện Quản trị hiển thị lỗi tại trường tương ứng.



Hình 24: Sequence Diagram — UC5.2: Quản lý tài khoản nhân viên

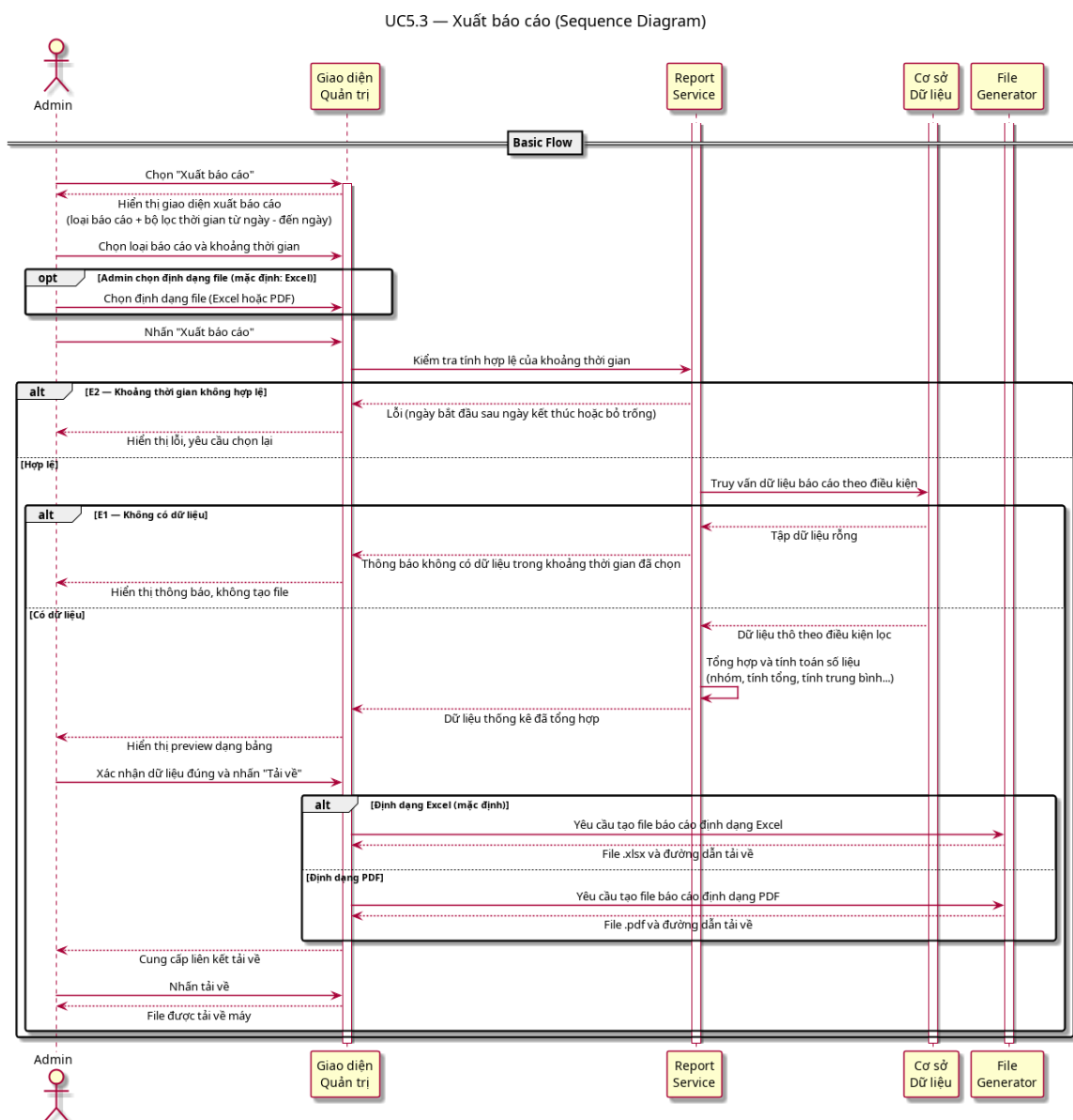


Hình 25: Activity Diagram — UC5.2: Quản lý tài khoản nhân viên

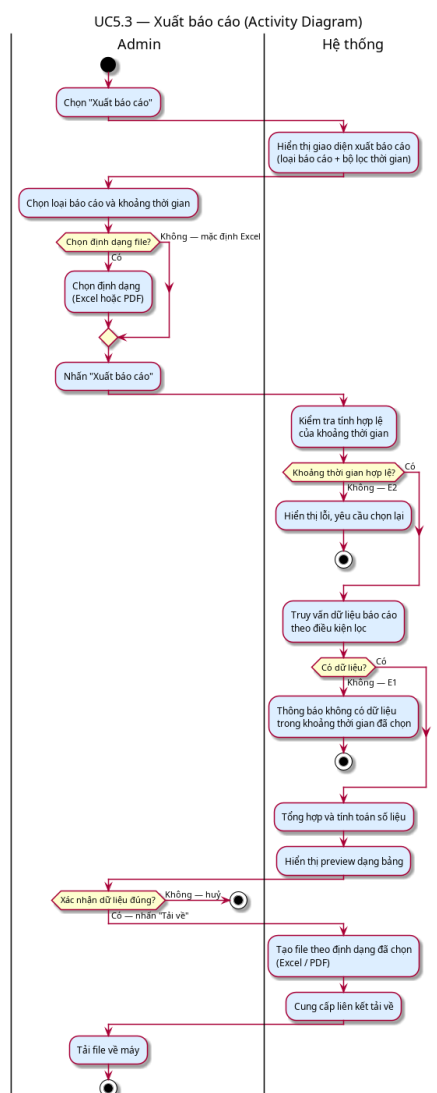
6.5.2 Use-case 5.3: Xuất báo cáo

Use-case ID	U5.3
Use-case name	Xuất báo cáo
Use-case overview	Cho phép quản trị viên tạo và tải về các báo cáo thống kê <i>đã được tổng hợp và tính toán</i> về hoạt động gửi xe, doanh thu, tình trạng bãi đỗ xe và hoạt động nhân viên, phục vụ công tác kiểm tra tài chính và đối soát định kỳ.
Actors	Primary: Admin
Internal Components	Report Service: Tổng hợp dữ liệu thô từ Cơ sở Dữ liệu thành các chỉ số thống kê (group by, sum, avg). File Generator: Chuyển đổi dữ liệu thống kê thành các định dạng file vật lý (Excel/PDF). Cơ sở Dữ liệu: Lưu trữ toàn bộ dữ liệu nghiệp vụ, tiếp nhận các yêu cầu đọc/ghi từ các service nội bộ thông qua lớp truy cập dữ liệu.
Preconditions	1. Admin đã đăng nhập và được phân quyền quản trị hệ thống. 2. Hệ thống đã ghi nhận dữ liệu hoạt động (lượt gửi xe, giao dịch thanh toán, nhật ký vận hành).
Trigger	Admin chọn mục “Xuất báo cáo” trên giao diện quản trị.
Steps	1. Giao diện Quản trị hiển thị giao diện xuất báo cáo với các tùy chọn: loại báo cáo (lượt gửi xe, doanh thu, tình trạng bãi đỗ, hoạt động nhân viên) và bộ lọc thời gian (từ ngày - đến ngày). 2. Admin chọn loại báo cáo và khoảng thời gian mong muốn. 3. Admin nhấn “Xuất báo cáo”. 4. Giao diện Quản trị gửi yêu cầu tới Report Service kèm điều kiện lọc. Report Service xác thực khoảng thời gian đầu vào, sau đó truy vấn dữ liệu thô từ Cơ sở Dữ liệu và thực hiện tổng hợp, tính toán các chỉ số thống kê. 5. Report Service trả về dữ liệu thống kê đã tổng hợp. Giao diện Quản trị hiển thị preview dữ liệu dạng bảng để Admin xem xét trước khi tải về. 6. Admin xác nhận dữ liệu đúng và nhấn “Tải về”. 7. Giao diện Quản trị gửi yêu cầu tạo file tới File Generator. File Generator nhận dữ liệu thống kê từ Report Service và tạo file theo định dạng đã chọn (mặc định: Excel). Giao diện Quản trị cung cấp liên kết tải về.

Use-case ID	U5.3
Use-case name	Xuất báo cáo
Alternative flows	AF1 - Admin chọn định dạng xuất (tại Step 3): 3.1. Trước khi nhấn “Xuất báo cáo”, Admin chọn định dạng file mong muốn: Excel hoặc PDF. 3.2. Giao diện Quản trị ghi nhận định dạng đã chọn và chuyển thông tin này đến File Generator ở Step 7. 3.3. Tiếp tục Step 4.
Post conditions	File báo cáo được tải về máy admin thành công. Dữ liệu hệ thống không bị thay đổi.
Exception flow	- E1 (Không có dữ liệu): Nếu Report Service truy vấn Cơ sở Dữ liệu với khoảng thời gian đã chọn và nhận về tập rỗng, Report Service trả về trạng thái không có dữ liệu. Giao diện Quản trị hiển thị thông báo “Không có dữ liệu trong khoảng thời gian đã chọn” và không tạo file. - E2 (Khoảng thời gian không hợp lệ): Nếu Report Service phát hiện fromDate lớn hơn toDate hoặc Admin bỏ trống trường bắt buộc, Report Service trả về lỗi validation. Giao diện Quản trị hiển thị lỗi và yêu cầu chọn lại.



Hình 26: Sequence Diagram — UC5.3: Xuất báo cáo

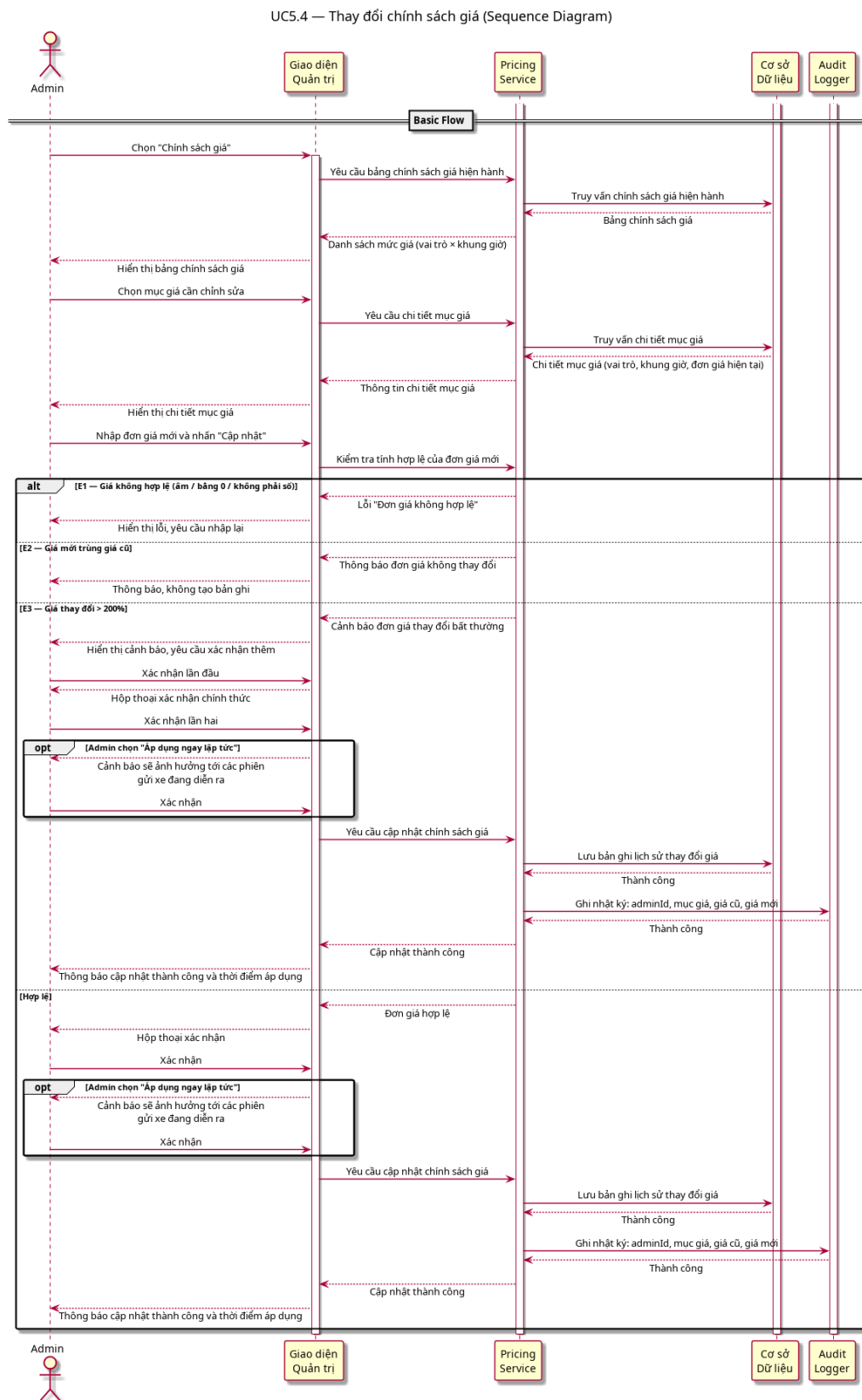


Hình 27: Activity Diagram — UC5.3: Xuất báo cáo

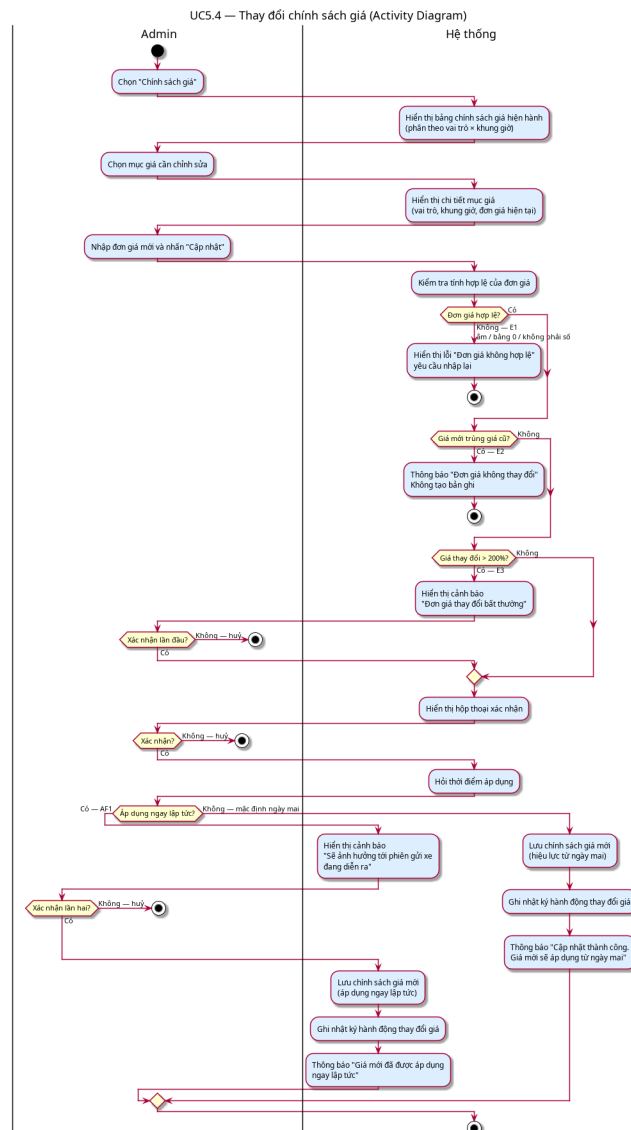
6.5.3 Use-case 5.4: Thay đổi chính sách giá

Use-case ID	U5.4
Use-case name	Thay đổi chính sách giá
Use-case overview	Cho phép quản trị viên cấu hình và cập nhật đơn giá gửi xe theo vai trò người dùng và khung giờ, phục vụ việc điều chỉnh chính sách giá theo quy định của trường.
Actors	Primary: Admin
Internal Components	Pricing Service: Quản lý bảng giá, xử lý logic về thời điểm áp dụng giá (effective date). Audit Logger: Hệ thống ghi vết, đảm bảo tính toàn vẹn và minh bạch của mọi hành động thay đổi dữ liệu từ Admin. Cơ sở Dữ liệu: Lưu trữ toàn bộ dữ liệu nghiệp vụ, tiếp nhận các yêu cầu đọc/ghi từ các service nội bộ thông qua lớp truy cập dữ liệu.
Preconditions	1. Admin đã đăng nhập và được phân quyền quản trị hệ thống. 2. Hệ thống đã có bảng chính sách giá hiện hành.
Trigger	Admin chọn mục “Chính sách giá” trên giao diện quản trị.
Steps	1. Giao diện Quản trị gửi yêu cầu lấy bảng chính sách giá tới Pricing Service. Pricing Service truy vấn Cơ sở Dữ liệu và trả về bảng chính sách giá hiện hành, phân theo vai trò (sinh viên, giảng viên, cán bộ, khách vãng lai) và khung giờ (6h30-18h, sau 18h, qua đêm). 2. Admin chọn mục giá cần chỉnh sửa. 3. Pricing Service truy vấn Cơ sở Dữ liệu và trả về thông tin chi tiết của mục giá đó bao gồm: vai trò áp dụng, khung giờ, đơn giá hiện tại. 4. Admin nhập đơn giá mới và nhấn “Cập nhật”. 5. Pricing Service kiểm tra tính hợp lệ của đơn giá mới. Nếu hợp lệ, Giao diện Quản trị yêu cầu Admin xác nhận thay đổi. 6. Admin xác nhận. 7. Pricing Service lưu chính sách giá mới vào Cơ sở Dữ liệu với effectiveDate = ngày tiếp theo, đồng thời gửi yêu cầu ghi nhật ký tới Audit Logger (bao gồm adminId, pricingId, giá cũ, giá mới, timestamp). Giao diện Quản trị hiển thị thông báo “Cập nhật chính sách giá thành công. Giá mới sẽ được áp dụng từ ngày mai”.

Use-case ID	U5.4
Use-case name	Thay đổi chính sách giá
Alternative flows	AF1 - Admin áp dụng giá ngay lập tức (tại Step 6): 6.1. Sau khi Admin xác nhận ở Step 6, Giao diện Quản trị hỏi: “Áp dụng từ ngày mai (mặc định) hay áp dụng ngay lập tức?”. 6.2. Admin chọn “Áp dụng ngay”. 6.3. Giao diện Quản trị hiển thị cảnh báo: “Thao tác này sẽ ảnh hưởng tới các phiên gửi xe đang diễn ra. Bạn có chắc chắn?” và yêu cầu xác nhận lần hai. 6.4. Admin xác nhận lần hai. 6.5. Pricing Service lưu chính sách giá mới vào Cơ sở Dữ liệu với <code>effectiveDate</code> = thời điểm hiện tại và trường <code>effective_immediately</code> = <code>true</code>. Audit Logger ghi nhận bản ghi thay đổi với trường <code>effective_immediately</code> = <code>true</code>. 6.6. Tiếp tục Step 7 của Basic Flow với thông báo điều chỉnh: “Giá mới đã được áp dụng ngay lập tức”.
Post conditions	Chính sách giá mới được lưu trong Cơ sở Dữ liệu và sẽ tự động áp dụng từ 0h00 ngày tiếp theo (hoặc ngay lập tức nếu chọn AF1). Chính sách giá cũ vẫn có hiệu lực trong ngày hiện tại trừ khi AF1 được kích hoạt. Lịch sử thay đổi được Audit Logger ghi nhận đầy đủ.
Exception flow	- E1 (Giá trị không hợp lệ): Nếu Pricing Service phát hiện đơn giá âm, bằng 0, hoặc không phải số, Pricing Service trả về lỗi validation. Giao diện Quản trị hiển thị thông báo “Đơn giá không hợp lệ” và yêu cầu nhập lại. - E2 (Giá mới trùng giá cũ): Nếu Pricing Service phát hiện đơn giá mới giống hết đơn giá hiện tại, Pricing Service trả về trạng thái không thay đổi. Giao diện Quản trị thông báo “Đơn giá không thay đổi” và không tạo bản ghi nhật ký. - E3 (Giá trị bất thường): Nếu Pricing Service phát hiện đơn giá mới chênh lệch hơn 200% so với đơn giá hiện tại, Pricing Service đẩy cảnh báo “Đơn giá thay đổi bất thường” lên Giao diện Quản trị và yêu cầu Admin xác nhận thêm một lần nữa trước khi lưu.



Hình 28: Sequence Diagram — UC5.4: Thay đổi chính sách giá



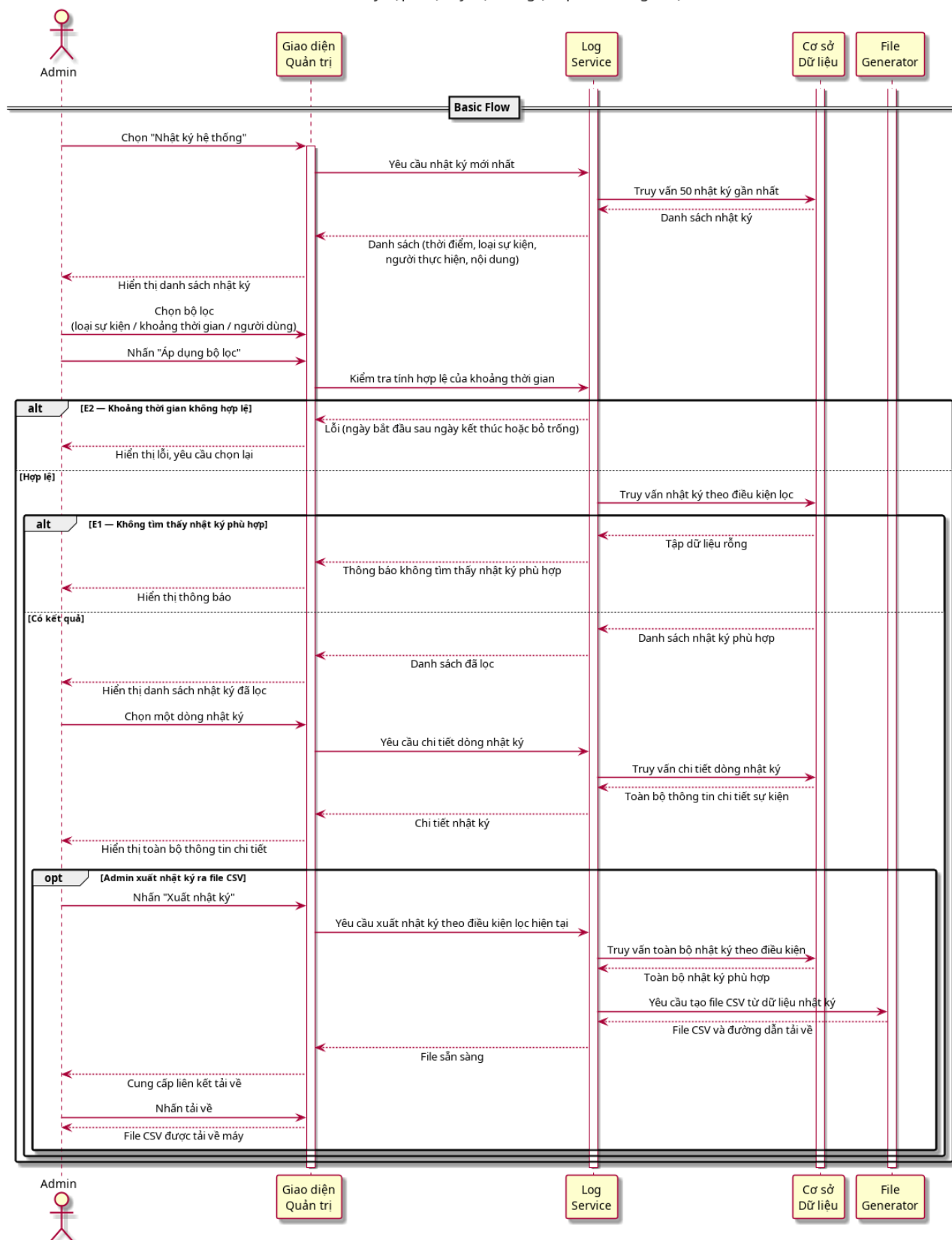
Hình 29: Activity Diagram — UC5.4: Thay đổi chính sách giá

6.5.4 Use-case 5.5: Truy cập nhật ký hệ thống

Use-case ID	U5.5
Use-case name	Truy cập nhật ký hệ thống
Use-case overview	Cho phép quản trị viên xem trực tiếp nhật ký <i>raw</i> về các hành động và sự kiện trong hệ thống - bao gồm lượt ra/vào bãi xe, giao dịch thanh toán, thay đổi chính sách giá, đăng nhập và thao tác của nhân viên vận hành - phục vụ mục đích điều tra sự cố và kiểm toán.
Actors	Primary: Admin
Internal Components	Log Service: Quản lý và truy vấn nhật ký hệ thống, hỗ trợ lọc theo nhiều chiều (loại sự kiện, khoảng thời gian, người dùng). File Generator: Chuyển đổi dữ liệu nhật ký thành file CSV để xuất về máy Admin. Cơ sở Dữ liệu: Lưu trữ toàn bộ dữ liệu nghiệp vụ, tiếp nhận các yêu cầu đọc/ghi từ các service nội bộ thông qua lớp truy cập dữ liệu.
Preconditions	- Admin đã đăng nhập và được phân quyền quản trị hệ thống. - Hệ thống đã ghi nhận dữ liệu nhật ký hoạt động.
Trigger	Admin chọn mục “Nhật ký hệ thống” trên giao diện quản trị.
Steps	- Giao diện Quản trị gửi yêu cầu lấy nhật ký mới nhất tới Log Service. Log Service truy vấn Cơ sở Dữ liệu (sắp xếp giảm dần theo timestamp, giới hạn 50 bản ghi) và trả về danh sách nhật ký, mỗi dòng gồm: thời điểm, loại sự kiện, người thực hiện, nội dung chi tiết. - Admin sử dụng bộ lọc để thu hẹp kết quả theo: loại sự kiện (lượt gửi xe, giao dịch, thay đổi giá, đăng nhập, thao tác nhân viên), khoảng thời gian (từ ngày - đến ngày), hoặc người dùng cụ thể. - Admin nhấn “Áp dụng bộ lọc”. - Giao diện Quản trị gửi yêu cầu kiểm tra tính hợp lệ của khoảng thời gian tới Log Service. Log Service xác thực đầu vào, sau đó truy vấn Cơ sở Dữ liệu theo điều kiện lọc đã nhập và trả về danh sách nhật ký phù hợp. Giao diện Quản trị hiển thị kết quả. - Admin chọn một dòng nhật ký để xem chi tiết. - Giao diện Quản trị gửi yêu cầu chi tiết sự kiện tới Log Service. Log Service truy vấn chi tiết theo logId từ Cơ sở Dữ liệu và trả về toàn bộ thông tin. Giao diện Quản trị hiển thị.

Use-case ID	U5.5
Use-case name	Truy cập nhật ký hệ thống
Alternative flows	AF1 - Export nhật ký đã lọc ra file (tại Step 4): 4.1. Sau khi Log Service trả về kết quả lọc, Admin chọn “Xuất nhật ký” thay vì đọc từng dòng. 4.2. Giao diện Quản trị gửi yêu cầu xuất nhật ký tới Log Service. Log Service truy vấn toàn bộ nhật ký phù hợp với điều kiện lọc hiện tại và chuyển dữ liệu tới File Generator. 4.3. File Generator tạo file CSV và trả về đường dẫn tải về. Giao diện Quản trị cung cấp liên kết tải về. 4.4. UC kết thúc thành công mà không cần Admin thực hiện Step 5-6.
Post conditions	Nhật ký hệ thống được hiển thị thành công trên màn hình. Dữ liệu nhật ký là append-only và không thể bị sửa đổi hoặc xóa bởi bất kỳ actor nào, kể cả Admin.
Exception flow	- E1 (Không có dữ liệu): Nếu Log Service truy vấn Cơ sở Dữ liệu với điều kiện lọc hiện tại và nhận về tập rỗng, Log Service trả về trạng thái không có kết quả. Giao diện Quản trị hiển thị thông báo “Không tìm thấy nhật ký phù hợp với điều kiện lọc”. - E2 (Khoảng thời gian không hợp lệ): Nếu Log Service phát hiện fromDate lớn hơn toDate hoặc bỏ trống, Log Service trả về lỗi validation. Giao diện Quản trị hiển thị lỗi và yêu cầu chọn lại.

UC5.5 — Truy cập nhật ký hệ thống (Sequence Diagram)



Hình 30: Sequence Diagram — UC5.5: Truy cập nhật ký hệ thống



Hình 31: Activity Diagram — UC5.5: Truy cập nhật ký hệ thống

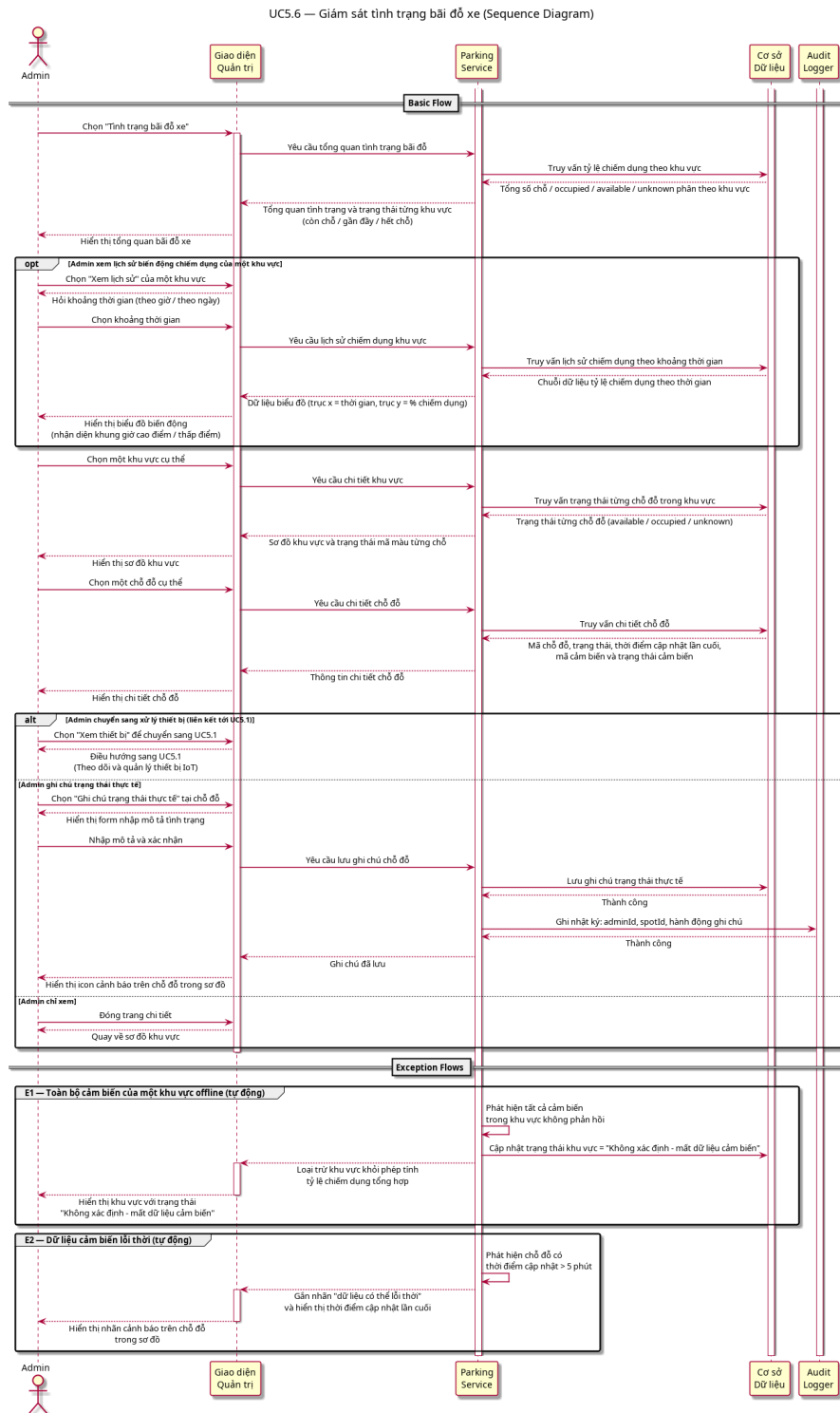
6.5.5 Use-case 5.6: Giám sát tình trạng bãi đỗ xe

Use-case ID	U5.6
Use-case name	Giám sát tình trạng bãi đỗ xe
Use-case overview	Cho phép quản trị viên theo dõi tình trạng chiếm dụng chỗ đỗ xe theo thời gian thực trên toàn bộ hệ thống, xem chi tiết từng khu vực và từng chỗ đỗ cụ thể, phục vụ công tác điều phối và xử lý sự cố liên quan đến sức chứa bãi đỗ.
Actors	1. Primary: Admin 2. Secondary: IoT Sensor (thông qua Gateway, cung cấp dữ liệu chiếm dụng của từng chỗ đỗ)
Internal Components	Parking Service: Quản lý và tổng hợp dữ liệu chiếm dụng chỗ đỗ xe theo thời gian thực từ cảm biến IoT, hỗ trợ truy vấn theo khu vực và từng chỗ đỗ cụ thể. Audit Logger: Hệ thống ghi vết, đảm bảo tính toàn vẹn và minh bạch của mọi hành động thay đổi dữ liệu từ Admin. Cơ sở Dữ liệu: Lưu trữ toàn bộ dữ liệu nghiệp vụ, tiếp nhận các yêu cầu đọc/ghi từ các service nội bộ thông qua lớp truy cập dữ liệu.
Preconditions	1. Admin đã đăng nhập và được phân quyền quản trị hệ thống. 2. Ít nhất một gateway đang kết nối và truyền dữ liệu cảm biến về hệ thống trung tâm. 3. Dữ liệu chiếm dụng chỗ đỗ đã được hệ thống tổng hợp từ các cảm biến.
Trigger	Admin chọn mục “Tình trạng bãi đỗ xe” trên giao diện quản trị.

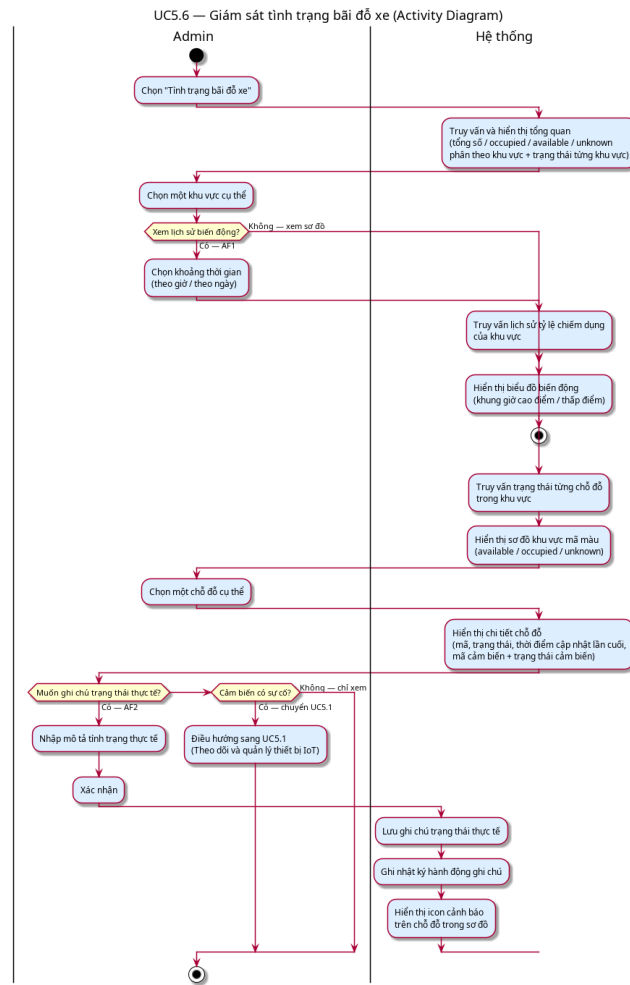
Use-case ID	U5.6
Use-case name	Giám sát tình trạng bãi đỗ xe
Steps	1. Giao diện Quản trị gửi yêu cầu tổng quan tình trạng bãi đỗ tới Parking Service. Parking Service truy vấn Cơ sở Dữ liệu và trả về tổng quan chiếm dụng, bao gồm: tổng số chỗ đỗ toàn hệ thống, số chỗ đang có xe (occupied), số chỗ trống (available), số chỗ không xác định được trạng thái (do cảm biến lỗi hoặc offline) - phân chia theo từng khu vực bãi đỗ, kèm trạng thái tổng hợp mỗi khu vực (còn chỗ / gần đầy / hết chỗ). 2. Admin chọn một khu vực bãi đỗ cụ thể để xem chi tiết. 3. Parking Service truy vấn Cơ sở Dữ liệu và trả về trạng thái từng chỗ đỗ trong khu vực được chọn. Giao diện Quản trị hiển thị sơ đồ khu vực với trạng thái từng chỗ đỗ được mã hóa bằng màu sắc: trống (available), có xe (occupied), không xác định (cảm biến offline hoặc báo lỗi). 4. Admin chọn một chỗ đỗ cụ thể trên sơ đồ. 5. Parking Service truy vấn Cơ sở Dữ liệu và trả về thông tin chỗ đỗ đó, bao gồm: mã chỗ đỗ, trạng thái hiện tại, thời điểm cập nhật trạng thái lần cuối, và mã cảm biến liên kết kèm trạng thái hoạt động của cảm biến đó.

Use-case ID	U5.6
Use-case name	Giám sát tình trạng bãi đỗ xe
Alternative flows	AF1 - Admin xem lịch sử biến động chiếm dụng của một khu vực (tại Step 2): 2.1. Thay vì xem sơ đồ trực tiếp, Admin chọn “Xem lịch sử” của khu vực đó. 2.2. Admin chọn khoảng thời gian cần xem (theo giờ hoặc theo ngày). 2.3. Parking Service truy vấn Cơ sở Dữ liệu và trả về chuỗi dữ liệu lịch sử chiếm dụng theo khu vực và khoảng thời gian đã chọn. Giao diện Quản trị hiển thị biểu đồ biến động tỷ lệ chiếm dụng theo trục thời gian, giúp Admin nhận diện khung giờ cao điểm và thấp điểm. 2.4. Admin xem xét và kết thúc UC, hoặc quay lại Step 2 để xem khu vực khác. AF2 - Admin ghi chú thủ công trạng thái một chỗ đỗ (tại Step 5): 5.1. Admin phát hiện dữ liệu cảm biến không khớp với thực tế (ví dụ: cảm biến báo trống nhưng thực tế có xe). 5.2. Admin chọn “Ghi chú trạng thái thực tế” và nhập mô tả tình trạng. 5.3. Parking Service lưu ghi chú vào Cơ sở Dữ liệu kèm thông tin người thực hiện và thời điểm. Audit Logger ghi nhận hành động (adminId, mã chỗ đỗ, hành động ghi chú, timestamp). Giao diện Quản trị hiển thị icon cảnh báo trên chỗ đỗ đó trong sơ đồ. 5.4. UC kết thúc thành công.
Post conditions	1. Tình trạng chiếm dụng bãi đỗ xe được hiển thị thành công, phản ánh dữ liệu cảm biến mới nhất mà hệ thống nhận được. 2. Nếu Admin thực hiện AF2: ghi chú trạng thái thực tế được lưu trong Cơ sở Dữ liệu và Audit Logger ghi nhật ký hành động đầy đủ. 3. Dữ liệu cảm biến không bị thay đổi bởi bất kỳ thao tác nào trong UC này.

Use-case ID	U5.6
Use-case name	Giám sát tình trạng bãi đỗ xe
Exception flow	E1 (Toàn bộ cảm biến của một khu vực offline): Nếu Parking Service phát hiện tất cả cảm biến trong một khu vực đều không phản hồi, Parking Service cập nhật trạng thái khu vực đó thành “Không xác định - mất dữ liệu cảm biến” và loại trừ khu vực này khỏi phép tính tỷ lệ chiếm dụng tổng hợp để tránh sai lệch. Giao diện Quản trị hiển thị trạng thái này kèm cảnh báo rõ ràng. E2 (Dữ liệu cảm biến lỗi thời): Nếu Parking Service phát hiện thời điểm cập nhật trạng thái của một chỗ đỗ vượt quá 5 phút so với thời điểm hiện tại, Parking Service gắn nhãn “dữ liệu có thể lỗi thời” trên chỗ đỗ đó và cung cấp thời điểm cập nhật lần cuối để Admin tự đánh giá độ tin cậy.

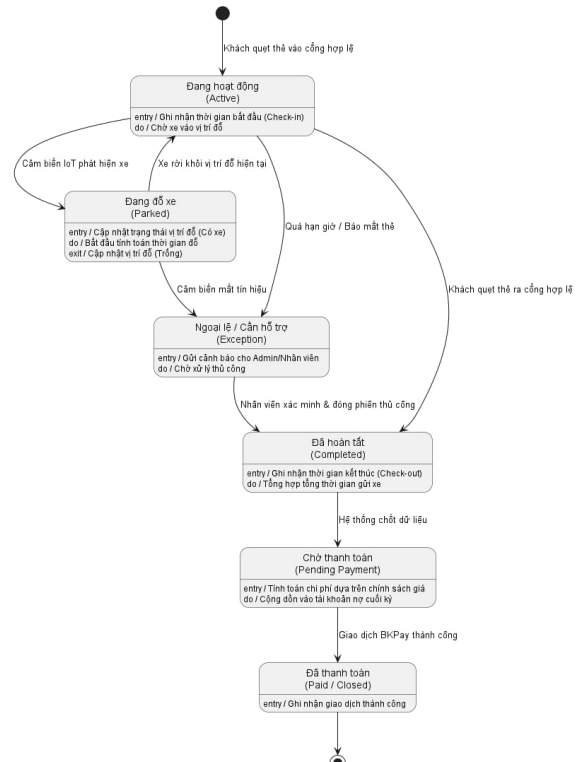
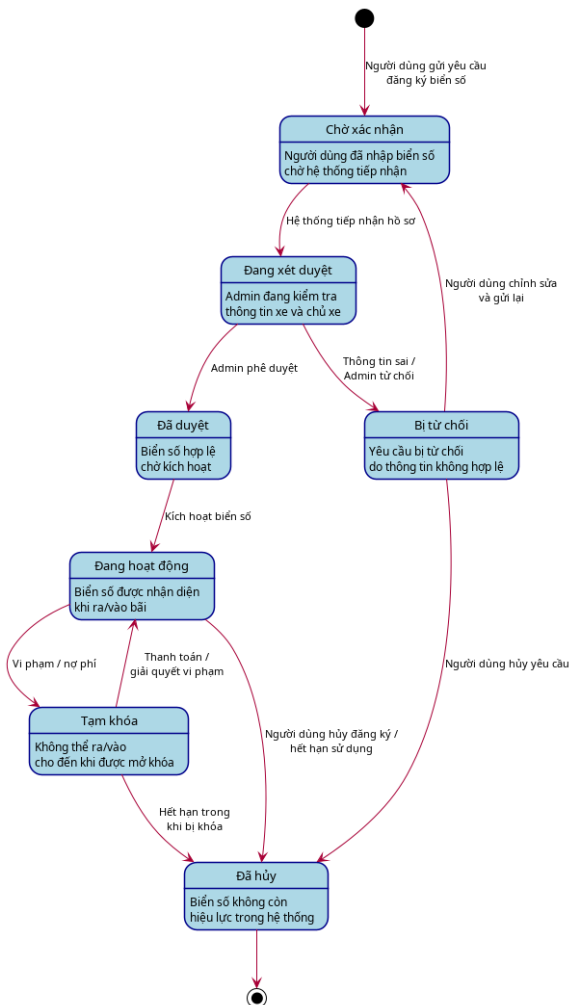


Hình 32: Sequence Diagram — UC5.6: Giám sát tình trạng bãi đỗ xe



Hình 33: Activity Diagram — UC5.6: Giám sát tình trạng bãi đỗ xe

6.6 Một số state chart cho toàn hệ thống



(a) State Chart — Phiên đăng ký biển số

(b) State Chart — Phiên gửi xe

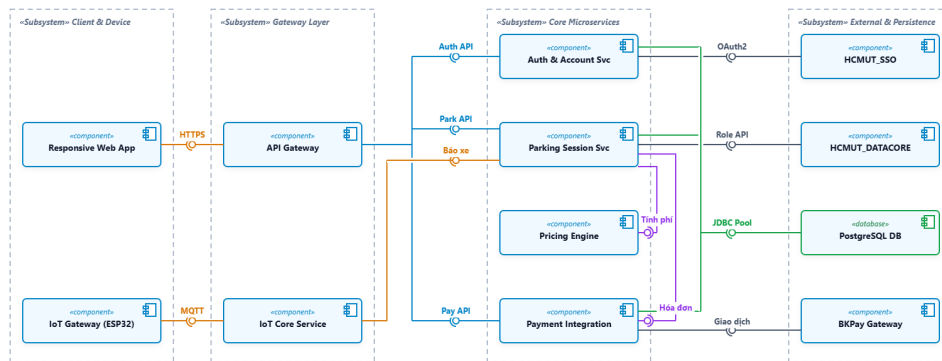
Hình 34: Các State Chart trong hệ thống

Hệ thống quản lý bãi xe thông minh được thiết kế dựa trên hai biểu đồ trạng thái chính, phân chia rõ ràng giữa khâu quản lý dữ liệu và khâu vận hành thực tế. Biểu đồ đầu tiên thể hiện quy trình quản lý vòng đời của một biển số xe trong hệ thống, bao quát các bước từ lúc người dùng nộp yêu cầu đăng ký, chờ quản trị viên xét duyệt để kích hoạt, cho đến các trạng thái phát sinh trong quá trình sử dụng như tạm khóa do vi phạm hoặc nợ phí, và cuối cùng là hủy bỏ dịch vụ. Song song đó, biểu đồ thứ hai tập trung vào luồng hoạt động cụ thể của một lượt gửi xe. Quy trình này mô tả chi tiết trình tự các sự kiện diễn ra từ thời điểm khách hàng quét thẻ vào cổng, hệ thống nhận diện xe thông qua cảm biến tại vị trí đỗ, cho đến lúc khách quét thẻ ra và hoàn tất thủ tục thanh toán để rời bãi.

6.7 Development / Implementation View

6.7.1 Component Diagram

Hệ thống được thiết kế theo kiến trúc hướng dịch vụ, đảm bảo tính đóng gói (Encapsulation) thông qua các giao diện (Interfaces) rõ ràng.



6.8 Cấu trúc mã nguồn và Tổ chức Package

Để đảm bảo tính bảo trì và khả năng mở rộng của hệ thống theo kiến trúc Microservices, mã nguồn được tổ chức dựa trên nguyên tắc thiết kế hướng miền (Domain-Driven Design - DDD). Cách tiếp cận này giúp tách biệt rõ ràng giữa logic nghiệp vụ, giao tiếp dữ liệu và cấu hình hạ tầng kỹ thuật.

6.8.1 Cấu trúc Backend Core (Spring Boot / Node.js)

Mã nguồn Backend được phân chia thành các package chức năng độc lập, giúp giảm thiểu sự phụ thuộc lẫn nhau (Low Coupling):

- `com.iot_spms.config`: Đóng vai trò là trung tâm cấu hình của hệ thống, quản lý các thiết lập về bảo mật (Security Context), cấu hình kết nối MQTT Broker và các tham số môi trường cho Database.
- `com.iot_spms.modules.auth`: Chịu trách nhiệm thực hiện quy trình định danh; tích hợp chặt chẽ với **HCMUT_SSO** để xác thực và đồng bộ thông tin người dùng từ hệ thống nhà trường.
- `com.iot_spms.modules.parking`: Chứa logic nghiệp vụ cốt lõi về quản lý phiên gửi xe, điều phối trạng thái vị trí đỗ và xử lý dữ liệu từ cảm biến.
- `com.iot_spms.modules.payment`: Chuyên biệt cho việc tính toán hóa đơn dựa trên chính sách giá và thực hiện giao tiếp với cổng **BKPay** thông qua các API thanh toán.

- `com.iot_spms.modules.iot`: Lớp xử lý trung gian cho các luồng dữ liệu thời gian thực, quản lý các kết nối WebSocket và chuyển đổi dữ liệu từ giao thức MQTT sang logic nghiệp vụ nội bộ.
- `com.iot_spms.shared`: Thư viện dùng chung chứa các đối tượng chuyển đổi dữ liệu (DTOs), các lớp Exception tùy chỉnh và các hàm tiện ích (Utilities) được sử dụng xuyên suốt các module.

6.8.2 Cấu trúc Frontend (React)

Phần giao diện được tổ chức theo kiến trúc Component-based hiện đại, giúp tối ưu hóa việc tái sử dụng mã nguồn:

- `/src/components`: Chứa các UI components như các bảng điều khiển và thành phần điều hướng.
- `/src/pages`: Tổ chức theo các luồng nghiệp vụ chính của người dùng như Dashboard quản lý, lịch sử giao dịch và Trang cá nhân.
- `/src/services`: Lớp trừu tượng hóa các lệnh gọi API (thông qua Axios), giúp tách biệt logic xử lý giao diện với logic giao tiếp mạng.
- `/src/store`: Trung tâm quản lý trạng thái toàn cục, đảm bảo dữ liệu (như trạng thái bài đồ) luôn nhất quán trên mọi màn hình.

6.8.3 Quản lý phụ thuộc và Thư viện sử dụng

.

Thành phần	Thư viện chính	Vai trò trong kiến trúc
Backend Core	Spring Security	Triển khai giao thức OAuth2, JWT và cơ chế phân quyền (RBAC) cho nhân viên và khách hàng.
	Spring Data JPA	Framework ORM mạnh mẽ giúp tương tác với PostgreSQL, tối ưu hóa hiệu năng truy vấn dữ liệu.
	Eclipse Paho	Thư viện MQTT Client chuyên dụng để duy trì kết nối ổn định và tin cậy với các thiết bị IoT Gateway.
Real-time Logic	Socket.io	Cung cấp khả năng giao tiếp hai chiều thời gian thực, đảm bảo cập nhật trạng thái vị trí đồ tức thời lên giao diện.
Frontend	Axios	Xử lý các yêu cầu HTTP/HTTPS tới API Gateway với cơ chế Interceptor để đính kèm Token bảo mật.
	Tailwind CSS	Framework CSS hiện đại giúp xây dựng giao diện Responsive linh hoạt, đáp ứng tiêu chuẩn trải nghiệm người dùng (NFR4).
IoT Gateway	PubSubClient	Giao thức truyền tin gọn nhẹ (Lightweight), tối ưu cho các vi điều khiển có tài nguyên hạn chế.

Bảng 6: Bảng tổng hợp các thư viện và công nghệ then chốt trong hệ thống IoT-SPMS.

6.9 Deployment View

6.9.1 Mô tả các thực thể và Các thành phần

- **Nhóm Thiết bị người dùng cuối:**

- **User Device (Mobile/PC):** Thiết bị của người học, cán bộ hoặc khách vãng lai. Chạy trình duyệt web để tải thành phần Frontend Application (ReactJS). Giao tiếp với hệ thống qua giao thức HTTPS.
- **Electronic Signage (Bảng LED chỉ dẫn):** Màn hình hiển thị tại các nút giao và cổng bãi xe. Nhận dữ liệu trạng thái bãi đỗ thời gian thực thông qua kết nối WebSocket.

- **Nhóm Thiết bị IoT & Phần cứng bãi xe:**

- **IoT Sensor Node:** Các cảm biến tại từng vị trí đỗ xe. Chạy firmware PubSubClient để phát hiện trạng thái (có/trống xe) và gửi tín hiệu qua sóng Wi-Fi.
- **IoT Gateway:** Thiết bị thu thập dữ liệu cục bộ từ nhiều cảm biến. Đóng gói dữ liệu và truyền về trung tâm thông qua giao thức MQTT.

- **Smart Gate:** Tích hợp đầu đọc thẻ từ (RFID/NFC) và máy phát vé tạm. Gửi yêu cầu xác thực thẻ và nhận lệnh mở barie từ hệ thống trung tâm.

- **Nhóm Máy chủ Trung tâm:**

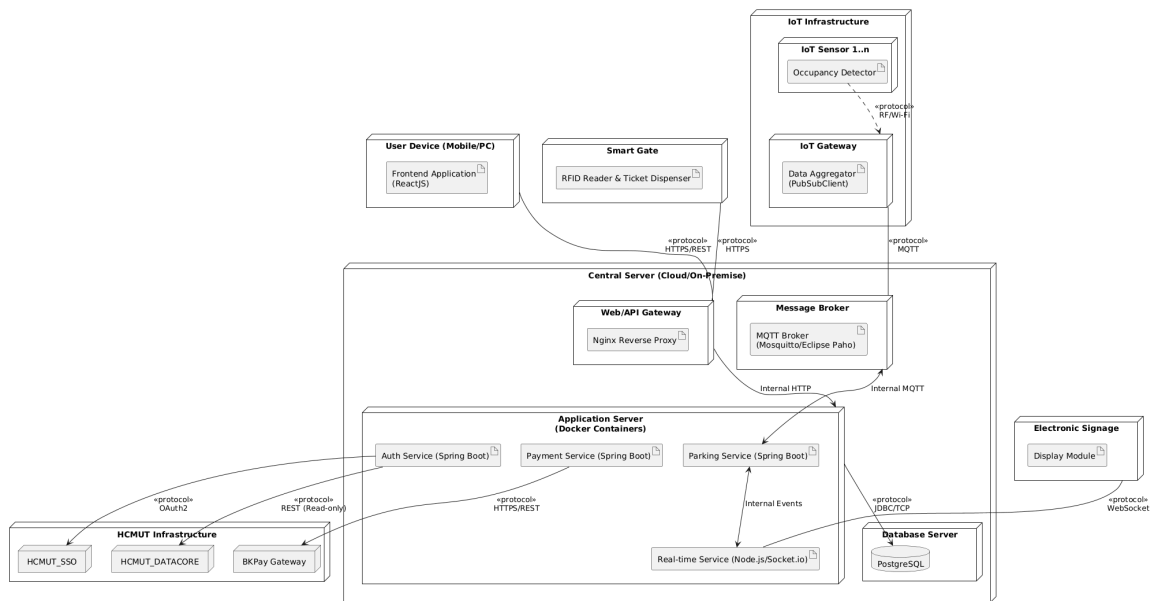
- **API Gateway / Reverse Proxy:** Điểm vào duy nhất của hệ thống, định tuyến các yêu cầu RESTful API từ Frontend đến các dịch vụ Backend tương ứng.
- **App Server:** Triển khai trên môi trường Docker/Kubernetes. Chứa các microservices viết bằng Spring Boot (Auth Service, Parking Service, Payment Service) và Node.js (Real-time Socket.io Server).
- **MQTT Broker Server:** Máy chủ chạy Eclipse Mosquitto hoặc phần mềm tương đương, chịu trách nhiệm nhận/phát dữ liệu thời gian thực giữa IoT Gateways và Backend.
- **Database Server:** Máy chủ lưu trữ dữ liệu chính thức, chạy hệ quản trị cơ sở dữ liệu PostgreSQL. Lưu trữ thông tin vé, lịch sử giao dịch và cấu hình bãi đỗ.

- **Nhóm Hệ thống Ngoại vi:**

- **HCMUT_SSO:** Cổng xác thực tập trung của trường, giao tiếp qua giao thức OAuth2/SAML.
- **HCMUT_DATACORE:** Hệ thống cơ sở dữ liệu dùng chung của trường, cung cấp API (Read-only) để đồng bộ thông tin sinh viên/cán bộ.
- **BKPay Gateway:** Cổng thanh toán nội bộ, tiếp nhận và xử lý các giao dịch tài chính từ Payment Service.

6.9.2 Giao thức giao tiếp (Communication Protocols)

- **HTTPS/RESTful API:** Giao tiếp an toàn giữa User Device/Smart Gate và API Gateway; và giữa Backend với các hệ thống ngoại vi (HCMUT_SSO, BKPay).
- **MQTT (Message Queuing Telemetry Transport):** Giao tiếp nhẹ, độ trễ thấp giữa IoT Gateways và MQTT Broker.
- **WebSocket:** Cập nhật dữ liệu thời gian thực (real-time) từ Backend tới Frontend và bảng điện tử (Electronic Signage).
- **TCP/IP & JDBC:** Giao tiếp nội bộ tốc độ cao giữa các Backend Services và Database Server.



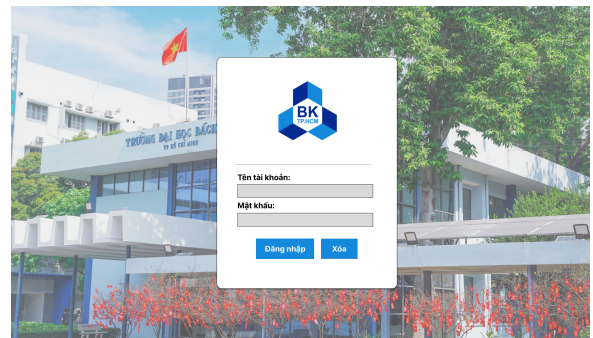
Hình 35: Deployment Diagram của hệ thống IoT-SPMS

7 UI design

7.1 Giao diện bắt đầu



(a) Giao diện đăng nhập đầu tiên



(b) Giao diện đăng nhập thứ hai

Hình 36: Ảnh chụp màn hình đăng nhập

Giao diện đăng nhập đóng vai trò là điểm chạm đầu tiên và là cửa ngõ tiếp cận toàn bộ hệ thống. Tại đây, mọi người dùng đều phải thực hiện bước xác thực danh tính bằng cách cung cấp các thông tin cơ bản bao gồm tên tài khoản và mật khẩu. Quá trình này đảm bảo tính bảo mật, đồng thời cấp quyền để người dùng có thể truy cập và sử dụng đầy đủ các tính năng mà ứng dụng web cung cấp.



Hình 37: Giao diện trang chủ web

Sau khi hoàn tất quá trình xác thực thành công, hệ thống sẽ tự động điều hướng người dùng đến Trang chủ. Đây được xem là không gian làm việc trung tâm và là điểm kết nối mọi hoạt động trên nền tảng.

7.2 Giao diện đăng ký biển số xe

(a) Giao diện nhập thông tin đăng ký biển số

(b) Giao diện chờ xét duyệt biển số

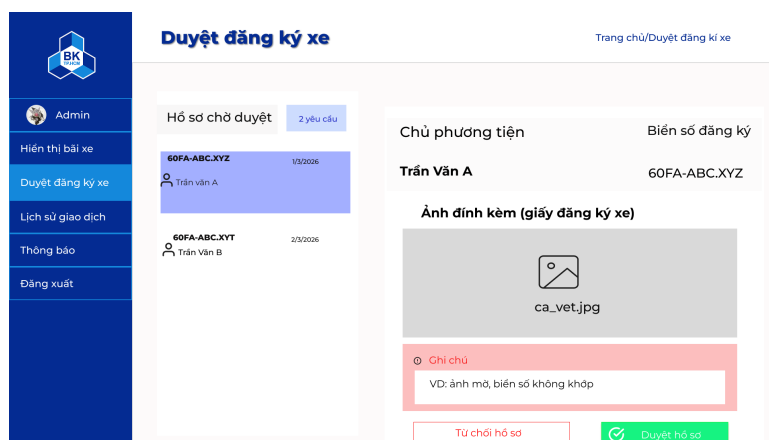
(c) Giao diện đăng ký thành công

(d) Giao diện đăng ký thất bại

Hình 38: Tổng hợp giao diện cho các bước đăng ký biển số xe

Để có thể tiếp cận và sử dụng trọn vẹn các dịch vụ tiện ích mà hệ thống cung cấp, điều quan trọng nhất là người dùng phải thực hiện đăng ký phương tiện cá nhân. Phần trên đây sẽ

trình bày chi tiết các màn hình sẽ xuất hiện trong quy trình khai báo và đăng ký biển số xe vào cơ sở dữ liệu của trang web.



Hình 39: Giao diện xét duyệt đăng ký biển số bên phía nhân viên

Ngay sau khi người dùng hoàn tất thủ tục gửi yêu cầu đăng ký, dữ liệu sẽ được hệ thống tiếp nhận và chuyển giao sang phân hệ quản trị. Tại đây, đội ngũ nhân viên sẽ được cung cấp một giao diện màn hình chuyên biệt để tiến hành quy trình kiểm tra, xác minh và xét duyệt tính hợp lệ của hồ sơ phương tiện.

7.3 Các giao diện cho các tiện ích khác của người dùng

(a) Giao diện đăng ký gói tháng

(b) Giao diện thực hiện thanh toán hóa đơn

(c) Giao diện hiện thị trạng thái bãi xe

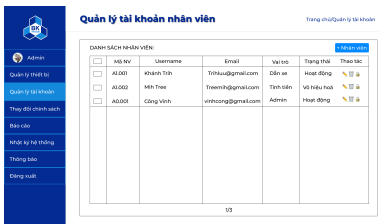
(d) Giao diện lịch sử giao dịch cá nhân

Hình 40: Tổng hợp các giao diện cho các tiện ích khác của người dùng

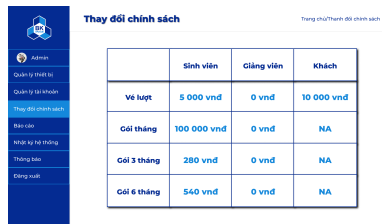
Tại nhóm giao diện này, hệ thống cung cấp các chức năng cốt lõi giúp người dùng chủ động quản lý dịch vụ và các tiện ích bãi đỗ. Cụ thể, người dùng được cấp quyền để thực hiện các thao tác nghiệp vụ sau:

- Đăng ký gói dịch vụ
- Thanh toán hóa đơn
- Tra cứu vị trí đỗ xe
- Xem lịch sử giao dịch của bản thân

7.4 Các giao diện cho Admin



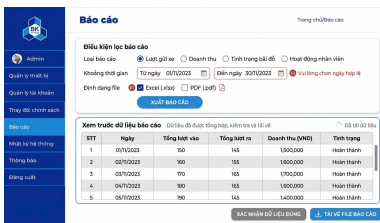
(a) Giao diện quản lý các tài khoản nhân viên



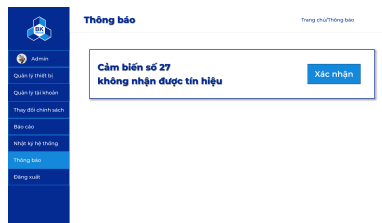
(b) Giao diện nối Admin thay đổi phí gửi xe



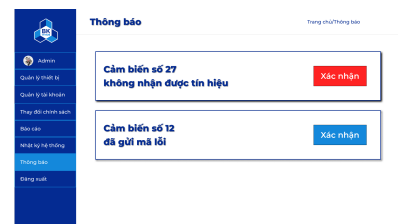
(c) Giao diện thông tin nhật ký hệ thống



(d) Giao diện Admin xuất ra các báo cáo cần thiết



(e) Giao diện thông báo khi thiết bị IoT gặp sự cố

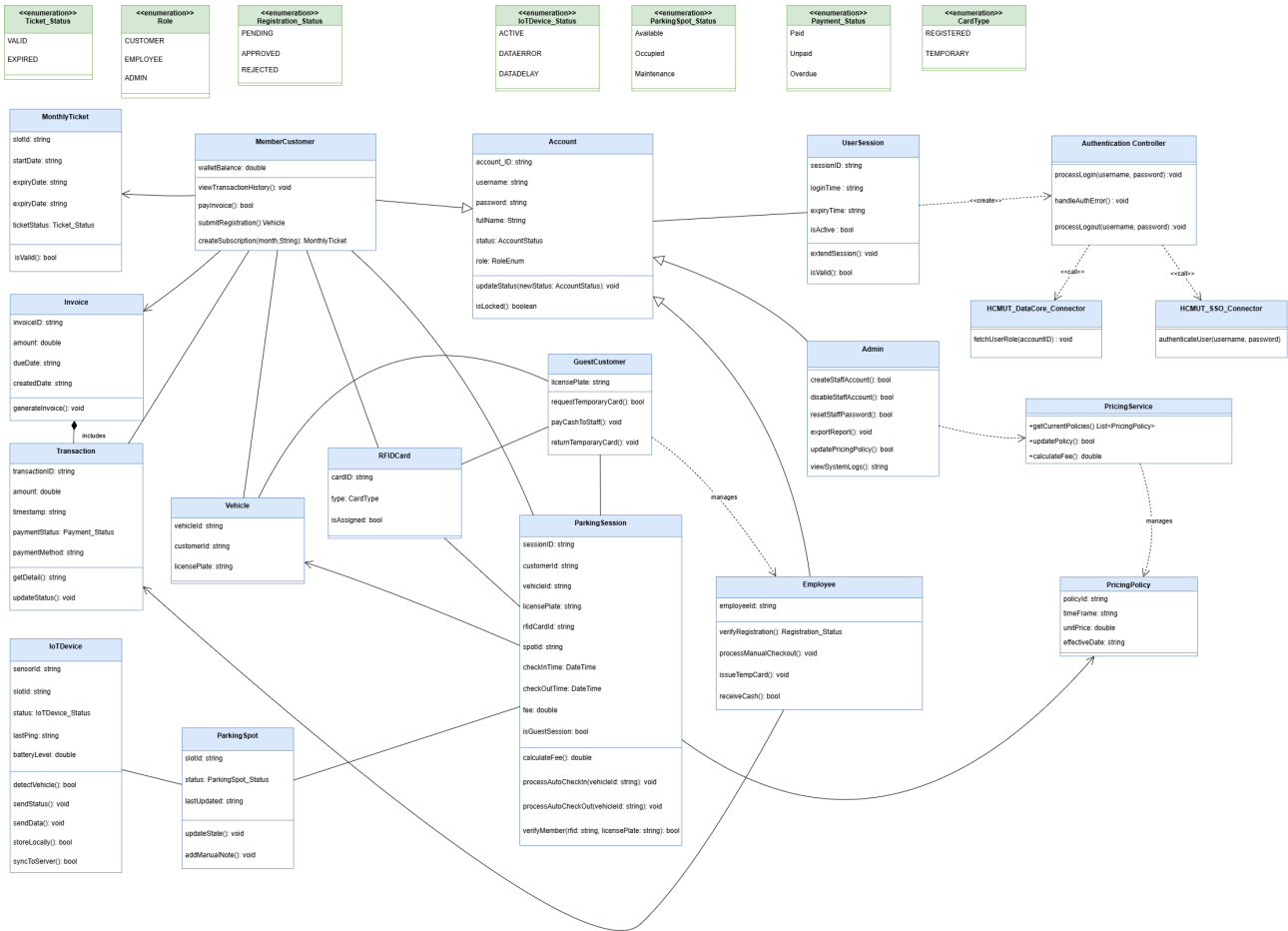


(f) Giao diện cảnh báo khi Admin chưa xác nhận các thiết bị IoT lỗi

Hình 41: Tổng hợp các giao diện cho các chức năng của Admin

Nhóm giao diện dành riêng cho Quản trị viên (Admin) được thiết kế với tiêu chí trực quan, hiện đại và tối ưu hóa trải nghiệm người dùng. Thông qua hệ thống chức năng toàn diện này, Admin có thể dễ dàng thực hiện và kiểm soát mọi hoạt động cốt lõi, từ việc quản lý tài khoản nhân viên, điều chỉnh linh hoạt các chính sách về giá phí, cho đến việc trích xuất báo cáo và theo dõi nhật ký hoạt động chi tiết. Đặc biệt, phân hệ giám sát thiết bị IoT tích hợp cơ chế cảnh báo thời gian thực và nhắc nhở tự động không chỉ giúp Admin nắm bắt ngay lập tức các sự cố phát sinh, mà còn ngăn ngừa rủi ro do sơ suất trong quá trình xử lý, đảm bảo toàn bộ quy trình vận hành luôn diễn ra an toàn, minh bạch và đạt hiệu suất cao nhất.

8 Danh sách các Method và Class Diagram



Hình 42: Class diagram cho toàn bộ hệ thống

8.1 Các Enumeration trong Class Diagram

Trong thiết kế của hệ thống, các kiểu dữ liệu liệt kê (Enumeration) được sử dụng để định nghĩa một tập hợp các hằng số cố định, giúp quản lý trạng thái, loại đối tượng và quyền hạn một cách rõ ràng, chuẩn hóa và tránh sai sót trong quá trình lập trình. Dựa vào Biểu đồ lớp (Class Diagram), hệ thống bao gồm các Enumeration sau:

- **Role** (hoặc RoleEnum): Định nghĩa vai trò và quyền hạn của các tài khoản (Account) trong hệ thống, bao gồm 3 mức:
 - CUSTOMER: Khách hàng (bao gồm khách văn lai và thành viên).
 - EMPLOYEE: Nhân viên vận hành bãi xe.
 - ADMIN: Quản trị viên hệ thống.
- **IoTDevice_Status**: Định nghĩa các trạng thái hoạt động của các thiết bị cảm biến (IoTDevice) tại bãi xe:

- ACTIVE: Thiết bị đang hoạt động bình thường.
- DATAERROR: Thiết bị gặp lỗi dữ liệu.
- DATADELAY: Thiết bị có dấu hiệu gửi dữ liệu bị trễ.
- **ParkingSpot_Status**: Thể hiện tình trạng hiện tại của từng vị trí đỗ xe (ParkingSpot):
 - Available: Vị trí đang trống, sẵn sàng để đỗ xe.
 - Occupied: Vị trí đã có xe chiếm dụng.
 - Maintenance: Vị trí đang trong quá trình bảo trì, không thể sử dụng.
- **Payment_Status**: Quản lý trạng thái thanh toán của các giao dịch (Transaction) hoặc hóa đơn (Invoice):
 - Paid: Đã thanh toán thành công.
 - Unpaid: Chưa thanh toán.
 - Overdue: Quá hạn thanh toán.
- **Registration_Status**: Trạng thái của các yêu cầu đăng ký phương tiện (Vehicle) từ phía khách hàng thành viên:
 - PENDING: Đang chờ nhân viên/hệ thống phê duyệt.
 - APPROVED: Đã được chấp thuận.
 - REJECTED: Bị từ chối đăng ký.
- **Ticket_Status**: Thể hiện trạng thái hiệu lực của vé tháng (MonthlyTicket):
 - VALID: Vé đang còn hiệu lực sử dụng.
 - EXPIRED: Vé đã hết hạn.
- **CardType**: Phân loại các thẻ RFID (RFIDCard) được sử dụng để ra/vào bãi xe:
 - REGISTERED: Thẻ đã được đăng ký, cấp cho khách hàng thành viên (sinh viên, cán bộ).
 - TEMPORARY: Thẻ tạm thời, cấp cho khách vãng lai khi vào bãi và thu hồi lại khi ra.

8.2 Các Class trong Class Diagram

8.2.1 Lớp thực thể: Account (Tài khoản)

Lớp này lưu trữ thông tin định danh và trạng thái của người dùng được đồng bộ từ hệ thống nhà trường.

- **Thuộc tính:**

- accountID: Mã định danh duy nhất của tài khoản (String).
- username: Tên đăng nhập của người dùng (String).
- password: Mật khẩu của tài khoản người dùng (String).
- fullName: Họ và tên đầy đủ của người dùng (String)
- role: Vai trò của người dùng (CUSTOMER, ADMIN, EMPLOYEE) để phân quyền và áp dụng chính sách giá.
- status: Trạng thái tài khoản (ACTIVE, DISABLED).

- **Phương thức:**

- updateStatus(newStatus): Cập nhật trạng thái mới cho tài khoản (ví dụ: khóa do nợ phí).
- **isLocked(): boolean (Class Account)**
 - * Mục đích: Kiểm tra trạng thái hoạt động của tài khoản khách hàng (xem có đang bị khóa hay không).
 - * Mô tả logic: Phương thức này truy xuất giá trị của thuộc tính isLocked trong cơ sở dữ liệu của người dùng. Hệ thống sẽ gọi hàm này ở các bước kiểm tra điều kiện (Ví dụ: trước khi cho phép xe vào bãi, hoặc ngay khi khách hàng đăng nhập) để xem tài khoản có đang bị khóa do nợ quá hạn hay không.

8.2.2 Lớp thực thể: UserSession (Phiên làm việc)

Quản lý trạng thái và thời gian duy trì đăng nhập của người dùng sau khi xác thực thành công.

- **Thuộc tính:**

- sessionID: Mã phiên làm việc duy nhất để bảo mật (String).
- loginTime: Thời điểm bắt đầu đăng nhập (Time).
- expiryTime: Thời điểm phiên làm việc hết hạn (Time).
- isActive: Trạng thái hoạt động của phiên (Boolean).

- **Phương thức:**

- isValid(): Kiểm tra phiên làm việc hiện tại còn hiệu lực hay không.
- extendSession(): void: Gia hạn thời gian hoạt động của phiên làm việc.

8.2.3 Lớp điều khiển: **AuthenticationController**

Đóng vai trò điều phối luồng đăng nhập, xử lý logic xác thực và quản lý vòng đời của session.

- **Phương thức:**

- `processLogin(username, password)`: Tiếp nhận thông tin, gọi các dịch vụ xác thực và khởi tạo phiên làm việc.
- `handleAuthError()`: Xử lý các tình huống lỗi đăng nhập (sai mật khẩu, lỗi kết nối hệ thống ngoài).
- `processLogout(sessionID)`: Thực hiện quy trình hủy phiên làm việc khi người dùng đăng xuất.

8.2.4 Lớp biên: **HCMUTSSOConnector** và **HCMUTDataCoreConnector**

Các lớp giao tiếp với hạ tầng công nghệ hiện có của HCMUT.

- **HCMUTSSOConnector:**

- `authenticateUser(user, pass)`: Gửi yêu cầu xác thực tới máy chủ HCMUT_SSO và nhận mã xác thực thành công hoặc lỗi.

- **HCMUTDataCoreConnector:**

- `fetchUserRole(accountID)`: Truy vấn vai trò người dùng từ DATACORE ở chế độ chỉ đọc để đồng bộ dữ liệu.

8.2.5 Lớp thực thể: **ParkingSession** (Phiên gửi xe)

Lớp này lưu trữ thông tin chi tiết về một phiên gửi xe (từ lúc vào đến lúc ra) của khách hàng.

- **Thuộc tính:**

- `sessionID`: Mã định danh duy nhất của phiên gửi xe (String).
- `customerID`: Mã khách hàng liên kết với phiên gửi (String).
- `vehicleID`: Mã phương tiện liên kết với phiên gửi (String).
- `licensePlate`: Biển số xe dùng để đối soát (String).
- `rfidCardID`: Mã thẻ từ được sử dụng (String).
- `spotID`: Mã vị trí đỗ xe (String).
- `checkInTime`: Thời điểm phương tiện vào bãi (DateTime).

- `checkOutTime`: Thời điểm phương tiện ra khỏi bãi (`DateTime`).
- `fee`: Tổng số tiền phí gửi xe của phiên (`Double`).
- `isGuestSession`: Trạng thái xác định đây là phiên của khách vắng lai (`Boolean`).

- **Phương thức:**

- `verifyMember(rfid, licensePlate)`: Xác thực quyền ra/vào tự động cho thẻ hoặc biển số của khách thành viên.
- `processAutoCheckIn(vehicleId)`: Xử lý logic tạo phiên gửi xe tự động khi hệ thống nhận diện xe vào cổng.
- `processAutoCheckOut(vehicleId)`: Xử lý logic đóng phiên gửi xe tự động khi xe ra khỏi bãi.
- **`calculateFee()`: `double` (Class `ParkingSession`)**
 - * Mục đích: Tính toán chính xác số tiền phí mà khách hàng phải trả cho một phiên gửi xe.
 - * Mô tả logic: Tính toán số tiền (dưới dạng số thực `double`) dựa trên sự chênh lệch giữa `checkInTime` và `checkOutTime`, đối chiếu với `PricingPolicy` (bảng giá).

8.2.6 Lớp thực thể: `ParkingSpot` (Vị trí đỗ xe)

Lớp đại diện cho một ô đỗ xe vật lý trong bãi.

- **Thuộc tính:**

- `spotID`: Mã định danh duy nhất của ô đỗ xe (`String`).
- `status`: Trạng thái hiện tại của ô đỗ (`Enum ParkingSpot_Status`: `Available`, `Occupied`, `Maintenance`).
- `lastUpdate`: Thời điểm cập nhật trạng thái gần nhất (`String`).

- **Phương thức:**

- `updateStatus()`: Cập nhật trạng thái ô đỗ thành có xe hoặc trống dựa trên tín hiệu từ cảm biến.
- `checkAvailability()`: Kiểm tra trạng thái hiện tại xem ô đỗ có sẵn sàng tiếp nhận xe hay không.

8.2.7 Lớp thực thể: `Vehicle` (Phương tiện)

Lưu trữ thông tin về phương tiện đã được đăng ký trên hệ thống.

- **Thuộc tính:**

- vehicleID: Mã định danh của phương tiện (String).
- customerID: Mã khách hàng sở hữu phương tiện (String).
- licensePlate: Biển số xe thực tế (String).

8.2.8 Lớp thực thể: RFIDCard (Thẻ từ)

Quản lý danh sách thẻ từ dùng để quét ra/vào cổng.

- **Thuộc tính:**

- cardID: Mã định danh vật lý của thẻ (String).
- type: Loại thẻ định danh (Enum CardType).
- isAssigned: Trạng thái xác định thẻ đã được cấp cho ai hay chưa (Boolean).

8.2.9 Lớp điều khiển: Employee (Nhân viên vận hành)

Đại diện cho tác nhân nhân viên, xử lý các nghiệp vụ thủ công và quản lý phê duyệt.

- **Thuộc tính:**

- employeeID: Mã định danh của nhân viên (String).

- **Phương thức:**

- issueTemporaryCard(): Cấp thẻ tạm, tạo phiên gửi xe mới và mở barie cho khách vắng lai.
- confirmManualCheckout(): Xác nhận thu hồi thẻ, kết thúc phiên gửi xe và mở cổng ra.
- receiveCash(): Cập nhật trạng thái đã nhận tiền mặt từ khách hàng.
- verifyRegistration(reg): Phê duyệt hoặc từ chối hồ sơ đăng ký xe của khách thành viên.

8.2.10 Lớp điều khiển: GuestCustomer (Khách vắng lai)

Đại diện cho đối tượng khách không có tài khoản trên hệ thống.

- **Thuộc tính:**

- licensePlate: Biển số xe tạm thời của khách (String).

- **Phương thức:**

- requestTemporaryCard(): Yêu cầu cấp thẻ tạm tại cổng khi vào bãi.
- returnTemporaryCard(): Trả lại thẻ tạm tại cổng để thanh toán khi ra bãi.
- payCashToStaff(): Thực hiện thanh toán phí gửi xe bằng tiền mặt.

8.2.11 Lớp điều khiển: MemberCustomer (Khách thành viên)

Đại diện cho người dùng đã đăng ký tài khoản (Sinh viên, Cán bộ).

- **Thuộc tính:**

- walletBalance: Số dư hiện tại trong ví nội bộ (Double).

- **Phương thức:**

- submitRegistration(): Gửi yêu cầu đăng ký phương tiện mới lên hệ thống, trả về đối tượng phương tiện chờ duyệt (Vehicle).
- createSubscription(month, String): Khởi tạo yêu cầu đăng ký hoặc gia hạn gói gửi xe theo tháng, trả về đơn hàng thanh toán (MonthlyTicket).
- **viewTransactionHistory(): void (Class MemberCustomer)**
 - * Mục đích: Kích hoạt luồng xem lịch sử giao dịch cá nhân của khách hàng có tài khoản. (UC 3.1)
 - * Mô tả logic: Phương thức này truy xuất danh sách các đối tượng Transaction liên kết với MemberCustomer hiện tại từ cơ sở dữ liệu. Sau khi lấy được dữ liệu, nó điều khiển giao diện (View/UI) hiển thị danh sách này ra màn hình, bao gồm các thông tin theo Use-case yêu cầu (Thời gian, Số tiền, Trạng thái thanh toán) và cho phép người dùng thực hiện thao tác sắp xếp.
- **payInvoice(): void (Class MemberCustomer)**
 - * Mục đích: Xử lý thao tác thanh toán một hóa đơn điện tử thông qua cổng thanh toán. (UC 3.2)
 - * Mô tả logic: Phương thức này tiếp nhận lệnh “Chuyển đến BKPay” từ người dùng và gọi đến API của cổng thanh toán BKPay. Nó chờ Webhook trả về và xử lý kết quả.

8.2.12 Lớp Invoice

Đại diện cho người dùng đã đăng ký tài khoản (Sinh viên, Cán bộ).

- **Thuộc tính:**

- invoiceID: string: Mã định danh duy nhất của hóa đơn.
- amount: double: Số tiền cước phí cần thanh toán.
- dueDate: string: Ngày đến hạn thanh toán hóa đơn.
- createdAt: string: Ngày giờ hệ thống khởi tạo hóa đơn.

- **Phương thức:**

– **generateInvoice(): void (Class Invoice)**

- * Mục đích: Khởi tạo thông tin chi tiết cho một hóa đơn mới. (UC 3.2)
- * Mô tả logic: Dựa vào phí gửi xe đã được tính toán từ các phiên gửi xe (ParkingSession), hàm này khởi tạo đối tượng Invoice, gán invoiceID, thiết lập số tiền amount, ngày tạo createdAt, và đặc biệt là hạn thanh toán dueDate. Dữ liệu này là tiền đề để hiển thị danh sách hóa đơn cho MemberCustomer thanh toán.

8.2.13 Lớp Transaction

Đại diện cho người dùng đã đăng ký tài khoản (Sinh viên, Cán bộ).

• **Thuộc tính:**

- transactionID: string: Mã định danh duy nhất của giao dịch.
- amount: double: Số tiền thực hiện trong giao dịch.
- timestamp: string: Thời gian (ngày và giờ) hệ thống ghi nhận giao dịch.
- paymentStatus: Payment_Status: Trạng thái thanh toán của giao dịch (sử dụng Enumeration Payment_Status như Paid, Unpaid, Overdue).
- paymentMethod: string: Phương thức thanh toán được khách hàng sử dụng (ví dụ: Tiền mặt, chuyển khoản qua BKPay).

• **Phương thức:**

– **getDetail(): string**

- * Mục đích: Đóng gói và trả về chuỗi thông tin chi tiết của một giao dịch.
- * Mô tả logic: Phương thức này định dạng các thuộc tính của Transaction thành một chuỗi có cấu trúc dễ đọc. Chuỗi này được sử dụng để hiển thị trong danh sách lịch sử giao dịch của khách hàng.

– **updateStatus(): void**

- * Mục đích: Thay đổi trạng thái thanh toán của giao dịch.
- * Mô tả logic: Truy xuất các thuộc tính của lớp Transaction (như transactionID, amount, timestamp, paymentStatus, paymentMethod) và định dạng chúng thành một chuỗi (String) hoặc chuỗi JSON có cấu trúc.
 - Trong U3.2: Được gọi sau khi payInvoice() nhận kết quả trả về từ BKPay để đổi trạng thái từ “Unpaid” sang “Paid” hoặc “Overdue” nếu quá hạn.
 - Trong U3.3: Được gọi khi Nhân viên bấm "Xác nhận đã nhận tiền" để đổi trạng thái giao dịch sang "Đã thanh toán"(Success) rồi lưu xuống cơ sở dữ liệu.

8.2.14 Lớp IoTDevice

- **Thuộc tính:**

- `sensorId`: string: Mã định danh duy nhất của thiết bị cảm biến.
- `slotId`: string: Mã định danh của vị trí đỗ xe mà cảm biến này đang phụ trách giám sát.
- `status`: `IoTDevice_Status`: Trạng thái hoạt động và kết nối hiện tại của thiết bị.
- `lastPing`: string: Mốc thời gian hệ thống nhận được tín hiệu phản hồi cuối cùng từ thiết bị.
- `batteryLevel`: double: Mức dung lượng pin hiện tại của thiết bị phần cứng.

- **Phương thức:**

- `detectVehicle()`: bool
 - * *Mục đích*: Phát hiện sự hiện diện của phương tiện tại vị trí đỗ xe theo thời gian thực.
 - * *Mô tả logic*: Thiết bị sử dụng cảm biến (siêu âm, hồng ngoại hoặc từ trường) để quét không gian tại vị trí đỗ. Nếu có vật thể che chắn hoặc thỏa mãn điều kiện về khoảng cách và thời gian duy trì, hệ thống vi điều khiển sẽ đánh giá là có xe và trả về true. Ngược lại, nếu không có vật cản, phương thức trả về false.
- `sendStatus()`: void
 - * *Mục đích*: Duy trì tín hiệu Heartbeat để hệ thống trung tâm có thể giám sát tình trạng sức khỏe của thiết bị.
 - * *Mô tả logic*: Theo một chu kỳ thời gian cố định, thiết bị tự động đóng gói các thông tin cơ bản bao gồm `sensorId`, `status` và `batteryLevel`. Gói tin này được gửi đến máy chủ. Nếu máy chủ không nhận được gói tin này vượt quá thời gian timeout cho phép, thiết bị sẽ bị đánh dấu là mất kết nối trên hệ thống.
- `sendData()`: void
 - * *Mục đích*: Truyền tải dữ liệu thay đổi trạng thái của vị trí đỗ xe (có xe/không có xe) lên hệ thống trung tâm ngay khi có sự kiện xảy ra.
 - * *Mô tả logic*: Ngay khi hàm `detectVehicle()` ghi nhận sự thay đổi trạng thái (từ trống sang có xe hoặc ngược lại), phương thức này được kích hoạt. Nó đóng gói mã `slotId` cùng trạng thái mới nhất và truyền qua mạng (MQTT/HTTP) lên máy chủ. Máy chủ sau đó sẽ tiếp nhận và tiến hành cập nhật cơ sở dữ liệu.
- `storeLocally()`: bool
 - * *Mục đích*: Xử lý sự cố mất mạng tạm thời, đảm bảo không bị mất mát dữ liệu trạng thái của bãi xe.

- * *Mô tả logic*: Khi `sendData()` phát hiện không thể thiết lập kết nối mạng với máy chủ, phương thức này tự động được gọi. Nó tiến hành lưu trữ gói tin dữ liệu trạng thái kèm theo mốc thời gian (timestamp) vào bộ nhớ cục bộ (như thẻ nhớ SD hoặc EEPROM) trên thiết bị. Trả về `true` nếu lưu thành công.
- `syncToServer()`: `bool`
 - * *Mục đích*: Đồng bộ hóa dữ liệu bị dồn đọng khi thiết bị có kết nối mạng trở lại.
 - * *Mô tả logic*: Khi thiết bị phát hiện kết nối mạng đã được khôi phục (ping thành công tới máy chủ), phương thức này tiến hành quét bộ nhớ cục bộ. Nó trích xuất các gói tin đã được lưu bởi `storeLocally()` và đẩy tuần tự lên hệ thống trung tâm. Sau khi máy chủ gửi tín hiệu xác nhận (ACK) đã nhận đủ, phương thức này sẽ xóa dữ liệu cục bộ để giải phóng bộ nhớ và trả về `true`.

8.2.15 Lớp ParkingSpot (Vị trí đỗ xe)

Lớp thực thể đại diện cho một ô đỗ xe vật lý trong bãi đỗ, nơi được liên kết trực tiếp với dữ liệu giám sát từ IoTDevice.

- **Thuộc tính:**

- `slotId`: `string`: Mã định danh duy nhất của ô đỗ xe.
- `status`: `ParkingSpot_Status`: Trạng thái hiện tại của ô đỗ.
- `lastUpdated`: `string`: Mốc thời gian cập nhật trạng thái vị trí đỗ gần nhất.

- **Phương thức:**

- `updateState()`: `void`
 - * *Mục đích*: Cập nhật thông tin mới nhất về tình trạng chiếm dụng của vị trí đỗ xe trên cơ sở dữ liệu trung tâm.
 - * *Mô tả logic*: Phương thức này (nằm ở phía Server/Hệ thống) được kích hoạt khi nhận được gói tin dữ liệu hợp lệ từ thiết bị IoT gửi thông qua hàm `sendData()`. Hệ thống tiến hành giải mã, truy vấn vị trí đỗ có `slotId` tương ứng và ghi đè trạng thái mới (Available hoặc Occupied) cùng với thời gian `lastUpdated` vào cơ sở dữ liệu. Giao diện app/web sẽ lấy dữ liệu mới này để render lại sơ đồ bãi xe.
- `addManualNote()`: `void`
 - * *Mục đích*: Cho phép quản trị viên hoặc nhân viên bãi xe can thiệp và ghi chú thủ công về tình trạng bất thường của vị trí đỗ.
 - * *Mô tả logic*: Khi có sự cố ngoại lệ (ví dụ: cảm biến hỏng, vị trí cần dọn dẹp vệ sinh, hoặc được khách VIP đặt trước), nhân viên thao tác trên giao diện phần mềm quản lý để gọi phương thức này. Hệ thống sẽ thay đổi status của vị trí đỗ

sang "Maintenance" hoặc trạng thái tương ứng, đồng thời lưu trữ nội dung ghi chú. Lúc này, người dùng thông thường sẽ không thể đặt hoặc xem vị trí này là chỗ trống trên ứng dụng.

8.2.16 Lớp: Admin

Chức năng và ý nghĩa: Lớp đại diện cho Quản trị viên của hệ thống bãi đỗ xe. Lớp này có quyền hạn cao nhất, chịu trách nhiệm quản lý nhân sự (nhân viên vận hành), thiết lập cấu hình hệ thống, điều chỉnh chính sách giá cả và theo dõi giám sát các hoạt động chung.

- **Phương thức:**

- `createStaffAccount()`: bool: Khởi tạo tài khoản đăng nhập mới cấp cho nhân viên vận hành bãi xe.
- `disableStaffAccount()`: bool: Vô hiệu hóa tài khoản của nhân viên khi cần thiết (ví dụ: khi nhân viên nghỉ việc hoặc vi phạm).
- `resetStaffPassword()`: bool: Cấp lại mật khẩu mới cho tài khoản của nhân viên.
- `exportReport()`: void: Xuất các báo cáo thống kê liên quan đến doanh thu, lưu lượng xe, và hoạt động của bãi đỗ.
- `updatePricingPolicy()`: bool: Thực hiện thao tác cập nhật hoặc thay đổi các chính sách tính phí gửi xe.
- `viewSystemLogs()`: string: Xem lại nhật ký hoạt động (audit logs) của toàn hệ thống nhằm mục đích truy vết và kiểm tra.

8.2.17 Lớp: PricingService

Chức năng và ý nghĩa: Đây là một Service class (lớp dịch vụ) chịu trách nhiệm cung cấp và xử lý các logic nghiệp vụ liên quan đến giá cả và chi phí gửi xe. Lớp này đóng vai trò trung gian để lấy dữ liệu chính sách và tính toán mức phí cụ thể cho từng lượt xe.

- **Phương thức:**

- `getCurrentPolicies()`: List<PricingPolicy>: Lấy danh sách toàn bộ các chính sách giá (PricingPolicy) đang có hiệu lực trong hệ thống.
- `updatePolicy()`: bool: Xử lý logic lưu trữ và áp dụng khi Quản trị viên yêu cầu cập nhật một chính sách giá mới.
- `calculateFee()`: double: Thực hiện tính toán chi phí gửi xe thực tế cho một lượt đỗ dựa trên thời gian gửi, loại phương tiện và chính sách đang áp dụng.

8.2.18 Lớp: PricingPolicy

Chức năng và ý nghĩa: Lớp thực thể (Entity class) đại diện cho một cấu hình chính sách giá cụ thể. Lớp này lưu trữ các mức giá được áp dụng cho từng khung thời gian cụ thể theo quy định của bãi đỗ xe.

- **Thuộc tính:**

- policyId (string): Mã định danh duy nhất của chính sách giá.
- timeFrame (string): Khung thời gian áp dụng mức giá này (ví dụ: giờ hành chính, gửi qua đêm).
- unitPrice (double): Đơn giá áp dụng cho khung thời gian đã quy định.
- effectiveDate (string): Thời điểm (ngày/giờ) mà chính sách giá này chính thức bắt đầu có hiệu lực.

8.2.19 Lớp: MonthlyTicket

Chức năng và ý nghĩa: Lớp đại diện cho vé tháng (gói gửi xe theo tháng) mà người dùng nội bộ đã đăng ký. Việc quản lý lớp này giúp hệ thống xác định nhanh quyền lợi vào ra bãi xe tự động của khách hàng mà không cần tính phí theo từng lượt.

- **Thuộc tính:**

- slotId (string): Mã định danh vị trí đỗ xe cố định (nếu vé tháng của khách hàng có đi kèm vị trí được chỉ định riêng).
- startDate (string): Ngày bắt đầu có hiệu lực của vé tháng.
- expiryDate (string): Ngày hết hạn của vé tháng (Lưu ý: trong biểu đồ của bạn thuộc tính này đang bị lặp lại 2 lần, bạn nên xóa bớt 1 dòng cho chuẩn xác).
- ticketStatus (Ticket_Status): Trạng thái hiện tại của vé tháng (ví dụ: đang kích hoạt, đã hết hạn, bị tạm khóa).

- **Phương thức:**

- isValid(): bool: Hàm kiểm tra xem vé tháng hiện tại có còn hiệu lực để sử dụng qua trạm kiểm soát hay không (thường dựa trên việc so sánh ngày hiện tại với expiryDate và kiểm tra ticketStatus).

9 Testcase

9.1 U3.1 – Xem lịch sử giao dịch cá nhân

Class: MemberCustomer, Transaction

Method: viewTransactionHistory(), getDetail()

Test ID	Description	Input	Expected Output	Test Type
T1	Xem lịch sử khi có giao dịch	MemberCustomer đã đăng nhập; DB có 3 Transaction liên kết	Hiển thị danh sách 3 giao dịch gồm: timestamp, amount, paymentStatus	Black-box
T2	Xem lịch sử khi chưa có giao dịch (E1)	MemberCustomer hợp lệ; DB không có Transaction nào	Hệ thống hiển thị danh sách trống, thông báo “Không có giao dịch nào”	Black-box
T3	Sắp xếp danh sách theo Số tiền	Danh sách đang hiển thị; người dùng chọn sắp xếp theo “Số tiền”	Danh sách sắp xếp theo amount giảm dần	Black-box
T4	getDetail() trả về chuỗi đầy đủ	Transaction hợp lệ: transactionID=“TX001”, amount=50000, timestamp=“2024-05-01 08:00”, paymentStatus=Paid, paymentMethod=“BKPay”	Trả về chuỗi: “TX001 50,000 VND 2024-05-01 08:00 Paid BKPay”	Unit
T5	getDetail() khi có thuộc tính null	Transaction có paymentMethod=null	Trả về chuỗi với trường paymentMethod hiển thị “N/A”, không ném exception	Unit

9.2 U3.2 – Thanh toán hóa đơn

Class: MemberCustomer, Invoice, Transaction**Method: payInvoice(), generateInvoice(), updateStatus()**

Test ID	Description	Input	Expected Output	Test Type
T6	Thanh toán thành công qua BKPay	MemberCustomer chọn Invoice hợp lệ; BKPay phản hồi SUCCESS	updateStatus() cập nhật paymentStatus=Paid; hiển thị “Giao dịch thành công”	Black-box

Test ID	Description	Input	Expected Output	Test Type
T7	BKPay phản hồi thất bại	MemberCustomer chọn Invoice hợp lệ; BKPay phản hồi FAILED	updateStatus() cập nhật paymentStatus=Unpaid; hiển thị thông báo “Thanh toán thất bại”	Black-box
T8	generateInvoice() tạo hóa đơn đúng	ParkingSession: checkInTime=“08:00”, checkOutTime=“10:30”; pricePerHour=10,000 VND	Invoice tạo với amount=25,000 VND; dueDate=createdDate+24h	Unit
T9	updateStatus() đổi paymentStatus thành Paid	paymentStatus=Unpaid; BKPay trả SUCCESS; gọi updateStatus()	paymentStatus chuyển thành Paid; lưu vào DB	Unit
T10	Tài khoản bị khóa vẫn được thanh toán (E1)	walletBalance đủ; tài khoản bị khóa do quá hạn	Hệ thống cho phép thanh toán qua BKPay; sau khi thành công, tài khoản được mở khóa	Black-box

9.3 U3.3 – Thanh toán thủ công

Class: Employee, Invoice, Transaction

Method: generateInvoice(), updateStatus()

Test ID	Description	Input	Expected Output	Test Type
T11	Quét thẻ tạm hợp lệ tại cổng ra	Khách dùng thẻ tạm hợp lệ; Invoice tương ứng tồn tại trong DB	Hệ thống hiển thị amount cho cả khách và nhân viên	Black-box
T12	Không tìm thấy giao dịch gửi xe (E1)	Thẻ tạm quét vào nhưng DB không tìm thấy Invoice tương ứng	Hệ thống báo lỗi; nhân viên xác minh bằng biển số, tạo Invoice thủ công qua generateInvoice()	Black-box

Test ID	Description	Input	Expected Output	Test Type
T13	Khách mất thẻ (E1 – mất thẻ)	Khách không có thẻ; nhân viên nhập biển số xe thủ công	generateInvoice() tạo Invoice với amount cộng thêm phí phạt mất thẻ	Black-box
T14	Nhân viên xác nhận đã nhận tiền	Khách đã trả tiền mặt; nhân viên bấm “Xác nhận đã nhận tiền”	Transaction.updateStatus() đổi paymentStatus thành Paid; lưu DB; barie mở	Black-box

9.4 U2.1 — Xác nhận và ghi nhận ra vào tự động

Class: ParkingSession, RFIDCard, Vehicle, ParkingSpot, IoTDevice **Method:** processAutoCheckIn(), processAutoCheckOut(),

Test ID	Description	Input	Expected Output	Test Type
T1	Check-in tự động thành công	RFIDCard hợp lệ, isAssigned=true; licensePlate khớp với Vehicle đã đăng ký; biển số xác nhận qua camera	verifyMember() trả về true; ParkingSession tạo với checkInTime=now, isGuestSession=false; barie mở	Black-box
T2	Thẻ RFID hết hạn hoặc bị khóa (E1)	RFIDCard quét tại cổng nhưng isAssigned=false hoặc thẻ không hợp lệ	verifyMember() trả về false; hệ thống hiển thị lỗi “Thẻ không hợp lệ”; barie không mở	Black-box
T3	Biển số không khớp với đăng ký (E2)	RFIDCard hợp lệ nhưng licensePlate camera đọc được không khớp với Vehicle trong DB	verifyMember() trả về false; hệ thống hiển thị lỗi “Biển số không khớp”; barie không mở	Black-box
T4	Check-out tự động thành công	ParkingSession đang mở với vehicleId hợp lệ; xe quét thẻ tại cổng ra	processAutoCheckOut() ghi checkOutTime=now; calculateFee() tính phí; phiên đóng; barie mở	Black-box

Test ID	Description	Input	Expected Output	Test Type
T5	Khách có gói tháng còn hiệu lực (A2)	MemberCustomer có MonthlyTicket còn hiệu lực; xe vào bãi	processAutoCheckIn() tạo ParkingSession; hệ thống bỏ qua bước tính phí; barie mở	Black-box
T6	Khách không có gói tháng, cộng dồn phí (A1)	MemberCustomer không có MonthlyTicket; xe vào bãi	processAutoCheckIn() tạo ParkingSession; hệ thống ghi nhận để tính phí khi ra	Black-box
T7	verifyMember() xác thực đúng	rfid="CARD001", licensePlate="51A-123.45"; cả hai khớp với Vehicle trong DB	verifyMember() trả về true	Unit
T8	ParkingSpot cập nhật trạng thái sau check-in	IoTDevice phát hiện xe vào ô đỗ slotId="A01"; gọi updateState()	ParkingSpot.status chuyển từ Available sang Occupied; cập nhật lastUpdated=now	Unit
T9	ParkingSpot cập nhật trạng thái sau check-out	IoTDevice phát hiện xe rời ô đỗ slotId="A01"; gọi updateState()	ParkingSpot.status chuyển từ Occupied sang Available; cập nhật lastUpdated=now	Unit

Link Github:

<https://github.com/KTX-0711/Software-Engineering-Assignment-HK252>

Tài liệu

- [1] Len Bass, Paul Clements **and** Rick Kazman. *Software Architecture in Practice*. 3rd. Addison-Wesley Professional, 2012.
- [2] Rajkumar Buyya **and** Amir Vahid Dastjerdi. *Internet of Things: Principles and Paradigms*. Morgan Kaufmann, 2016.
- [3] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd. Addison-Wesley Professional, 2003.
- [4] Craig Larman. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd. Prentice Hall, 2004.
- [5] Don Norman. *The Design of Everyday Things*. Revised and expanded. Basic Books, 2013.



- [6] Roger S. Pressman **and** Bruce Maxim. *Software Engineering: A Practitioner's Approach*. 8th. New York, NY: McGraw-Hill Education, 2014.
- [7] Leonard Richardson **and** Sam Ruby. *RESTful Web Services*. O'Reilly Media, 2013.
- [8] Ian Sommerville. *Software Engineering*. 10th. Boston, MA: Pearson, 2015.